

Dual Bachelor in Data Science and Engineering and
Telecommunication Technologies Engineering
2025-2026

Bachelor Thesis

“Adjusted by Text: A Neuro-Symbolic Approach to Metadata-Enriched Time Series Forecasting”

Iker Rosales Saiz

Emilio Parrado Hernández
Leganés, May 2026



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Every time series carries meaning beyond its numbers. A name that identifies what is being measured, the units, the sampling frequency, categorical attributes or descriptive tags are pieces of context that any human analyst would use to interpret the data, and that are readily available at inference time in most real-world databases. Yet current foundation models for time series forecasting are trained almost exclusively on numerical history, discarding this textual and categorical metadata entirely. This work investigates whether closing that gap leads to measurable forecasting gains.

To answer this question, a multimodal forecasting pipeline is built on top of the FRED macroeconomic database, the largest public repository of U.S. economic indicators, where each series is paired with its full metadata. NeuroSym-CBF, the neuro-symbolic architecture developed in this work, takes both the numerical context and the T5-encoded metadata embedding as input and routes their fused representation through a TopK Sparse Autoencoder that plays the role of a concept bottleneck. Next a FiLM modulation block uses it to condition a closed-form symbolic decoder of intercept, trend and monthly seasonal components. The model is compared against exponential smoothing (AutoETS), a unimodal ablation that shares the full neuro-symbolic backbone, Chronos-bolt-base, Chronos-2 and TimesFM-200M over 194,442 series and five context lengths.

NeuroSym-CBF achieves the lowest mean MAE and the lowest 90th-percentile MAE across all five context lengths evaluated. Under the two shortest context regimes, where the autoregressive history is insufficient to fully characterise the series and the metadata signal carries the most weight, the gains over the unimodal ablation reach 7.5% in mean MAE and 7.1% in MASE at $T = 8$, and 7.6% and 7.3% respectively at $T = 10$. At the sample level, NeuroSym-CBF wins more often than it loses against every baseline across all the evaluation windows, with a win rate of 0.61 against the unimodal ablation and 0.52–0.53 against the strongest zero-shot foundation models. Beyond accuracy, the concept layer offers an interventional form of interpretability: forecasts can be steered by editing individual concept activations at inference time, and counterfactual ablations confirm that suppressing a concept produces a coherent, structurally readable change in the symbolic coefficients. The findings suggest that for domains rich in structured textual context, integrating metadata is a more efficient path to better forecasts than further scaling unimodal foundation models.

Keywords: time series forecasting, deep learning, interpretable machine learning, multimodal learning, neuro-symbolic models, concept bottleneck models, sparse autoencoder, macroeconomic forecasting, open dataset.

DEDICATION

First of all, I want to thank Pablo, for being the kind of researcher that is hard to find. For those long meetings where forecasting stopped feeling that difficult, for all the blackboard diagrams explanations, and for the feedback that made me keep pushing even more. This project grew the way it did because of his guidance.

Same goes for Emilio. Even from Spain, your expertise and flexibility made a real difference. For the thorough revisions that pushed me to get the best out of myself before the presentation, and for understanding the situation of my exchange.

To all my friends on my exchange in Sydney. For putting up with my days entirely “locked in” with a laptop, and for making the other days the kind you do not forget. To Lucía especially, and to every coffee we shared, which got me through more grey days than I can count.

To my parents and my brother, for always being there through everything I get myself into, even when the words I use to explain my work make no sense to them.

And to Marina. Even from the other side of the world, she was my daily anchor. My comfort on the hard days, and the reason I kept going when the research felt impossible. This one is for her too.

CONTENTS

1. INTRODUCTION.	1
1.1. Motivation and Context	1
1.2. Objectives.	3
1.3. Document Structure	4
2. OVERVIEW OF TIME SERIES FORECASTING.	6
2.1. Time Series Forecasting	6
2.2. Benchmark Datasets and Limitations.	6
2.2.1. The Multimodal Gap: Motivation for FRED	7
2.3. From Classical to ML and Foundational Models in Time Series Forecasting.	8
2.3.1. Classical Statistical Approaches	8
2.3.2. Machine Learning and Deep Learning Approaches.	8
2.3.3. Time Series Foundation Models	9
2.4. Towards Interpretable Forecasting: Symbolic Decomposition and Concept-Based Mechanisms	9
2.4.1. Neuro-Symbolic Models and Interpretable Forecasting Architectures	10
2.4.2. Concept Bottleneck Models and Sparse Autoencoders	10
2.5. Conditional Modulation: FiLM.	12
2.6. Multimodal Learning in Time Series	12
3. DATA GATHERING	14
3.1. Federal Reserve Economic Data (FRED) Database.	14
3.2. Data Collection via the FRED API	16
3.2.1. Unimodal Data Retrieval	17
3.2.2. Multimodal Data: Metadata and Tags	19
3.3. Data Preprocessing.	21
3.3.1. Unimodal Preprocessing Pipeline.	22
3.3.2. Preprocessing Pipeline for the Multimodal information	25
3.4. Final Dataset Structure.	28
3.4.1. Dataset for This Work	28

3.4.2. Dataset for Publication	28
4. METHODOLOGY	30
4.1. Problem Formulation: Multimodal Time Series Forecasting.	30
4.1.1. Task Definition	30
4.1.2. Input Normalisation and Padding.	31
4.1.3. Notation Summary	32
4.2. Neuro-Symbolic Forecasting Basis.	32
4.2.1. Basis-Function Decoder	32
4.3. Concept Bottleneck Models in the Forecasting Context	34
4.3.1. The Original Concept Bottleneck Model.	34
4.3.2. Adaptation to the Forecasting Setting	35
4.4. Feature-wise Linear Modulation (FiLM).	36
4.4.1. FiLM Modulation of the Symbolic Coefficients.	36
4.4.2. Why FiLM for This Problem	37
5. MODEL ARCHITECTURE	38
5.1. Architecture Overview.	38
5.2. Time Series Encoder	38
5.2.1. Patch and Flatten	40
5.2.2. Input Projection	42
5.2.3. Residual Blocks	42
5.3. Multimodal Fusion and Concept Bottleneck.	43
5.3.1. Multimodal Fusion.	43
5.3.2. TopK Sparse Autoencoder as Concept Bottleneck	44
5.4. Concepts influence through FiLM Modulation	44
5.5. Symbolic Forecasting Head	45
5.5.1. Trend Component	46
5.5.2. Seasonal Component.	47
5.5.3. Forecast Equation	48
5.6. Loss Function.	48
5.7. Training Setup and Evaluation Protocol	49
5.7.1. Train and Test Split	49

5.7.2. Evaluation across Context Lengths	50
5.7.3. Optimisation Details	50
6. EXPERIMENTAL WORK	52
6.1. Evaluation Metrics	52
6.1.1. Mean Absolute Error (MAE)	52
6.1.2. Symmetric Mean Absolute Percentage Error (sMAPE)	52
6.1.3. Mean Absolute Scaled Error (MASE)	53
6.1.4. Pairwise Win Rate	54
6.1.5. Mean Rank	54
6.2. Baseline Models	55
6.2.1. Chronos-2.	55
6.2.2. Chronos-bolt-base	56
6.2.3. TimesFM	56
6.2.4. Unimodal Neuro-Symbolic Ablation.	57
6.2.5. Exponential Smoothing (AutoETS)	57
6.3. Main Results	58
6.3.1. Aggregate Forecasting Accuracy	58
6.3.2. Pairwise Win Rates and Mean Rank	59
6.3.3. Error Evolution Across the Forecast Horizon	62
6.4. Qualitative Forecast Analysis	65
6.5. Interpretability and Concept-Level Analysis.	67
6.5.1. Concept Space Overview	68
6.5.2. Symbolic Coefficients: Ablation vs. Multimodal Model	70
6.5.3. The Role of FiLM Modulation	71
6.5.4. Concept Intervention.	74
7. REGULATORY FRAMEWORK	77
7.1. Dataset licensing and the FRED Terms of Use	77
7.1.1. The June 2024 update and the scientific TDM exception.	77
7.1.2. Copyright statuses and citation requirements	78
7.2. Model and dataset publication: an exploratory analysis	78
7.2.1. Source code and architecture	79

7.2.2. Trained model weights.	79
7.2.3. The derived dataset.	79
7.2.4. Software dependencies and third-party model licences.	80
7.3. Ethical considerations in macroeconomic data usage.	80
8. SOCIO-ECONOMIC ENVIRONMENT	82
8.1. Project Planning and Timeline	82
8.2. Budget Estimation	83
8.2.1. Human costs	83
8.2.2. Hardware and Software	83
8.2.3. Total Budget	85
8.3. Broader Socio-Economic Impact	85
9. CONCLUSION	87
9.1. Summary of Contributions	87
9.2. Future Work and Limitations	88
BIBLIOGRAPHY.	90
LIST OF ABBREVIATIONS	98
APPENDIX A: SUPPLEMENTARY RESULTS.	
APPENDIX B: PROBABILISTIC FORECASTING METRICS	
APPENDIX C: DECLARATION ON THE USE OF GENERATIVE AI	

LIST OF FIGURES

3.1	Main interface components of the Federal Reserve Economic Data (FRED) platform.	15
3.2	Example of an extremely short FRED series (USPTOISSUED051155, New Patent Assignments in Pulaski County, VA), spanning only five monthly observations. Series of this nature are excluded before training.	22
3.3	Optimal cutoff date selection procedure	23
3.4	Series-Level Metadata Prompt Construction	26
4.1	Structure of the neuro-symbolic basis-function decoder.	33
5.1	NeuroSym-CBF architecture overview.	39
5.2	Time-series encoder of NeuroSym-CBF.	40
5.3	Patch and flatten operation.	41
5.4	FiLM modulation of the symbolic coefficients.	46
6.1	Average pairwise win-rate matrix.	61
6.2	MAE at each forecast step ($h = 1, \dots, 18$) for all six models at context lengths $T = 8$ (left) and $T = 30$ (right).	63
6.3	Absolute gap in MAE between each baseline and NeuroSym-CBF at each forecast step, for $T = 8$ (left) and $T = 30$ (right).	64
6.4	Qualitative comparison of all evaluated models on nine randomly sampled test series with context length $\text{ctx} = 8$	66
6.5	Qualitative comparison of all evaluated models on nine randomly sampled test series with context length $\text{ctx} = 30$	67
6.6	Structure of the learned concept space at $\text{ctx} = 8$	69
6.7	Distribution of the 13 neuro-symbolic coefficients for the unimodal ablation (Neuro-Sym (Unimodal)) and the proposed multimodal model (NeuroSym-CBF)	71
6.8	Distribution of the coefficient difference $\theta^{\text{CBF}} - \theta^{\text{NS}}$ across the evaluation set, at $\text{ctx} = 8$ (top) and $\text{ctx} = 30$ (bottom).	72
6.9	Distribution of the internal FiLM modulation ($\theta_{\text{mod}} - \theta_0$) within NeuroSym-CBF, at $\text{ctx} = 8$ (top) and $\text{ctx} = 30$ (bottom).	73

6.10	Concept intervention at $\text{ctx} = 8$ for the two most active concepts.	75
6.11	Concept intervention at $\text{ctx} = 30$ for the two most active concepts.	76
8.1	Gantt chart of the project.	82

Appendix

1	Pairwise win-rate matrix at context length $T = 8$
2	Pairwise win-rate matrix at context length $T = 10$
3	Pairwise win-rate matrix at context length $T = 15$
4	Pairwise win-rate matrix at context length $T = 24$
5	Pairwise win-rate matrix at context length $T = 30$
6	MAE per horizon step at $T = 8$
7	MAE per horizon step at $T = 10$
8	MAE per horizon step at $T = 15$
9	MAE per horizon step at $T = 24$
10	MAE per horizon step at $T = 30$
11	Structure of the learned concept space at $\text{ctx} = 30$

LIST OF TABLES

2.1	Comparison of major time series forecasting benchmarks.	7
3.1	Schema of the per-release Parquet files produced by the unimodal pipeline.	19
3.2	Fields of a FRED tag object returned by the <code>/fred/series/tags</code> endpoint.	20
3.3	Two sample rows of the merged multimodal dataset (metadata + expanded tags).	21
3.4	Schema of the model-ready dataset used for training and evaluation.	28
3.5	Schema of the public release Parquet file.	29
4.1	Notation used throughout the methodology and architecture chapters.	32
5.1	Number of training windows for each evaluated context length.	50
6.1	Forecasting accuracy of all models	60
6.2	Mean rank of each model averaged over all five context lengths, under three metric sets.	62
6.3	Percentage MAE reduction of NeuroSym-CBF over each baseline at selected horizon steps, per context length.	65
6.4	SAE concept usage statistics	68
8.1	Human costs.	84
8.2	Hardware and software costs.	84
8.3	Indirect costs.	84
8.4	Total project budget.	85

Appendix

1	Per-context mean rank breakdown for all models under three metric sets.
2	Absolute MAE at selected horizon steps, $T = 8$
3	Absolute MAE at selected horizon steps, $T = 10$
4	Absolute MAE at selected horizon steps, $T = 15$
5	Absolute MAE at selected horizon steps, $T = 24$
6	Absolute MAE at selected horizon steps, $T = 30$

1. INTRODUCTION

Most of what gets measured over time can be treated as a time series, and the amount of data produced in that form today is enormous. Sensor readings, transactions, prices and web activity accumulate continuously, often at the second level. The series that reflect the economy work very differently. Gross domestic product, inflation, employment and industrial production are constructed through surveys and statistical adjustments by national agencies so then they are usually reported with a lag of several weeks or months. Decisions, however, cannot be deferred until the data arrives and must act on whatever information is available at the time. Forecasting addresses this asymmetry by processing the historical record into estimates of what has not yet been observed.

Forecasting is extrapolation. The analyst producing one in practice, rarely works from numbers alone. They also know what the series measures, the unit it is reported in, the agency that publishes it, the category it belongs to, and broader context such as related indicators, external data sources, or the assessments of other analysts. The problem is that the forecasting models that dominate the field today, from classical statistical baselines to large pretrained foundation models, ignore all of these and just take as context the numerical data.

Beyond that, the field has long operated along a familiar trade-off. Classical statistical models such as ARIMA or state-space formulations are transparent by design: every parameter has a name and a meaning, and the analyst can inspect, override or correct any part of the forecast. Large-scale time series foundation models (TSFMs) push the accuracy frontier on standard benchmarks, but they do so as black boxes. There is no coefficient to inspect, so there is no way for a domain expert to tell the model *this series should not grow that fast given what I know about it*.

The problem is sharpest in short-context regimes: when a series is new or recently relaunched, numerical history alone is not enough for any model to identify the underlying process, and a TSFM with little to work from tends to fall back on generic patterns learned from its pretraining distribution. The same applies after structural breaks, where a currency union, a sharp policy shift, or a regulatory change resets the dynamics of a series. This showcases that the numerical signal on its own, is not always sufficient.

1.1. Motivation and Context

The trade-off sketched above masks a far more complex and revealing reality. Standard benchmarks are dominated by series with strong periodicities and seasonalities, which inflates the apparent gap between modern architectures and well tuned classical baselines; once evaluation is stratified by series taxonomy, the gains of large pretrained models over

classical methods such as AutoARIMA or AutoETS are substantially smaller, and in many categories disappear altogether [1]. This was already the picture in the original M4 competition, where the winning entry was a hybrid that combined a classical structural model with a neural component [2]. The implication is not that pretrained models should be discarded, but that the choice between accuracy and interpretability is much weaker than it appears, and that hybrid approaches combining structural priors with learned components remain a viable design space.

In practice, an analyst forecasting a macroeconomic series wants more than a number. They want to know why the model expects the series to grow, when the seasonality kicks in, how strong the trend is, and whether any of those components is at odds with their own reading of the data. A single forecasted value, even an accurate one, does not answer those questions. A decomposition into intercept, trend and seasonal components does, because each piece is a quantity that the analyst can read, measure, and if needed override. In macroeconomic analysis this decomposition is often the goal in itself: isolating the underlying trend from seasonal noise is the actual question being asked, for instance when checking questions such as "has unemployment really gone up, or is this just the usual end-of-summer effect?". This type of question is more of estimation than forecasting, and is answered precisely by inspecting the seasonal and trend components separately.

That last word matters. Interpretability that the user can only look at is far less useful than interpretability the user can act on. The distinction goes beyond explaining a forecast to a stakeholder: in high-stakes settings, what matters is the ability to identify failure modes when the model produces erroneous forecasts and to intervene on its internal state to correct them. A hallucination in a black-box model is hard to recover from, and that alone has been enough to slow the adoption of large pretrained models in regulated environments. Classical statistical models score well on both counts, which is the main reason they remain in use despite their known limitations in raw accuracy [3]. The same expectation applies to the symbolic components developed in this work: they should be readable, editable, and exposed in a form that allows targeted intervention.

The second strand of the motivation is the contextual information that surrounds every real time series and that current forecasting models simply ignore. The Federal Reserve Economic Data (FRED) repository [4], the source used throughout this thesis, distributes more than a million series, and each one arrives with a title naming what it measures, a unit indicating whether the values are dollars, indices or percentages, and a set of analyst assigned tags placing it inside a hierarchy of economic concepts. A unimodal TSFM sees none of this. An analyst working with the same input would never proceed that way, because the meaning of a forecast depends on what is being forecasted. The subset of FRED retained for this work amounts to nearly 200,000 monthly series, which is larger in number of series than most pretraining corpora used by current TSFMs at the monthly frequency, so the scale argument that typically favours pretraining does not, in this domain, work against a specialised model.

Combining the two modalities is not the only contribution, since multimodal forecasting architectures already exist [5], [6]. Existing multimodal time series datasets, however, are either small, drawn from domains where text adds little to forecasting (weather, biomedical signals), or carry text that is purely synthetic or derived from the numeric data itself; the FRED metadata used in this work, in contrast, is curated by expert analysts at the level of the individual series. The relevant question is how they should interact. The position taken in this work is that the textual modality should intervene on the symbolic components and only on the symbolic components. The metadata can amplify a trend, dampen a seasonal pattern or shift a baseline. It cannot bypass the decomposition and write the forecast directly. The restriction is deliberate: by forcing the metadata to act through the symbolic channel, anything the metadata changes remains readable in the same terms the analyst already uses.

This design is particularly relevant in a regime where existing models struggle most. Many real forecasting problems involve series with very limited history: newly issued indicators, recently relaunched products, indices defined only a few periods ago. Limited observations force any model to lean on its prior, on the regularities accumulated during training rather than on the signal in the series itself. A TSFM pretrained on a broad mixture of domains carries a generic prior that may be ill-suited to the series at hand, leading it to commit to patterns that match unrelated training data; a macroeconomic series showing a few months of accelerated growth, for instance, can be extrapolated as an exponential trajectory plausible in epidemiological or energy demand data but implausible for the macroeconomic indicator being forecasted. A model trained on a homogeneous macroeconomic corpus develops a prior that is narrower but better aligned with the series it will encounter at inference, and conditioning the forecast on what the series is about recovers part of the predictive signal that a purely numerical model loses by construction.

1.2. Objectives

The main goal of this thesis is to develop a forecasting model that decomposes its forecasts into explicit, readable components, that lets textual metadata steer those components through a structured and editable channel, and that remains competitive with current time series foundation models on macroeconomic data. To this end, the following intermediate objectives are pursued:

- The first objective is to construct a multimodal time series dataset on which the rest of the work can be developed and evaluated. This requires extracting and cleaning a substantial portion of the macroeconomic series published by the Federal Reserve [4], designing a metadata schema that captures the contextual information already curated by analysts, and producing both a model-ready version of the dataset and a standardised release format intended for future public distribution. Given the nature of the data source, this objective also includes a regulatory and ethical anal-

ysis of the conditions under which the data can be used, published and redistributed for research purposes.

- The second objective is to formulate the multimodal time series forecasting problem in a rigorous notation and to design an interpretable forecasting head built on a symbolic basis-function decomposition, where each output is decomposed into intercept, trend and seasonal components with explicit, readable coefficients rather than being produced by a dense output layer.
- The third objective is to design and implement a multimodal forecasting architecture that integrates the numeric and textual modalities in a structured way, ensuring that the metadata reaches the forecast exclusively through the interpretable symbolic channel rather than through unconstrained end-to-end fusion. The architecture must be editable by design: any change the metadata introduces to the forecast should remain readable in the same symbolic terms an analyst already understands.
- The fourth objective is to evaluate the proposed model against both an ablation of the same architecture and strong state of the art time series foundation models, in order to isolate the contribution of the multimodal pathway and situate the work within the literature. Beyond metrics, the evaluation should characterise under which conditions the metadata signal is most beneficial and where the model’s limitations lie.
- The fifth and final objective is to validate that the interpretability built into the architecture is genuine and actionable. The model should not only have readable components but should also, and most importantly, allow a user or analyst to probe and intervene on them in ways that produce predictable, meaningful changes in the forecast.

1.3. Document Structure

After this chapter, the document is organised as follows.

Chapter 2 traces the evolution of time series forecasting from classical statistical models through deep learning and large pretrained foundation models, reviews existing benchmark datasets and their limitations. It also introduces the three technical concepts the proposed architecture builds on: neuro-symbolic forecasting, concept bottleneck mechanisms, and feature-wise linear modulation. Chapter 3 describes how the dataset was built, from extracting monthly macroeconomic series from FRED and processing both the numeric and textual modalities, to the structure of the two dataset versions produced.

Chapters 4 and 5 develop the model. The first one defines the forecasting problem formally and introduces the symbolic basis-function decoder, the concept bottleneck, and the FiLM modulation mechanism. The second assembles these components into the full architecture and describes the training setup and evaluation protocol.

Chapter 6 presents the experimental work. Forecasting accuracy is evaluated across different context lengths against a unimodal ablation and several foundation model baselines, with additional analysis of how error evolves along the forecast horizon. A separate section probes the concept space, the symbolic coefficients, and what happens to the forecast when individual concepts are silenced.

Chapter 7 covers the legal and ethical side of working with Federal Reserve data, including the conditions under which the dataset and weights could eventually be published. Chapter 8 describes the planning, budget, and broader socio-economic impact. Finally, Chapter 9 closes with the main findings and directions for future research.

2. OVERVIEW OF TIME SERIES FORECASTING

This chapter provides an overview of the time series forecasting landscape, from classical methods to modern deep learning approaches. It reviews the benchmark datasets that have shaped the field and their limitations, before introducing the modelling techniques most relevant to this work. Particular attention is given to interpretable and symbolic approaches, conditional modulation mechanisms, and the integration of multimodal information.

2.1. Time Series Forecasting

Time series forecasting is the task of predicting future values of a sequence of observations indexed in time. According to [3], the forecasting theory relies on both use of current and historical data to generate predictions; the underlying assumption is that the patterns present in past observations (trends, seasonality, cycles, etc.) carry information about the future.

This idea is foundational across virtually every domain where sequential data arises: economics, energy systems, healthcare and climate science, to name a few. What makes the problem genuinely hard is that real-world series are rarely clean or stationary: they exhibit non-linear dynamics, structural breaks, and irregular noise, which means that both the choice of model and the amount of available historical context matter enormously. This is particularly pronounced in macroeconomic series, where high stochasticity and structural uncertainty impose hard limits on predictability, regardless of the sophistication of the model being used [7].

2.2. Benchmark Datasets and Limitations

The development of forecasting models has been closely tied to the availability of standardised benchmark datasets. Over time, a small set of repositories has come to dominate model evaluation, shaping both the methods that get proposed and the way progress is measured. Table 2.1 reviews the most influential ones and identifies a shared structural limitation that motivates the dataset introduced in this work.

The M4 Competition [8] introduced a large-scale benchmark of 100,000 time series spanning six domains and frequencies, becoming a cornerstone for evaluating both classical and modern forecasting methods. Its scale and diversity established a new standard for rigorous comparison, and many subsequent models have reported results on it. However, the series are stripped of any identifying metadata information.

The Monash Forecasting Repository [9] extended this vision by aggregating 20 datasets

TABLE 2.1. COMPARISON OF MAJOR TIME SERIES
FORECASTING BENCHMARKS.

Benchmark	Series	Domains	Frequencies	Metadata
M4 [8]	100,000	6	6	X
Monash [9]	20 datasets ranging from (1-145,063 series)	+10	Multiple	X
GIFT-Eval [10]	144,000	23	Multiple	X
FRED-MD [11]	128	Macroeconomic	Monthly	Partial
FRED-QD [12]	248	Macroeconomic	Quarterly	Partial

from heterogeneous sources into a unified archive, covering more than 13 domains ranging from banking and energy to transport and nature, with series lengths spanning from a few dozen to millions of observations. It became a reference point especially for global and deep learning models. It includes well-known datasets such as a subset of FRED-MD [11], Electricity, or Web Traffic among others. Yet its design is purely numeric: no metadata, no textual descriptions, no contextual information of any kind accompanies the series.

More recently, GIFT-Eval [10] was introduced as a benchmark specifically designed to evaluate time series foundation models, covering 23 datasets across diverse domains with a non-leaking pretraining split. Despite its scope and rigour, it remains a numeric-only benchmark as well.

Closer to the domain of this work, FRED-MD [11] and FRED-QD [12] are curated macroeconomic databases built on top of the Federal Reserve Economic Data (FRED) service, providing 128 monthly and 248 quarterly U.S. macroeconomic time series respectively, both starting in 1959 and updated in real time. Organised into broad economic categories such as labour market and housing they have become standard benchmarks for macroeconomic forecasting. Both databases include a transformation code per series indicating the recommended differencing or log-transformation to achieve stationarity, and a short mnemonic label. However, the contextual metadata available in FRED for each series such as full titles, units, seasonal adjustment information, and categorical tags is not part of either database.

2.2.1. The Multimodal Gap: Motivation for FRED

A clear pattern arises from the benchmarks above: the field has invested heavily in scale and domain diversity, but has largely treated series as anonymous numeric sequences, discarding the contextual information that accompanies them in practice. This is particularly evident in the macroeconomic domain, where FRED-MD [11] and FRED-QD [12] despite being grounded in the FRED ecosystem, are designed as numeric references, and strip away most of the rich metadata that FRED associates with each series: full titles,

units, detailed descriptions, seasonal adjustment information, and categorical tags.

This is not a minor omission. As shown in [6], textual context can be essential for forecasting when numeric history alone is ambiguous, and this is precisely the regime that macroeconomic series often inhabit. Furthermore, both FRED-MD [11] and FRED-QD [12] are limited to curated subsets of around 128 and 248 series respectively, leaving the vast majority of the FRED catalogue unexploited for model training. The full FRED catalogue is larger and richer than any existing forecasting benchmark, including GIFT-Eval [10] and Monash [9]. Additionally, macroeconomic series are precisely the domain where statistical and machine learning models dominate, unlike datasets drawn from physical phenomena such as weather or solar activity (e.g., solar power series in [9] and [10] Nature domain), where domain-specific architectures exploit known physical priors. The Federal Reserve Economic Data ecosystem provides a natural setting to address both limitations: it offers a much larger collection of well-documented macroeconomic series with rich, structured metadata, making it the ideal foundation for a metadata-enriched forecasting dataset. The construction of this dataset is described in detail in Chapter 3.

2.3. From Classical to ML and Foundational Models in Time Series Forecasting

Forecasting methods have evolved considerably over the past decades, moving from hand-crafted statistical models to large neural architectures pre-trained on billions of observations. This section provides a concise overview of that trajectory, focusing on the approaches most relevant to understanding the contributions of this work.

2.3.1. Classical Statistical Approaches

The earliest and most established methods for time series forecasting are grounded in statistical theory. The ARIMA family [13] models a series as a linear combination of its own past values and past forecast errors, with differencing to handle non-stationarity. These models remain competitive in many settings, particularly for short, low-noise series with clear periodic structure. Exponential Smoothing [14] takes a complementary approach, weighting recent observations more heavily than older ones, with extensions such as Holt-Winters handling trend and seasonality. Despite their simplicity, both families have proven remarkably robust: [15] showed that such classical methods consistently outperformed machine learning approaches across a wide set of benchmarks; a result that demonstrates that model complexity is not always the answer.

2.3.2. Machine Learning and Deep Learning Approaches

The growing availability of large datasets and compute power shifted attention towards data-driven approaches. Recurrent Neural Networks (RNNs), and in particular Long

Short-Term Memory networks [16], became the dominant paradigm for sequential modelling through the 2010s. LSTMs address the vanishing gradient problem of vanilla RNNs via gating mechanisms that selectively retain or discard information over time, making them well-suited for learning long-range temporal dependencies [17].

Temporal Convolutional Networks (TCNs) [18] emerged as a convolutional alternative, demonstrating that causal dilated convolutions can match or outperform recurrent architectures on a range of sequence tasks while enabling fully parallel computation during training. The use of dilated convolutions allows TCNs to capture patterns at multiple temporal scales without the sequential bottleneck of recurrent models.

Residual connections [19], originally introduced for image recognition, proved equally valuable in time series settings. By adding shortcut connections that bypass one or more layers, residual architectures mitigate the vanishing gradient problem in deep networks and allow the model to learn incremental corrections rather than full transformations, a particularly useful inductive bias when dealing with noisy or non-stationary sequences.

2.3.3. Time Series Foundation Models

More recently, the success of large pre-trained models in NLP and computer vision has inspired an analogous direction in time series forecasting: Time Series Foundation Models (TSFMs). These models are trained on large, heterogeneous corpora of time series spanning multiple domains, with the goal of learning general-purpose temporal representations that transfer well to new datasets without retraining.

A key advantage of TSFMs is their ability to perform zero-shot forecasting¹, making them practical in data-scarce scenarios where training a task-specific model from scratch is not feasible. Two prominent examples evaluated in this work are Chronos [20], developed by Amazon and TimesFM [21], developed by Google. Both models achieve near state-of-the-art zero-shot performance across diverse domains. They will be further discussed and evaluated in detail in Chapter 6.

2.4. Towards Interpretable Forecasting: Symbolic Decomposition and Concept-Based Mechanisms

There is a detail in how these large pretrained models are actually built that is easy to miss. Their training corpora are not made up only of real time series: a large fraction of what they see during pretraining is synthetic data generated from classical structural models, with the explicit purpose of exposing the network to dynamics that real datasets alone do not cover densely enough. Chronos [20] relies on KernelSynth, which produces series from Gaussian Process kernels chosen to mimic trend, seasonal and noise components;

¹Zero-shot forecasting refers to the ability of a model to generate predictions on a previously unseen dataset without any task-specific fine-tuning, relying entirely on patterns learned during pre-training.

Chronos-2 [22] extends this with multivariatizers that inject cross-series dependencies through similar generative recipes. In other words, what TSFMs learn during pretraining is, to a large extent, the behaviour of ARIMA, exponential smoothing and state space processes, only absorbed implicitly into a much larger parameter budget. The interpretable architectures discussed in the rest of this chapter take the opposite route: they encode it directly into the model and use the neural component only for the parameter estimation.

As forecasting models grew in complexity, so did concerns about their interpretability. A neural network that maps a context window directly to a sequence of future values offers little insight into why it produces a particular forecast. Two distinct lines of research address this concern from different angles. The first develops forecasting architectures that are interpretable by construction, constraining the predicted output to be decomposable into human readable components. The second, originating outside the forecasting literature, develops general purpose interpretability mechanisms that make the internal reasoning of a network explicit and editable; these mechanisms are not forecasting models on their own, but they provide building blocks that can be imported into a forecasting setting, as is done in this work. This section reviews both lines and clarifies the role each one plays in motivating the architecture proposed in Chapter 4.

2.4.1. Neuro-Symbolic Models and Interpretable Forecasting Architectures

Neuro-symbolic AI is the family of approaches that combine neural networks with symbolic reasoning, targeting the limitations of each paradigm in isolation: neural models learn from raw data but are opaque, while symbolic systems are interpretable but fragile under noisy or high-dimensional inputs [23], [24]. The combination allows each component to do what it does well. In image classification, a neural backbone handles low-level perception while a symbolic layer reasons over the extracted features to produce an auditable decision. The same logic applies in question answering systems, where neural encoders process language and symbolic modules execute structured reasoning steps over a knowledge base, making the inference chain readable and editable [24].

In time series forecasting, this philosophy translates into architectures that constrain the predicted output to be decomposable into meaningful temporal components. N-BEATS [25] is the clearest example: it builds its forecast as a sum of learned expansions over interpretable bases, one for trend and one for seasonality, achieving state-of-the-art accuracy while keeping the prediction readable in terms of what the model is actually attributing to each component. This interpretable-by-construction design directly inspires the symbolic forecasting head developed in this work, as described in Chapter 4.

2.4.2. Concept Bottleneck Models and Sparse Autoencoders

A complementary line of research focuses on making the internal reasoning of a network interpretable and editable, not on improving predictive accuracy per se. The methods cov-

ered here were not designed as forecasting models; they are interpretability mechanisms that this work borrows and integrates into a forecasting pipeline.

Concept Bottleneck Models (CBMs), introduced by [26], propose a two-stage pipeline in which a network first maps its input to an intermediate layer of human-meaningful concepts before producing the final prediction. This design makes the model’s reasoning auditable and correctable: one can read off which concepts were activated for a given input and, crucially, intervene on them at test time to steer the output. The original formulation requires concept-level supervision, but subsequent work has relaxed this constraint. [27] demonstrated that hybrid CBM architectures (where some concepts are supervised and others are discovered automatically) can improve task accuracy over purely supervised or purely unsupervised baselines. In a similar direction, [28] applied CBMs to two-stage binary classification of extreme wind speed events, building each concept from a positive-class branch of a boosted ensemble of shallow decision trees fit on OHLC-style features borrowed from financial time series analysis, with sparsity enforced downstream by an L1/L2 logistic regressor with recursive feature elimination. The output remains categorical, as is natural for CBMs, but the methodological precedent matters: a sparse, human-readable concept layer can be built from time series inputs without manual annotation. Extending this idea to a continuous forecasting output is one of the central design problems addressed in Chapter 4.

This perspective is closely related to recent work on sparse autoencoders (SAEs) for concept discovery in neural networks [29], [30]. Rather than relying on predefined semantic concepts, SAEs learn a dictionary of sparse latent features from the internal activations of a network, producing representations that are more monosemantic and easier to inspect than the raw neurons. The technique has been particularly influential in mechanistic interpretability of large language models, where it has been used to disentangle thousands of human-meaningful features from polysemantic activations [30]. A practical refinement of the original formulation is the use of a Top- K activation [31], which directly controls the sparsity level by keeping only the K largest latents active, simplifying training and improving the reconstruction-sparsity trade-off. In the time series domain, TimeSAE [32] shows that such sparse representations can deliver strong forecasting performance while improving the quality of the learned features.

A complementary and very recent line of work places the SAE inside a pretrained TSFM. [33] freeze a Chronos-2 model and train a BatchTopK sparse autoencoder on its residual-stream activations, with an $8\times$ expansion of the $d_{\text{model}} = 768$ dimension and $K = 100$. Without any supervision, the resulting features fire on coherent and high-level patterns: stabilisation after a downward transition, or activation in the few patches that immediately precede a decline. Their objective is to interpret an existing black-box forecaster with an underlying idea similar to this work: a Top- K sparse bottleneck over a learned dictionary makes the internal logic of a time series model readable. The authors themselves point to multimodal time series models as the natural next step, and that is exactly where the architecture proposed in this thesis sits, with the sparse bottleneck inside

the forward pass and operating on a representation that fuses the numeric context with its textual metadata.

2.5. Conditional Modulation: FiLM

Feature-wise Linear Modulation (FiLM), introduced by [34] in the context of visual question answering, is a general-purpose conditioning mechanism that allows one information stream to modulate another through a learned affine transformation. Given a feature vector $\mathbf{f}$ and a conditioning signal $\mathbf{c}$, FiLM computes a scale $\gamma(\mathbf{c})$ and a shift $\beta(\mathbf{c})$ as functions of $\mathbf{c}$, and applies them element-wise to each feature dimension i :

$$f_i^{new} = \gamma_i(\mathbf{c}) \cdot f_i^{old} + \beta_i(\mathbf{c}), \quad (2.1)$$

where γ_i and β_i are the i -th entries of $\gamma(\mathbf{c})$ and $\beta(\mathbf{c})$ respectively.

Despite its simplicity, this mechanism proved highly effective at injecting conditional information into deep networks without disrupting the primary processing stream. Its key appeal is that the effect of the conditioning signal is explicit and spatially localised: each dimension of $\mathbf{f}$ is scaled and shifted independently, making the modulation readable.

Although FiLM itself has seen limited adoption in time-series forecasting, recent work points towards a growing interest in conditioning forecasting models through structured affine transformations. For instance, LeMoLE [35] uses text-derived embeddings to modulate linear forecasting experts, while APT [36] generates affine parameters from temporal representations, explicitly linking its design to FiLM-style conditioning.

In this work, FiLM is applied directly to the symbolic forecasting coefficients conditioned on the sparse concept vector, a design choice whose motivation and implementation are described in detail in Chapter 4.

2.6. Multimodal Learning in Time Series

In the time series domain, the term "multimodal" can refer to two distinct settings. The first, intrinsic multimodality, involves series where multiple synchronised data streams are recorded simultaneously as part of the observation itself, such as a patient monitoring system recording ECG and respiratory rate together. The second, and the one relevant to this work, involves a numeric series accompanied by external contextual information (typically text).

The authors in [6] formalised this second setting and demonstrated through the Context is Key benchmark that textual context can be essential for accurate forecasting precisely when the numeric history is too short or too ambiguous to identify the correct regime. [37] showed, through a dual-tower architecture combining a patch-based time

series encoder with an LLM-based text encoder via cross-attention, that incorporating textual descriptions consistently improves accuracy in high-dimensional forecasting, particularly at longer horizons where the numeric signal alone fades. In a follow-up work, [38] introduced MoTime, a suite of eight datasets pairing time series with static external modalities such as text, metadata, and images. The benchmark, however, has several limitations relevant to this work: its datasets come exclusively from e-commerce, web traffic, and media domains rather than macroeconomic series, and the temporal resolution is heterogeneous across datasets and not controlled. The authors themselves report that their initial hypothesis on the utility of external modalities only holds in specific datasets, rather than universally [38].

The closest existing work to the architecture proposed in this thesis is MetaTST [5], which integrates metadata into a Transformer-based forecaster by encoding textual descriptions with a frozen LLM and combining the resulting metadata tokens with patch-wise series tokens through a Transformer encoder, allowing cross-attention to fuse both modalities. Despite the conceptual similarity, three differences are worth highlighting. First, most of the metadata used in MetaTST is generated from the numeric series itself, such as starting timestamps, means, and standard deviations, rather than from independent descriptions provided by domain experts. Second, the use of a Transformer encoder as the fusion mechanism makes the contribution of the metadata to the final forecast opaque, in the sense that there is not a structured way to attribute changes in the prediction to specific metadata fields. Third, and as a direct consequence of the previous point, the model exposes no structured intermediate representation that a practitioner can inspect or intervene on: the metadata’s influence reaches the forecast exclusively through the cross-attention mechanism, leaving no handle through which it can be probed or adjusted at inference time. The architecture proposed in this work, NeuroSym-CBF, addresses these three points explicitly through the combination of expert-provided metadata, a sparse concept bottleneck, and the FiLM modulation discussed in Section 2.5.

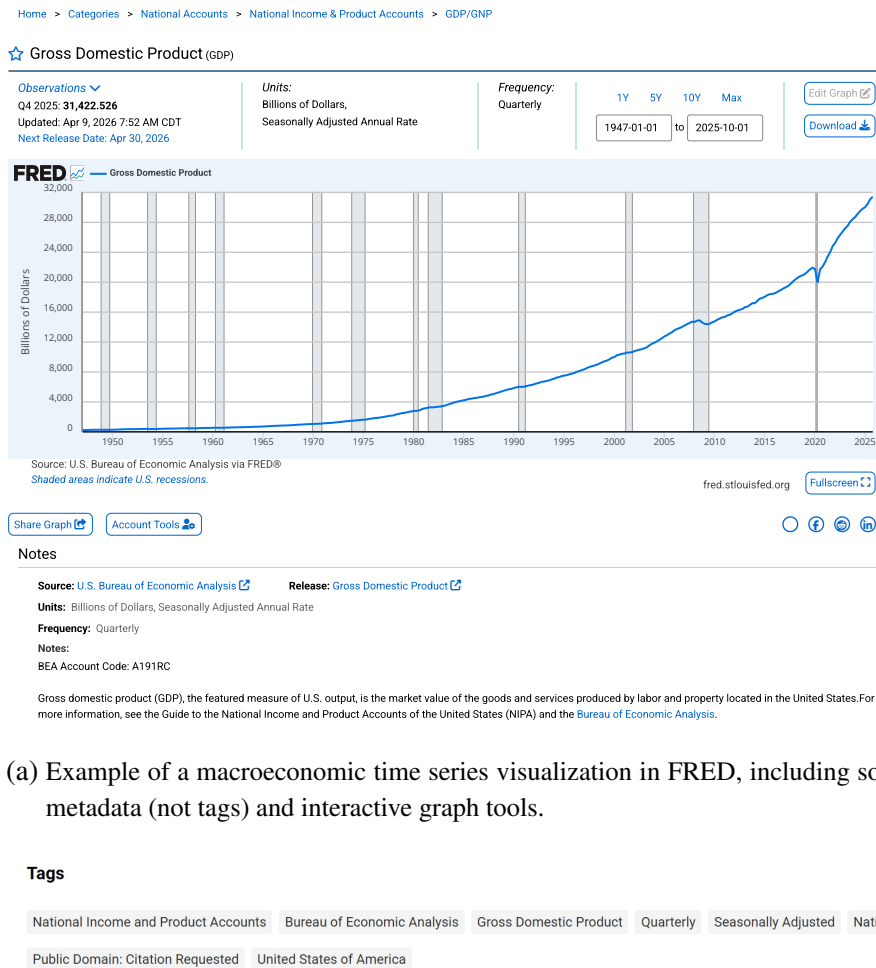
3. DATA GATHERING

Building a dataset for multimodal time-series forecasting involves more than simply downloading observations from a public source. It requires, first, a selection of a data source that is rich enough in both numerical content and associated metadata to support a careful retrieval and preprocessing pipeline. This chapter details all of those steps: the selection of the Federal Reserve Economic Data (FRED) as this project’s data source, the retrieval of time-series observations and their accompanying metadata via the FRED API, the preprocessing decisions applied to both modalities, and the final dataset construction. The pipeline produces two distinct artefacts: a compact, model-ready dataset used throughout this work, and a richly annotated public release intended as a standalone contribution to the research community.

3.1. Federal Reserve Economic Data (FRED) Database

The Federal Reserve Economic Data (FRED) is a large-scale, publicly accessible economic database maintained by the Research Division of the Federal Reserve Bank of St. Louis, one of the twelve regional banks that compose the United States Federal Reserve System. Launched in the early 1990s, FRED has grown to become one of the most widely used repositories of macroeconomic and financial data in the world, both in academic research and in professional economic analysis. The database is updated continuously, with new releases published almost every working day (typically 30 to 40 per day during a regular week [4]). These daily releases do not reflect revisions of a single figure but rather distinct series that receive their scheduled update on a given day, each following its own frequency and predetermined publication date as set by the official FRED release calendar [4]. The database therefore grows in a continuous yet orderly and predictable manner, which is why the figures reported below correspond to the state of FRED on the date of the retrieval carried out for this work, the *March 11, 2026*. At that point the database hosted 844,714 time series drawn from 120 national, international, and intergovernmental sources, including the U.S. Bureau of Labor Statistics, the Bureau of Economic Analysis, the International Monetary Fund, the World Bank, and the Organisation for Economic Co-operation and Development (OECD) [4].

The series available in FRED span a broad spectrum of economic and financial domains: labor markets (unemployment rates, nonfarm payrolls), price indices (CPI, PCE deflator), monetary aggregates and interest rates (federal funds rate, money supply measures), national accounts (GDP, personal consumption), trade balances, housing indicators, and financial market variables, to name just a few. The geographic scope is heavily skewed towards the United States, which alone accounts for roughly 680,000 series, with the remainder covering international, regional, or cross-country aggregates. Obser-



(a) Example of a macroeconomic time series visualization in FRED, including some metadata (not tags) and interactive graph tools.

(b) Tag-based metadata system used in FRED for dataset categorization and discovery.

Fig. 3.1. Main interface components of the Federal Reserve Economic Data (FRED) platform.

variations are reported at multiple sampling frequencies², dominated by annual (~490,000 series) and monthly (~210,000) data, followed by quarterly (~110,000), daily (11,472), and weekly (3,631), with smaller groups at semiannual, biweekly, and five-year resolutions. Series lengths also vary widely: some span several decades, while others cover only recent years or a specific economic event. 11.1% of the series are flagged as seasonally adjusted, with the remaining 88.9% reported as raw, non-seasonally adjusted observations[4]. In terms of licensing, 72.2% of the series are released under a public-domain citation-requested licence, while the rest are subject to citation or pre-approval requirements depending on the originating source[4]; the implications of these licensing terms for the publication of the derived dataset are discussed in Chapter 7.

What makes FRED particularly compelling for multimodal analysis is that each series is associated with a rich layer of structured metadata curated and maintained by FRED

²Approximate figures correspond to the values reported on the FRED website at the time of the extraction; exact counts are not exposed by the public API for these categories.

analysts: a readable title and description, the measurement units and frequency, the seasonal adjustment status (whether the series has been seasonally adjusted or not), the originating source organization, textual notes providing methodological context, and a set of analyst-assigned tags that categorize the series thematically (e.g., `gdp`, `monthly`, `usa`, `nsa`, `labor market`). This tag taxonomy, though informal in structure, constitutes a form of expert knowledge: it encodes domain-relevant information that purely numerical analysis cannot recover on its own. Crucially, much of this metadata mirrors the contextual knowledge that an expert analyst would consult when modelling a series: sampling frequency, seasonal adjustment status, geographic scope or any other information that directly influences modelling decisions. It is therefore reasonable to say that a multimodal model could leverage these structured signals, even in cases where richer but noisier fields such as the series title or country of origin may contribute less consistently.

Beyond its metadata richness, FRED is also one of the largest publicly available macroeconomic repositories. It is continuously updated in real time, and has demonstrated long-term reliability over decades of operation. This combination of scale, metadata depth, and forecasting relevance makes FRED a compelling fit for multimodal time-series research.

FRED also provides a clean, user-friendly web interface accessible at <https://fred.stlouisfed.org>, which allows researchers and practitioners to search, visualize, and download individual series with minimal friction. The interface offers interactive charting tools, the ability to apply transformations (log, percentage change, year-on-year growth) directly in the browser, and a category-based navigation tree that organizes series by economic theme. This makes it straightforward to inspect and understand a given series in isolation before incorporating it into a larger dataset, which was useful during the exploratory phase of this work. The accompanying public REST API, described in the next section, extends this accessibility to programmatic workflows, enabling larger-scale data collection.

It is important to note that FRED’s relevance as a research resource is well established in the macroeconometrics and forecasting literature. Subsets of it have been formalized into curated benchmark databases (most notably FRED-MD and FRED-QD, monthly and quarterly macroeconomic panels designed for rigorous empirical and forecasting research [11]) which have become standard baselines in the time-series literature, as discussed in Chapter 2.

3.2. Data Collection via the FRED API

FRED exposes its data programmatically through a REST API available at <https://fred.stlouisfed.org/docs/api/fred/>, which allows querying series observations, metadata, categories, tags, and releases via standard HTTP requests authenticated with a personal API key. The key is obtained free of charge through registration on the FRED

website and must be included in every request either as a query parameter or as a Bearer token in the request header. Responses are available in JSON or XML format and follow a consistent structure across endpoints.

The main entities exposed by the API are the following:

- **Series:** the fundamental unit of FRED. Each series has a unique identifier (e.g., CPIAUCSL for the Consumer Price Index or PAYEMS for the Total Nonfarm Payroll) and contains time-indexed observations along with metadata fields such as `title`, `units`, `frequency`, `seasonal_adjustment`, `observation_start`, `observation_end`, and `popularity`.
- **Observations:** the actual numeric data points of a series, returned as a list of (date, value) pairs. Since observations are indexed by date rather than by position, the resulting series may be irregularly sampled or contain gaps, which requires explicit alignment and imputation before use in any temporal model. This is addressed in the preprocessing pipeline described in Section 3.3.2.
- **Tags:** analyst-assigned labels that categorize a series thematically. Each series can carry multiple tags, retrievable via a dedicated endpoint.
- **Categories:** a hierarchical taxonomy organizing series by topic (e.g., National Accounts, Labor Market).
- **Releases:** named publication groups that bundle series published together by a source institution. Every series in FRED belongs to exactly one release, and release identifiers are guaranteed to be disjoint, that is to say, that no series appears in more than one release.

A Python wrapper library, `fredapi` [39], abstracts over the raw HTTP layer and makes individual series retrieval straightforward. Through it, fetching observations, metadata, and tags for a given series reduces to a few method calls: `get_series_observations()`, `get_series()`, and `get_series_tags()`, respectively. This interface is well suited for exploratory analysis and for working with a handful of series at a time, as illustrated in the early stages of this project with canonical series such as CPIAUCSL (CPI), UNRATE (unemployment rate), or FEDFUNDS (federal funds rate).

3.2.1. Unimodal Data Retrieval

Among the frequencies available in FRED, monthly series are the focus of this work for several reasons:

- Monthly is the frequency at which the broadest range of macroeconomic indicators are natively published, a fact reflected in FRED-MD [11], which adopts monthly frequency for coverage and relevance.

- Granularity is fine enough to capture seasonal patterns lost at annual frequency and coarse enough to provide sufficiently long contexts for meaningful temporal modelling.
- At this frequency, series are updated frequently enough that recent observations remain available, which matters when comparing against foundation models whose pretraining set may already cover annual data up to the current year but are less likely to include the most recent months.

The construction of a dataset at the scale of FRED’s full monthly catalogue quickly exposed a fundamental limitation of the v1 API: all endpoints are series-level, meaning every request operates on a single series identifier. Retrieving observations for one series typically required approximately five seconds end-to-end, including network latency and the library overhead. Even with parallelisation, FRED enforces an undocumented but empirically observed rate limit of roughly 120 requests per minute, beyond which requests return HTTP 429 errors. Under these constraints, a full retrieval of approximately 200,000 monthly series available in FRED would have taken an estimated³ 38 hours under the conditions of this project, and would only grow as FRED continues to expand its catalogue or by an exploration of a different subset. This is an impractical ceiling for any research workflow intended to be reproducible or updated over time.

To overcome this, the v2 of the FRED API was adopted. Unlike its predecessor, the v2 exposes a single bulk endpoint specifically designed for large-scale extraction: `/fred/v2/release/observations`, which returns all observations for all series belonging to a given release in a single paginated response. This design maps naturally onto the release structure of FRED: since series identifiers are unique to their release and no series is shared across releases, iterating over all releases and issuing one bulk request per release is both complete and non-redundant. In addition to the numeric observations, each response record includes a subset of per-series metadata: `title`, `units`, `frequency`, and `notes`, providing partial multimodal information at no extra cost.

The retrieval pipeline therefore consisted of two steps:

1. First, all available release identifiers were fetched from the `/fred/releases` endpoint, which ended up being exactly 322 on the 11th of March 2026.
2. Second, for each release, a bulk request was issued to the v2 endpoint, and the response was filtered to retain only series with a `frequency` field equal to "Monthly". Since some releases contained significantly more observations than the per-response limit, cursor-based pagination was used to iterate through all pages, with each page appended incrementally to a per-release Parquet⁴ file to avoid holding large pay-

³Estimation based on average processing time of a subset of size 1,000.

⁴Parquet was chosen for its columnar storage, efficient compression, and native support for nested types. It has become the standard for large-scale dataset distribution, adopted widely in industry over platforms such as HuggingFace Datasets.

loads in memory.

Finally, this extraction results in one Parquet file per release, each containing all monthly observations for that release along with their associated metadata fields. The total number of monthly series gathered is: 219,346. This per-release structure avoids loading the full dataset into memory at once, making the subsequent preprocessing steps both tractable and fault-tolerant; if a release fails or needs to be re-fetched, only that file is affected. The schema of each resulting file is summarised in Table 3.1.

TABLE 3.1. SCHEMA OF THE PER-RELEASE PARQUET FILES
PRODUCED BY THE UNIMODAL RETRIEVAL PIPELINE.

Column	Type	Description
series_id	string	Unique FRED series identifier (e.g., CPIAUCSL)
date	datetime	Observation timestamp (monthly, day-normalised to 1st)
value	float64	Numeric observation
title	string	Human-readable series name assigned by FRED analysts
units	string	Unit of measurement (e.g., <i>Thousands of Persons</i> , <i>Millions of USD</i> , or index notation such as <i>Index 1982-84=100</i> for series normalised to a base period)
notes	string	Methodological notes; it could be empty

Rate limiting was handled reactively: upon receiving a HTTP 429 response, the pipeline paused for 20 seconds before retrying, and already-processed releases were tracked in a skip-set to make the pipeline fully resumable across interruptions.

3.2.2. Multimodal Data: Metadata and Tags

The unimodal retrieval pipeline described above did not only produce numeric observations. As shown in Table 3.1, each record in the per-release Parquet files already carries a set of metadata fields such as `title`, `units`, `frequency` and `notes` which are returned by the v2 endpoint at no additional cost. These fields constitute the first layer of multimodal information available for each series and are reused directly in the multimodal pipeline.

While these metadata fields already provide useful context, FRED’s tag system offers a complementary and more structured layer of information: concise, analyst-assigned labels that characterise each series along dimensions such as economic domain, geographic scope, and seasonal adjustment status, enriching the metadata associated to each series.

Tags in FRED are short, analyst-assigned labels that categorise each series along multiple dimensions simultaneously. A series like the Consumer Price Index might carry tags such as `usa`, `monthly`, `cpi`, `inflation`, and `seasonally adjusted`, each independently conveying a meaningful piece of information. Beyond the tag name itself, FRED’s

tag object exposes an optional `notes` field which, when present, contains a longer description of what the tag means. This is particularly useful for abbreviated or potentially ambiguous tags such as `nsa` or `gdp`, which are clear to an economist but carry real risk of misinterpretation by a text encoder. Whenever a tag’s `notes` field is available, it is used in place of the raw tag name during retrieval, replacing the abbreviation with a more explicit description. The schema of a tag object as returned by the API is summarised in Table 3.2.

TABLE 3.2. FIELDS OF A FRED TAG OBJECT RETURNED BY THE `/FRED/SERIES/TAGS` ENDPOINT.

Field	Type	Description
<code>name</code>	string	Short tag label (e.g., <code>cpi</code> , <code>usa</code> , <code>nsa</code>)
<code>group_id</code>	string	Broad category the tag belongs to (e.g., <code>geo</code> , <code>seas</code>)
<code>notes</code>	string	Extended description; present for ambiguous or abbreviated tags
<code>created</code>	string	Timestamp of tag creation
<code>popularity</code>	integer	Usage count across FRED series
<code>series_count</code>	integer	Number of series carrying this tag

Unlike the unimodal observations, there is no bulk endpoint available for tag retrieval, therefore each series requires an individual request to the `/fred/series/tags` endpoint of the version 1, so it brings back the same rate-limiting constraints as the v1 API. To handle this at scale across the roughly 200,000 monthly series in the dataset, an asynchronous retrieval pipeline was implemented using Python’s `asyncio` and `aiohttp` libraries. The key design decisions are outlined below:

- **Concurrency with rate control.** A sliding-window `RateLimiter` enforces a ceiling of 120 requests per minute, matching FRED’s empirically observed limit. Up to 20 concurrent connections are allowed simultaneously via an `asyncio.Semaphore`, maximising throughput.
- **Fault tolerance.** Each request is retried up to 5 times with exponential backoff on server errors (5xx) and a 60-second wait on explicit rate-limit responses (429).
- **Resumability.** Processed series IDs are persisted to a checkpoint file after every batch of 5,000 series, so the pipeline can be safely interrupted and resumed without re-fetching already completed work.
- **Tag expansion.** A `tags_notes_lookup` dictionary is built and updated incrementally during retrieval. Whenever a tag carries a non-trivial `notes` value, the tag name is replaced by that expanded description before saving, so the final output contains human-readable text rather than raw abbreviations.

The result of this process is a single Parquet file containing one row per series, with tags stored as a comma-separated string after the expansion step. This file is subsequently merged with the unimodal metadata to form the complete multimodal view of each series. Two representative samples of the merged dataset are shown in Table 3.3.

TABLE 3.3. TWO SAMPLE ROWS OF THE MERGED
MULTIMODAL DATASET (METADATA + EXPANDED TAGS).

series_id	title	units	tags
CPIAUCSL	Consumer Price Index for All Urban Consumers	Index 1982-84=100	United States of America, monthly, consumer price index, inflation, seasonally adjusted, Bureau of Labor Statistics, nation
UNRATE	Unemployment Rate	Percent	United States of America, monthly, unemployment, labor market, seasonally adjusted, Bureau of Labor Statistics, nation, rate

3.3. Data Preprocessing

Despite FRED’s reputation as one of the most curated and reliable sources of macroeconomic data, the raw extracted dataset is not directly usable for training a forecasting model. Two structural issues arise naturally from the heterogeneous nature of FRED’s catalogue. First, while the majority of series are well-populated, some contain isolated missing values scattered across their history, probably as a consequence of reporting gaps, source revisions, or delayed publication by the originating agency. Second, and more fundamentally, series in FRED span vastly different historical periods. Some indicators, such as the major national aggregates, date back to the 1940s or 1950s; others are recent additions covering niche regional or sectoral phenomena, sometimes with only a handful of observations. A concrete example of the latter is USPTOISSUED051155 — *New Patent Assignments in Pulaski County, VA* a monthly series with only five observations, running from September 2000 to January 2001 (see Figure 3.2). Series of this kind are simply not usable for any meaningful temporal modelling: they are too short to provide a meaningful context window for holdout evaluation, and they complicate normalisation across the dataset.

These two issues, missing values and series that are too short to provide a valid context window, motivate the preprocessing pipeline applied to the raw Parquet files prior to model training. The pipeline is structured in three sequential steps, each implemented as a separate pass over the data to avoid loading the full dataset into memory at once, given the scale of the extracted corpus.

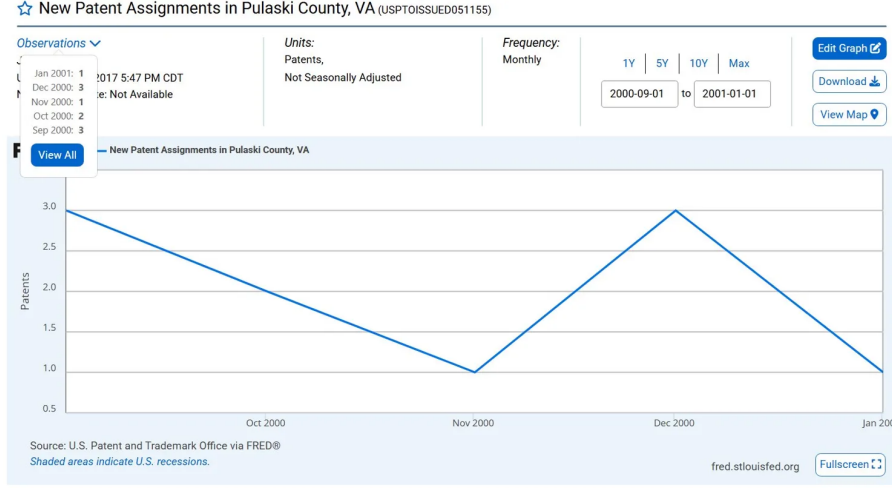


Fig. 3.2. Example of an extremely short FRED series (USPTOISSUED051155, New Patent Assignments in Pulaski County, VA), spanning only five monthly observations. Series of this nature are excluded before training.

3.3.1. Unimodal Preprocessing Pipeline

A key requirement for training the forecasting model is that every series used must satisfy two conditions simultaneously: it must provide a minimum context window of observations for the model to learn from, and all series must share a common end date, so that training windows are temporally aligned across the dataset to avoid data leakages. The three steps below operationalise these requirements.

Step 1 — Cutoff date selection and context filtering

The goal of this step is to select a fixed-length context window, defined by a context length Δ_{ctx} and an end date d^* , such that the number of series with complete, valid observations over that window is maximised. This avoids imposing an arbitrary cutoff and instead lets the data determine the most inclusive time frame.

For each series, the true first and last valid observation dates are computed by ignoring leading and trailing NaN values (which sometimes can appear at the edges of a series due to late reporting or discontinued publication). Using these bounds as a proxy for boundary imputation would mean fabricating data at the extremes, which is avoided.

The optimal cutoff date is then found by exhaustive search over all candidate month-start dates. For each candidate d , the number of series that have at least Δ_{ctx} months of valid data ending at or before d is counted. Formally, a series qualifies if:

$$\text{first_valid} \leq d - \Delta_{\text{ctx}} \quad \text{and} \quad \text{last_valid} \geq d \quad (3.1)$$

where Δ_{ctx} denotes the context length in months. Figure 3.3 illustrates this criterion: series that start after $d^* - \Delta_{\text{ctx}}$ or end before d^* are rejected, while those covering the full window

qualify. The right panel shows $C(d)$, the count of qualifying series across all candidate dates, with the optimal d^* highlighted. The date d^* that maximises this count is selected as the cutoff, as formalised in Algorithm 1.

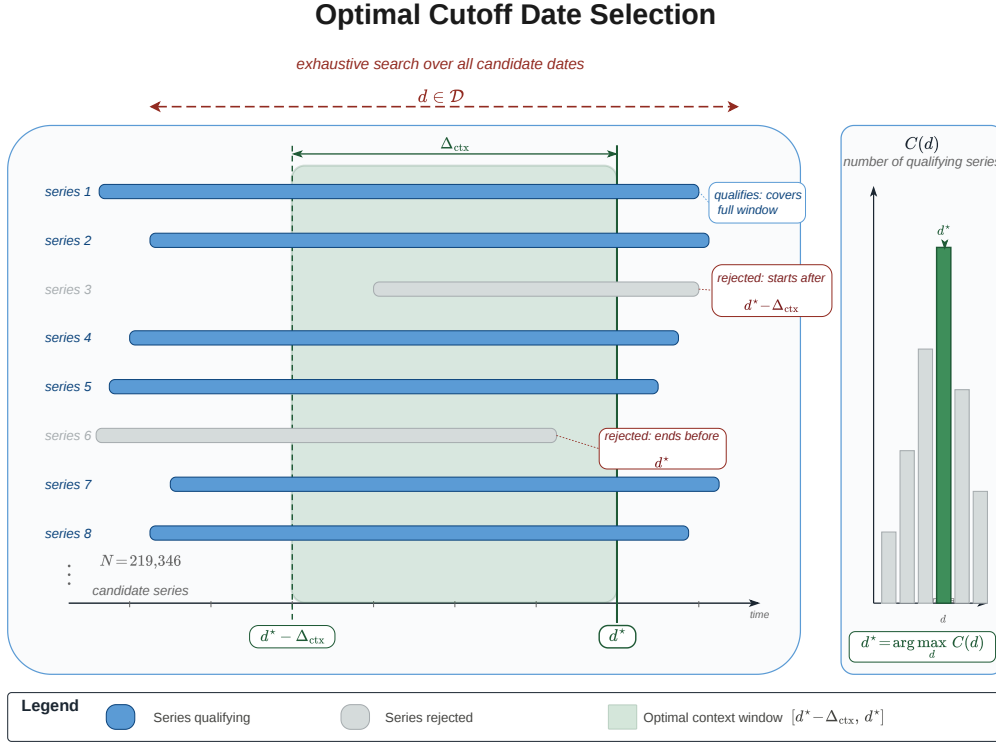


Fig. 3.3. Optimal cutoff date selection. Each row represents a candidate series; blue series cover the full context window $[d^* - \Delta_{\text{ctx}}, d^*]$ and qualify, while grey series are rejected for starting too late or ending too early. The right panel shows $C(d)$, the number of qualifying series for each candidate date d , with the optimal d^* corresponding to the peak.

Applied to the extracted dataset with $\Delta_{\text{ctx}} = 48$ months, this procedure identifies a cutoff of December 2021, with a context window spanning January 2018 to December 2021. The choice of 48 months responds to two criteria: it exceeds the minimum of 24 observations typically required for reliable parameter identification in classical statistical models such as ARIMA, while remaining short enough to retain as many series as possible from the original catalogue. Out of the original 219,346 monthly series, 198,116 pass this filter.

Step 2 — Missing value filtering and edge trimming

After the cutoff is fixed, each retained series is sliced down to its context window $[d^* - \Delta_{\text{ctx}}, d^*]$, and all subsequent operations in this step and the next one apply exclusively to that window: observations outside the window are not used for any filtering or imputation decision.

Retaining series with a small number of scattered missing values inside the window is acceptable, provided those gaps can be filled without introducing substantial fabricated

Algorithm 1 Selection of the optimal cutoff date

Require: Series validity bounds $\mathcal{S} = \{(f_i, \ell_i)\}_{i=1}^N$, where f_i and ℓ_i denote the first and last valid timestamps of series i ; context length Δ_{ctx}

1: Define the set of feasible cutoff dates:

$$\mathcal{D} = \left[\min_i(f_i) + \Delta_{\text{ctx}}, \max_i(\ell_i) \right]$$

sampled at monthly frequency.

2: **for** each candidate cutoff date $d \in \mathcal{D}$ **do**

3: Count the number of series with a complete context window:

$$C(d) = |\{i : f_i \leq d - \Delta_{\text{ctx}} \wedge \ell_i \geq d\}|$$

4: **end for**

5: Select the cutoff maximizing the number of usable series:

$$d^* = \arg \max_{d \in \mathcal{D}} C(d)$$

6: Define the final context window:

$$[d^* - \Delta_{\text{ctx}}, d^*]$$

7: Retain only the series satisfying:

$$f_i \leq d^* - \Delta_{\text{ctx}} \quad \wedge \quad \ell_i \geq d^*$$

signal. Series with a high proportion of missing values, however, may degrade model training regardless of the imputation strategy applied. A threshold of 25% maximum missingness within the window is applied: any series for which more than a quarter of its observations inside $[d^* - \Delta_{\text{ctx}}, d^*]$ are NaN is discarded.

The selection of this threshold is guided by the literature regarding missing data, which finds that standard imputation yields valid inferences only when missingness is small enough and performance deteriorates as missingness increases [40]. In macroeconomic and financial time series, PCA based imputation studies show that predictive accuracy declines beyond about 20% missingness, suggesting that higher proportions should be treated with caution [41].

This step then eliminates 3,330 additional series, leaving 194,786.

A secondary edge-trimming pass removes series that, within the window, still carry a NaN at either the first or the last position of the window. These boundary NaN cannot be filled by forward fill: the first position has no past observation inside the window to copy from, and the last position would only be reachable through extrapolation, which would silently introduce fictitious data at the very positions the model is trained on. Critically, no observation from outside the window is used to fill these boundary positions, even when one is available in the raw series, since allowing it would break the per-window

self-contained design of the pipeline. This pass eliminates only 344 series, bringing the retained count to 194,442.

Step 3 — Imputation

The remaining interior missing values, those that fall strictly inside the window $[d^* - \Delta_{\text{ctx}}, d^*]$ and are surrounded by valid observations on both sides within that same window, are filled using forward fill (LOCF, Last Observation Carried Forward). LOCF is a well established pragmatic choice for time series imputation, adopted in large scale repositories such as the Monash Forecasting Archive [9], particularly when missing intervals are short and the underlying process is stable. Its key advantage is that it introduces no look ahead bias, as the filled value is always drawn from a past observation, and it avoids the instability that more sophisticated imputation methods can introduce when applied at scale. More advanced alternatives exist in the literature [42], but their added complexity is difficult to justify when missingness is already low after the previous filtering steps. No series are discarded at this stage; all 194,442 retained series pass through unchanged in terms of count.

The processed observations are written back to per-release Parquet files, mirroring the structure of the raw extraction and keeping the memory footprint of each processing step bounded. All three steps are executed sequentially and in a single pass per file to avoid holding multiple releases in memory simultaneously.

3.3.2. Preprocessing Pipeline for the Multimodal information

Compared to the unimodal pipeline, the multimodal preprocessing is considerably more straightforward. The input is purely textual: a set of metadata fields per series and the transformations required are minimal. No filtering, no imputation, and no windowing are needed; the only objective is to organise the available fields into a form that a pretrained language encoder can process effectively.

It is important to note that this preprocessing step is specific to the modelling pipeline described in this work. The dataset that will be made publicly available retains the raw metadata fields separately (title, units, notes, and the expanded tag string), so that other researchers can apply their own text formatting, normalisation, or encoding strategies without being constrained by the choices made here.

Metadata sentence construction

The multimodal input for each series is constructed as a short natural-language paragraph that concatenates the available metadata fields into a single string. The rationale is that a pretrained encoder operates on free text and benefits from input that resembles natural language. The template used is:

Time series about {title}. It is associated with the tags: {tags}. Measured in {units}.

Each clause is included only if the corresponding field is present and non-empty after cleaning. Fields containing NaN, empty strings, or placeholder values ("none", "NA") are silently dropped, so the resulting sentence is always formed from real information. The construction logic is summarised in Figure 3.4.

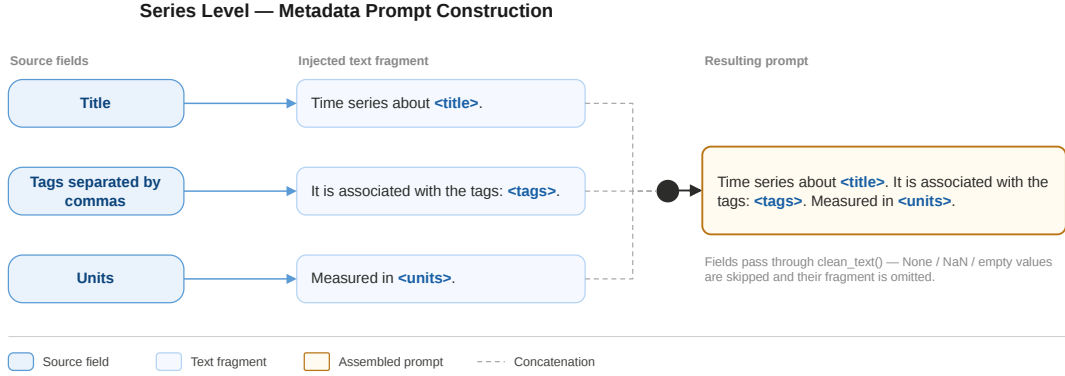


Fig. 3.4. Series-Level Metadata Prompt Construction

Two concrete examples of the resulting sentences are shown below; note that the `units` field is preserved verbatim from FRED [4], including its native notation for index-type series (e.g., *Index 1982-84=100*, denoting a price index normalised to 100 over the 1982-1984 base period)

- **CPIAUCSL**: “*Time series about Consumer Price Index for All Urban Consumers. It is associated with the tags: headline figure, all items, urban, Consumer Price Index, price index, inflation, Seasonally Adjusted, Bureau of Labor Statistics. Measured in Index 1982-84=100.*”
- **UNRATE**: “*Time series about Unemployment Rate. It is associated with the tags: headline figure, civilian, Current Population Survey Household Survey, unemployment, rate, Seasonally Adjusted, Bureau of Labor Statistics. Measured in Percent.*”

This approach shares its general motivation with MetaTST [5] as described in Chapter 2, which similarly encodes dataset-level and variate-level metadata as natural language into a Transformer forecasting model. However, whereas MetaTST relies on LLM-generated textual descriptions at the dataset level, the approach followed here uses analyst-curated, series-level metadata directly sourced from FRED providing richer and more specific context for each individual series.

Embedding Generation

Once the natural-language sentence for each series has been constructed, the next step is to convert it into a fixed-dimensional vector representation that a neural network can

consume. This is the standard role of a text encoder: map a variable-length string to a dense embedding in a continuous space that captures its semantic content.

Since training a text encoder from scratch falls outside the scope of this work, a pre-trained model from the state of the art is reused for this purpose. The choice of encoder is informed by the ablation study reported in MetaTST [5], which evaluates several pre-trained LLMs, including decoder-only (such as GPT-2, Llama) and encoder-only (such as BERT, T5) as metadata encoders for time-series forecasting. As shown in Table 10 of that work, **T5** achieves the best average performance across datasets (MAE = 0.258), outperforming BERT and all decoder-only variants on the majority of benchmarks. This result aligns with the general finding that encoder-only or encoder-decoder architectures tend to produce richer contextual representations for short descriptive texts than decoder-only models, which are optimised for generation rather than representation [5], [43].

T5: Text-To-Text Transfer Transformer. T5 is a Transformer-based model introduced by Raffel et al. [43] at Google Research. Its defining characteristic is a unified text-to-text framework: every NLP task (translation, summarisation, classification) is cast as mapping an input text string to an output text string, which allows a single architecture and training objective to cover a wide range of tasks. T5 is pretrained on the Colossal Clean Crawled Corpus (C4) using a masked span corruption objective, where contiguous spans of tokens are replaced with sentinel tokens and the model learns to reconstruct them. Tokenisation is handled by a SentencePiece subword tokeniser, which operates on raw text without requiring language-specific preprocessing. The **t5-base** variant used here has 220 million parameters and a maximum input length of 512 tokens — well above the length of the metadata sentences constructed in this work, so no chunking or truncation beyond the 128-token limit set in the implementation is needed in practice.

Encoding procedure. Only the encoder stack of T5 is used, discarding the decoder entirely. The encoder processes a tokenised and padded batch of sentences and produces a sequence of contextual hidden states, one per input token. To obtain a single fixed-size vector per sentence, mean pooling is applied over the token dimension, weighted by the attention mask to exclude padding positions:

$$\mathbf{m} = \frac{\sum_{t=1}^T m_t \mathbf{h}_t}{\sum_{t=1}^T m_t} \quad (3.2)$$

where $\mathbf{h}_t \in \mathbb{R}^{d_m}$ is the hidden state at position t , $m_t \in \{0, 1\}$ is the corresponding attention mask value, and $d_m = 768$ for **t5-base**. The model runs in inference mode with gradients disabled, and sentences are processed in batches of 32 to keep GPU memory usage low. The resulting embedding matrix has shape (N, d_m) , where N is the number of series, and is serialised to disk as a Pickle file. The embeddings are precomputed once and remain frozen throughout all downstream model training: the forecasting models in

this work consume them as fixed inputs rather than recomputing them on the fly. This design decouples the language understanding capacity, fully delegated to the pretrained T5 encoder, from the forecasting model itself, which only learns to use the semantic representation rather than to produce it. In this sense, $\mathbf{m} \in \mathbb{R}^{d_m}$ is a rich, fixed semantic fingerprint of each time series.

3.4. Final Dataset Structure

The preprocessing steps described above produce two distinct dataset artefacts, serving different purposes: one tailored to the specific modelling pipeline of this work, and one designed for public release and reuse by the broader research community.

3.4.1. Dataset for This Work

The dataset used for training and evaluating the models in this work is a compact, model-ready structure. For each of the 194,442 retained series, it contains three elements summarised in Table 3.4.

TABLE 3.4. SCHEMA OF THE MODEL-READY DATASET USED FOR TRAINING AND EVALUATION.

Column	Type	Description
series_id	string	Unique FRED series identifier
observations	list[float32]	Preprocessed monthly observations over the context window (Jan 2018 – Dec 2021)
embedding	float32[768]	T5 encoder output from the meta-data sentence

These three components are all that the model requires at training time, and keeping the dataset in this reduced form minimises I/O overhead during the training loop.

3.4.2. Dataset for Publication

Beyond its use in this work, the dataset is intended for public release as a standalone contribution. The motivation is basic: no large-scale, publicly available macroeconomic time-series dataset currently provides structured multimodal metadata at the individual series level that is, analyst-curated descriptive information specific to each series rather than to the dataset as a whole. Making this resource available in a clean, standardised format would lower the barrier to entry for future research in multimodal time-series modelling.

The public release format follows industry-standard conventions for large tabular datasets. Each row corresponds to one series, and the full observation history is stored as a list rather than in long format, keeping the file compact and self-contained. The schema is defined using Apache Arrow and serialised as a Parquet file with `zstd` compression at level 7, which offers a good balance between compression ratio and read speed:

TABLE 3.5. SCHEMA OF THE PUBLIC RELEASE PARQUET FILE.

Column	Type	Description
<code>series_id</code>	<code>string</code>	Unique FRED series identifier
<code>title</code>	<code>string</code>	Human-readable series name
<code>observations</code>	<code>list[float32]</code>	Full observation history as a numerical array
<code>units</code>	<code>dict[int32, string]</code>	Unit of measurement (dictionary-encoded)
<code>notes</code>	<code>string</code>	Methodological notes from FRED
<code>start_date</code>	<code>timestamp[ns]</code>	Date of the first observation
<code>tags</code>	<code>list[string]</code>	Analyst-assigned tags (expanded where applicable)
<code>frequency</code>	<code>dict[int32, string]</code>	Reporting frequency (dictionary-encoded)

The raw metadata fields are retained separately and in their original form; no sentence construction, no embedding, no filtering beyond the monthly frequency constraint. This is a deliberate choice: different research groups may want to encode the textual modality differently, apply their own cleaning criteria, or use only a subset of the available fields. Providing the raw components rather than a preprocessed version maximises the dataset’s reusability and avoids locking future users into the design decisions of this work.

The dataset will be made publicly available and the intention is to host it on a standard platform such as HuggingFace Datasets [44] to ensure long-term accessibility and versioning.

4. METHODOLOGY

This chapter introduces the formal foundations upon which the proposed model is built. It begins by defining the multimodal time series forecasting problem in a rigorous notation that will be used throughout the remainder of this work. From there, each of the three core conceptual building blocks of the architecture is introduced: the neuro-symbolic basis that gives the model its interpretable forecasting structure, the Concept Bottleneck mechanism that serves as the interface between the two modalities, and the Feature-wise Linear Modulation (FiLM) layer that allows the metadata representation to intervene directly on the symbolic parameters of the forecast.

4.1. Problem Formulation: Multimodal Time Series Forecasting

4.1.1. Task Definition

The core task addressed in this work is univariate time series forecasting under a multimodal input setting. Given a historical sequence of scalar observations and a set of textual metadata describing the series, the goal is to produce a forecast over a fixed future horizon.

Formally, let $\mathbf{x} = (x_1, x_2, \dots, x_T) \in \mathbb{R}^T$ denote a univariate time series of length T . Given a context window of L observations ending at time T where $L \ll T$,

$$\mathbf{x}_{\text{ctx}} = (x_{T-L+1}, \dots, x_T) \in \mathbb{R}^L, \quad (4.1)$$

the objective is to predict the next H values (horizon values), where H as a whole is referred to as the forecast horizon:

$$\hat{\mathbf{y}} = (\hat{x}_{T+1}, \dots, \hat{x}_{T+H}) \in \mathbb{R}^H. \quad (4.2)$$

In an unimodal setting, the model would only have access to $\mathbf{x}_{\text{ctx}}$ to produce $\hat{\mathbf{y}}$. In the multimodal setting considered here, each series is additionally associated with a static metadata descriptor $\mathbf{m} \in \mathbb{R}^{768}$, the precomputed T5 embedding of the natural-language description built for that series in Subsection 3.3.2. Although $\mathbf{m}$ is static with respect to time, it varies across series since every series in the dataset carries its own descriptor. The forecasting function is therefore conditioned on both the numeric history and the metadata associated with each individual series, and the full forecasting problem is:

$$\hat{\mathbf{y}} = f(\mathbf{x}_{\text{ctx}}, \mathbf{m}), \quad (4.3)$$

where f is the model to be learned. As discussed in Chapter 3, the dataset consists entirely of monthly macroeconomic series from FRED, so a horizon of $H = 18$ months is adopted, covering one and a half years of future observations. This length is long enough to capture a complete annual seasonal cycle and still observe a second partial cycle, which is particularly informative for evaluating how well the model extrapolates periodic structure beyond the training window. Context lengths $L \in \{8, 10, 15, 24, 30\}$ are evaluated, with $L_{\max} = 30$ being the maximum used during training. This range is deliberately wide: at the lower end ($L = 8$, roughly two thirds of a year), the numeric history alone provides very limited signal, which is precisely the regime where metadata is expected to contribute the most. At the upper end ($L = 30$, two and a half years), the model has access to enough history to identify seasonal patterns on its own, making it a natural baseline to measure how much the multimodal information still adds when context is abundant.

4.1.2. Input Normalisation and Padding

FRED time series span vastly different scales and units: GDP figures in trillions of dollars alongside unemployment rates in percentages, for instance. To ensure that the numerical range of the context window does not dominate the learning signal, a per-instance min-max normalisation is applied at the window level:

$$\tilde{x}_t = \frac{x_t - \min(\mathbf{x}_{\text{ctx}})}{\max(\mathbf{x}_{\text{ctx}}) - \min(\mathbf{x}_{\text{ctx}}) + \varepsilon}, \quad \varepsilon = 10^{-5}, \quad (4.4)$$

where the statistics are computed exclusively from the context window $\mathbf{x}_{\text{ctx}}$ and then applied to normalise both the context and the horizon. This instance-level normalisation, also referred to as reversible instance normalisation in related literature [45], avoids any leakage of future information and allows the model to operate in a scale-free space. The original scale can be recovered at inference time using the stored min and range values. Normalisation is applied per context window at data loading time, so the model always receives scale-free inputs by construction regardless of the original units of the series. To handle the degenerate case of a constant context window, the denominator is lower-bounded by $\varepsilon = 10^{-5}$, preventing division by zero.

Since series in the dataset vary in length within a given experiment, shorter context windows are left-padded with a padding value of -1 to reach the target length L . A binary mask $\mathbf{k}_{\text{ctx}} \in \{0, 1\}^L$ tracks which positions correspond to real observations, ensuring that padded positions do not contribute to the loss computation. The model and the loss function operate exclusively on unmasked positions in both cases.

The normalised ground-truth observations over the forecast horizon, $\tilde{y}_h$ for $h = 1, \dots, H$, are additionally clipped to the range $[-7, 8]$ prior to training, which acts as a soft bound against extreme outliers that could otherwise destabilise the gradient signal and the final metrics.

4.1.3. Notation Summary

For reference and easier reading, Table 4.1 summarises the main symbols used throughout this and the following chapters.

TABLE 4.1. NOTATION USED THROUGHOUT THE METHODOLOGY AND ARCHITECTURE CHAPTERS.

Symbol	Description
T	Total length of the observed time series
L	Context (input) window length
H	Forecast horizon length ($H = 18$)
$\mathbf{x}_{\text{ctx}} \in \mathbb{R}^L$	Normalised context window
$\mathbf{y} \in \mathbb{R}^H$	Ground-truth horizon values
$\hat{\mathbf{y}} \in \mathbb{R}^H$	Model forecast
$\mathbf{m} \in \mathbb{R}^{d_m}$	Metadata embedding ($d_m = 768$)
$\mathbf{k} \in \{0, 1\}^L$	Padding mask
$\mathbf{z} \in \mathbb{R}^{d_z}$	Sparse concept vector (bottleneck)
$\boldsymbol{\theta} \in \mathbb{R}^{d_\theta}$	Symbolic basis-function coefficients
$\gamma, \beta \in \mathbb{R}^{d_\theta}$	FiLM modulation parameters

4.2. Neuro-Symbolic Forecasting Basis

A central design decision in this work is to replace the conventional direct-output forecasting head, where a neural network maps a hidden representation straight to a sequence of predicted values, with an interpretable, structured decoder based on symbolic basis functions. Rather than predicting a raw sequence of numbers, the model predicts why the series is expected to behave in a certain way: the forecast is decomposed into an intercept, a trend, and a seasonal pattern, each governed by an explicit, readable coefficient.

Figure 4.1 illustrates the overall structure of this decoder, which serves as the forecasting head for both the unimodal and multimodal models developed in this work.

As discussed in Section 2.4.1, combining neural flexibility with symbolic structure has a solid grounding in the forecasting literature [25]. The basis-function decoder adopted here follows that tradition: by constraining the network to produce forecasts that are decomposable into interpretable components, the model gains transparency by construction without sacrificing the expressive power of a learned representation.

4.2.1. Basis-Function Decoder

The forecasting head is built around a closed-form basis-function equation evaluated over the forecast horizon. Given a set of learnable symbolic coefficients $\boldsymbol{\theta} \in \mathbb{R}^{d_\theta}$ predicted by

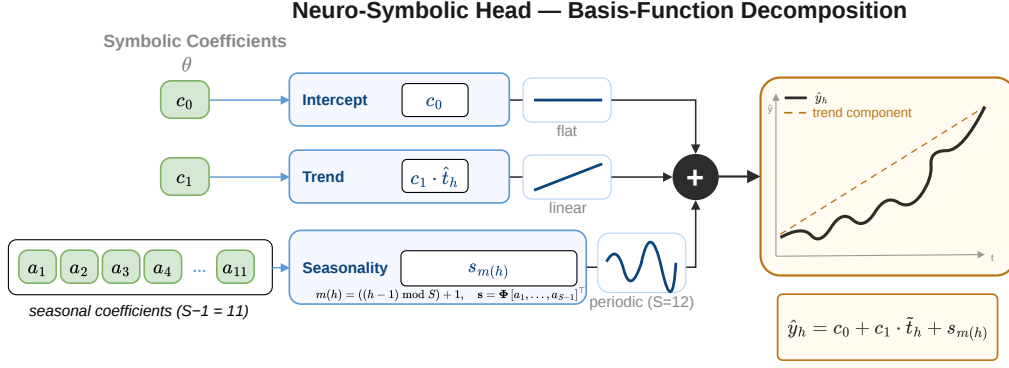


Fig. 4.1. Structure of the neuro-symbolic basis-function decoder.

the network from the encoded context, the forecast at step $h \in \{1, \dots, H\}$ is computed as:

$$\hat{y}_h = \underbrace{c_0}_{\text{intercept}} + \underbrace{c_1 \cdot \tilde{t}_h}_{\text{trend}} + \underbrace{s_{m(h)}}_{\text{seasonality}}, \quad (4.5)$$

where $\tilde{t}_h$ is the standardised time index over the horizon, $m(h) = ((h-1) \bmod S) + 1 \in \{1, \dots, S\}$ is the calendar month that horizon step h falls on, and $s_1, \dots, s_S$ are the seasonal deviations of each month. These S values are not all free: they are produced from $S-1 = 11$ learnable coefficients $a_1, \dots, a_{S-1}$ under a sum-to-zero constraint (described below), so the full coefficient vector is $\theta = [c_0, c_1, a_1, \dots, a_{S-1}] \in \mathbb{R}^{S+1}$, giving $d_\theta = 13$ parameters in total.

Intercept. The coefficient c_0 captures the baseline level of the forecast. Since the context window is min-max normalised prior to encoding, c_0 operates in the normalised scale and is mapped back to the original scale at inference time using the stored statistics.

Trend. The coefficient c_1 governs a linear trend component. The time index is standardised over the horizon as:

$$\tilde{t}_h = \frac{h - \mu_t}{\sigma_t + \varepsilon}, \quad \text{where} \quad \mu_t = \frac{1}{H} \sum_{h=1}^H h, \quad \sigma_t = \sqrt{\frac{1}{H} \sum_{h=1}^H (h - \mu_t)^2}, \quad (4.6)$$

which centres the trend basis around zero and prevents its scale from dominating the other components, keeping the optimisation landscape well-conditioned regardless of the horizon length.

Seasonality. The seasonal component assigns a deviation s_m to each calendar month $m \in \{1, \dots, S\}$. To ensure that the seasonal pattern does not introduce a spurious level shift into the forecast, a sum-to-zero constraint is enforced over a full cycle, $\sum_{m=1}^S s_m = 0$.

This removes one degree of freedom, so the network predicts only $S - 1 = 11$ coefficients $a_1, \dots, a_{S-1}$ and the twelfth monthly value is determined by the rest through $s_S = -\sum_{m=1}^{S-1} s_m$. This is a standard constraint in classical seasonal decomposition [46] and ensures that the intercept c_0 retains its interpretation as the unconditional level of the forecast. The concrete map from the $S - 1$ free coefficients to the S monthly values, a fixed projection matrix Φ , is detailed in Chapter 5.

4.3. Concept Bottleneck Models in the Forecasting Context

The basis-function decoder introduced in the previous section produces interpretable forecasts by decomposing the output into symbolic components. However, on its own, it operates purely on the numeric history of the series which has no mechanism to incorporate the rich semantic information encoded in the metadata embedding $\mathbf{m}$. This section introduces the second conceptual building block of the proposed architecture: a Concept Bottleneck that serves as the structured interface through which the metadata modality influences the forecast.

4.3.1. The Original Concept Bottleneck Model

As conceptually defined in Chapter 2, Concept Bottleneck Models (CBMs), introduced by Koh et al. [26], propose a two-stage prediction pipeline designed to make neural networks interpretable in terms of high-level, human-meaningful concepts. Rather than learning a direct mapping from raw input $\mathbf{x}$ to target y , a CBM first maps the input to an intermediate vector of concepts $\hat{\mathbf{c}} \in \mathbb{R}^k$, and then uses those concepts to produce the final prediction:

$$\hat{y} = f(g(\mathbf{x})), \quad \hat{\mathbf{c}} = g(\mathbf{x}), \quad (4.7)$$

where $g : \mathbb{R}^d \rightarrow \mathbb{R}^k$ is the concept predictor and $f : \mathbb{R}^k \rightarrow \mathbb{R}$ is the task predictor. The key property of this design is that the prediction $\hat{y}$ depends on the input $\mathbf{x}$ exclusively through the bottleneck $\hat{\mathbf{c}}$, which makes the model’s reasoning transparent as long as f is itself interpretable: one can read off which concepts the model activated and, more importantly, intervene on them. As demonstrated in [26], replacing a predicted concept value $\hat{c}_j$ with its true value at test time leads to measurable improvements in task accuracy, a property they call: test-time intervention.

As discussed in Section 2.4.2, this design makes predictions auditable and correctable: one can read off which concepts the model activated and intervene on them at test time to steer the final output [26].

In the original formulation, the concept vector $\hat{\mathbf{c}}$ is trained to align component-wise with human-annotated concept labels $\mathbf{c}$ via an auxiliary concept loss, combined with the task loss:

$$\mathcal{L} = \mathcal{L}_{\text{task}}(f(\hat{\mathbf{c}}), y) + \lambda \sum_{j=1}^k \mathcal{L}_{c_j}(\hat{c}_j, c_j). \quad (4.8)$$

The hyperparameter λ controls the trade-off between concept alignment and task performance. Setting $\lambda \rightarrow 0$ recovers a standard end-to-end model, while $\lambda \rightarrow \infty$ forces the bottleneck to match the concepts exactly before any task signal is considered.

4.3.2. Adaptation to the Forecasting Setting

The original CBM framework requires concept annotations at training time that is, for each input, a human expert must provide the ground-truth values of the K concepts. In the time series forecasting setting considered here, no such annotations are available. This motivates the use of automatically discovered concepts through a sparsity-inducing mechanism, in the spirit of self-supervised sparse representations.

The adaptation proposed in this work departs from the original CBM in three important ways, described here conceptually and detailed architecturally in Chapter 5.

Concepts as an inductive bias for editability. The primary motivation for introducing a sparse bottleneck in this work is editability. By constraining the intermediate representation to have at most K active dimensions out of d_z , the model exposes a small set of adjustable activations that a practitioner can inspect and modify at inference time without needing to interact with the full parameter space of the network. This is conceptually aligned with the symbolic head described in Section 4.2: just as the basis-function coefficients θ can be overridden to correct a forecast, the concept activations $\mathbf{z}$ can be adjusted to steer how the metadata modulates those coefficients. The sparsity constraint is therefore not a means to obtain richer semantic labels for each dimension, but rather a structural choice that keeps the intervention space tractable.

Concept discovery without supervision. Rather than training the concept predictor to align with annotated labels via a concept loss, the bottleneck produces a sparse concept vector $\mathbf{z} \in \mathbb{R}^{d_z}$ through a Top- K selection mechanism (detailed in Section 5.3). This departs from the original CBM formulation in an important way: in the original framework, all d_z concepts receive a value for every input and the task predictor f combines them in a dense weighted sum. Here, at most K out of d_z dimensions are non-zero for any given input; the remaining $d_z - K$ activations are hard-zeroed.

The value of K is a fixed hyperparameter determined by experimentation and reported in Chapter 5. No auxiliary concept loss is used: the model is trained end-to-end on the forecasting objective alone, and sparsity is enforced structurally through the Top- K selection.

Joint inputs from both modalities. The bottleneck does not act on either modality alone but on a fused representation that combines the encoded time-series hidden state $\mathbf{h} \in \mathbb{R}^{d_{\text{model}}}$ with the metadata embedding $\mathbf{m} \in \mathbb{R}^{d_m}$. Concretely, the two vectors are concatenated and processed by a small projection block Φ_{fuse} consisting of a linear layer, layer normalisation and a non-linear activation, before being passed to the TopK-SAE that produces the sparse concept vector:

$$\mathbf{z} = \text{TopK-SAE}(\Phi_{\text{fuse}}([\mathbf{h} \parallel \mathbf{m}])) , \quad (4.9)$$

where $[\cdot \parallel \cdot]$ denotes concatenation. It is worth clarifying what interpretability means in this context. Unlike the original CBM formulation [26], where each concept dimension is trained to align with a human-annotated semantic label, the dimensions of $\mathbf{z}$ here do not carry predefined names nor correspond directly to input variables. The non-linear projection Φ_{fuse} means that no single concept can be trivially traced back to a specific metadata field or time-series feature. The interpretability claim is therefore of a different, but equally practical, kind: it is interventional rather than semantic. Because $\mathbf{z}$ is sparse and its active dimensions are explicit, a practitioner can inspect which concepts fired for a given series, compare activation patterns across series, and modify individual concept values at inference time to observe how the forecast changes.

Furthermore, although the dimensions of $\mathbf{z}$ are not assigned labels by design, each active concept could in principle be characterised a posteriori by analysing which series systematically activate it: a concept that fires consistently for seasonally adjusted labour market indicators, for instance, would effectively be answering an implicit yes/no question about the series identity, even if no one formulated that question in advance. This kind of emergent concept discovery has been documented in the sparse autoencoder literature [30], [31], and it is a natural direction for future work. The vector $\mathbf{z}$ is then passed to the FiLM layer as the sole channel through which multimodal information reaches the symbolic forecast (Section 4.4).

4.4. Feature-wise Linear Modulation (FiLM)

At this point in the pipeline, two things are available: a set of symbolic coefficients $\theta_0 \in \mathbb{R}^{d_\theta}$ predicted from the numeric history of the series, and a sparse concept vector $\mathbf{z} \in \mathbb{R}^{d_z}$ distilling the joint signal from both modalities. FiLM (Section 2.5) provides a natural and transparent mechanism to let $\mathbf{z}$ intervene on θ_0 , as described in the rest of this section.

4.4.1. FiLM Modulation of the Symbolic Coefficients

In this work, FiLM is applied directly to the symbolic coefficients θ_0 , conditioned on the sparse concept vector $\mathbf{z}$. A small two-layer MLP takes $\mathbf{z}$, the concepts representation, as input and predicts the modulation parameters:

$$(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \text{MLP}_{\text{FiLM}}(\mathbf{z}), \quad \boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_\theta}, \quad (4.10)$$

which are then applied to the base coefficients:

$$\hat{\boldsymbol{\theta}} = \boldsymbol{\gamma} \odot \boldsymbol{\theta}_0 + \boldsymbol{\beta}. \quad (4.11)$$

The modulated coefficients $\hat{\boldsymbol{\theta}}$ are passed to the symbolic head (Equation 4.5) to produce the final forecast. The full forward pass of the model can therefore be summarised as:

$$\hat{\mathbf{y}} = \text{SymbolicHead}(\boldsymbol{\gamma}(\mathbf{z}) \odot \boldsymbol{\theta}_0(\mathbf{h}) + \boldsymbol{\beta}(\mathbf{z})), \quad (4.12)$$

where $\mathbf{h} = \text{TSEncoder}(\mathbf{x}_{\text{ctx}})$ and $\mathbf{z} = \text{TopK-SAE}(W_{\text{fuse}} [\mathbf{h} || \mathbf{m}])$. This single equation captures the entire multimodal pipeline: the numeric history determines the base symbolic coefficients, and the joint representation of history and metadata intervenes on them through a transparent, parameter-wise operation.

Two implementation details are worth noting here. First, the MLP is initialised so that $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$ at the start of training. This means the modulation begins as an identity transformation; the multimodal model starts from exactly the same point as the unimodal one and gradually learns to deviate from it as the metadata signal proves useful. If a series carries no informative metadata, the model can simply leave the coefficients unchanged. Second, because FiLM acts directly on $\boldsymbol{\theta}$ rather than on a dense hidden state, its effect is immediately readable: $\gamma_i > 1$ amplifies the i -th symbolic component, $\gamma_i < 1$ attenuates it, and β_i shifts its baseline.

4.4.2. Why FiLM for This Problem

Beyond the general properties of FiLM discussed in Section 2.5, there is a specific reason why it is particularly well suited to this problem: it modulates parameters, not activations in a black-box layer. Because the features being modulated are the symbolic coefficients $\boldsymbol{\theta}$, each with a direct physical interpretation (intercept, trend slope, seasonal amplitudes), the modulation itself inherits that interpretability. A cross-attention mechanism applied to a dense hidden state might produce equally competitive forecasts, but its effect on the final decomposition would be opaque.

There is also a deeper conceptual alignment with the neuro-symbolic philosophy of this work: the textual modality does not replace the numeric forecast, it intervenes on it and crucially, it can only do so through the structured channel that FiLM provides. This is not a limitation but a deliberate constraint: by forcing the metadata to act exclusively through $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$, the model ensures that the metadata cannot produce arbitrary modifications to the forecast, only affine adjustments to its symbolic components. The metadata says, in effect, how much adjustment is needed in the trend, seasonality, or baseline.

5. MODEL ARCHITECTURE

With the dataset constructed and the forecasting task formally defined in the previous chapters, the remaining piece is the model itself: an architecture capable of producing accurate forecasts while exposing the reasoning behind each prediction. This chapter presents NeuroSym-CBF (*Neuro-Symbolic Concept-Bottleneck Forecaster*), the multimodal neuro-symbolic model proposed in this thesis, and walks through each of its components in turn. The chapter will provide (1) a high-level view of the pipeline, (2) deeper description of each module in detail, (3) the training objective and (4) the training setup and evaluation protocol.

5.1. Architecture Overview

NeuroSym-CBF is built on a simple premise: a univariate forecast based on past values alone can be substantially enriched once the textual metadata describing the series, namely its title, units, notes and tags, is brought into the model. Rather than fusing this information as an opaque vector, NeuroSym-CBF routes it through a small set of concepts that act as switches over the forecast and expose a natural interface for human inspection and intervention. These concepts then condition a neuro-symbolic decoder which, instead of producing the prediction directly, adjusts the coefficients of a closed-form combination of intercept, trend and seasonality basis functions.

Concretely, the pipeline reads as follows: a time-series encoder summarises the past context window, a metadata encoder produces an embedding of the series description, both representations are fused and compressed into a sparse concept bottleneck, and the resulting concepts modulate, via FiLM, the coefficients of the symbolic basis functions whose weighted sum yields the final forecast. Each component is detailed in the sections that follow; Figure 5.1 serves as the visual reference.

5.2. Time Series Encoder

The time-series encoder of NeuroSym-CBF is intentionally simple. Its role is not to be the centrepiece of the model, but to produce a faithful and compact representation of the past context window so that the downstream multimodal fusion and concept bottleneck can operate over a stable input.

The encoder has three stages, illustrated in Figure 5.2: patching of the input window, linear projection to the model dimension, and a stack of residual blocks. Each stage is described in the following subsections.

The same encoder is applied independently to every series, since NeuroSym-CBF

NeuroSym-CBF Architecture

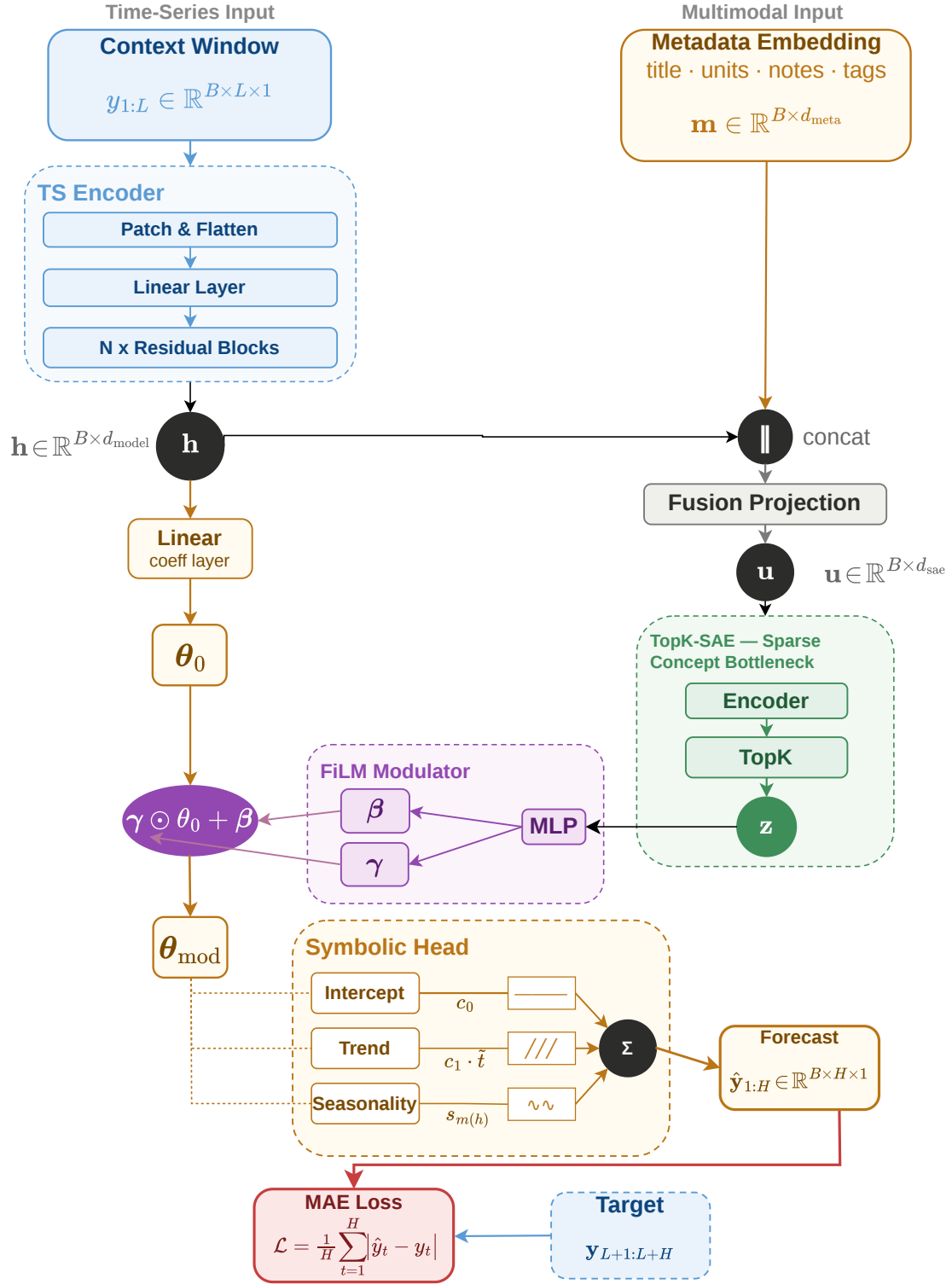


Fig. 5.1. End-to-end overview of the NeuroSym-CBF architecture. Here B denotes the batch size.

treats forecasting as a univariate problem and lets all cross-series context enter through the metadata modality rather than the numerical channel. Its weights are shared across the entire collection, so a single set of parameters has to capture the dynamics of series as heterogeneous as those gathered from FRED. This is part of why the encoder is kept small: it is asked only to summarise each window, leaving the harder modelling to the stages that follow.

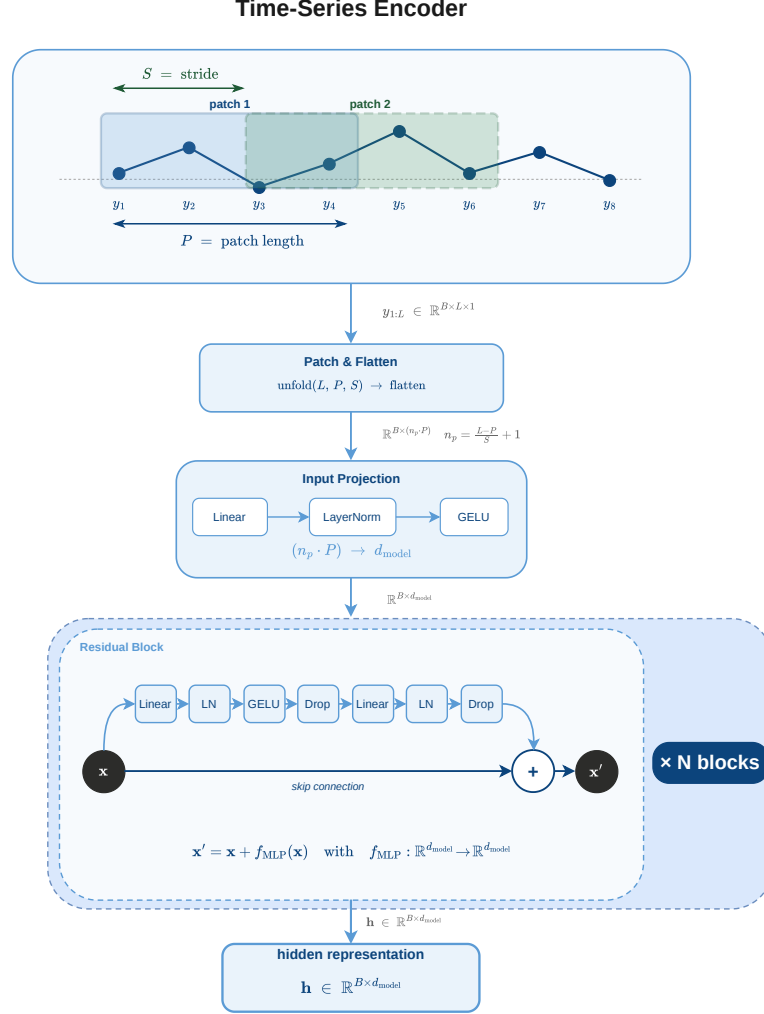


Fig. 5.2. Time-series encoder of NeuroSym-CBF. The univariate context window is split into overlapping patches, flattened, projected to a d_{model} -dimensional space and refined by a stack of N residual blocks, producing a single hidden representation $\mathbf{h} \in \mathbb{R}^{B \times d_{\text{model}}}$.

5.2.1. Patch and Flatten

Following the patching idea introduced in PatchTST [47], the input series is not consumed value by value but in small overlapping windows. Given the normalised context $\mathbf{x}_{\text{ctx}} \in \mathbb{R}^L$ defined in Section 4.1, the encoder applies an **unfold** operation with patch length P and stride s :

$$\mathbf{X}_p = \text{unfold}(\mathbf{x}_{\text{ctx}}, P, s) \in \mathbb{R}^{n_p \times P}, \quad n_p = \frac{L-P}{s} + 1, \quad (5.1)$$

which produces n_p patches per series. The motivation for patching is twofold. On the one hand, it raises the granularity at which the model reasons: each token now represents a short fragment of the series rather than a single point, which provides local context and reduces the depth at which the network has to operate. On the other hand, it shortens the sequence by a factor of s , which is precisely what makes the residual encoder tractable and stable.

In our configuration we use $P = 4$ and $s = 2$, that is, patches of four points with a 50% overlap. These values were selected by manual search over a small set of candidate configurations, namely $(P, s) \in \{(4, 2), (6, 3), (12, 6)\}$, evaluated informally during the development phase on a subset of the training data; the $(4, 2)$ configuration consistently produced the best trade-off between error and training stability. An additional practical advantage is that $P = 4$ divides evenly into most of the evaluated context lengths $L \in \{8, 10, 15, 24, 30\}$, which minimises boundary effects at the edges of shorter windows. The overlap is an intentional choice: a non-overlapping tiling ($s = P$) would create artificial boundaries between consecutive patches and mask short-range patterns that straddle two patches, while a finer stride ($s < P/2$) would inflate n_p without adding information [47].

It must be noted that, unlike PatchTST [47], NeuroSym-CBF does not embed each patch independently and feed the resulting sequence into a Transformer. Instead, the matrix of patches is flattened into a single vector

$$\mathbf{p} = \text{flatten}(\mathbf{X}_p) \in \mathbb{R}^{n_p \cdot P}, \quad (5.2)$$

and processed jointly by the rest of the encoder. The patching step is therefore borrowed as a representational tool, not as the gateway to an attention mechanism. Figure 5.3 illustrates the three stages of this transformation.

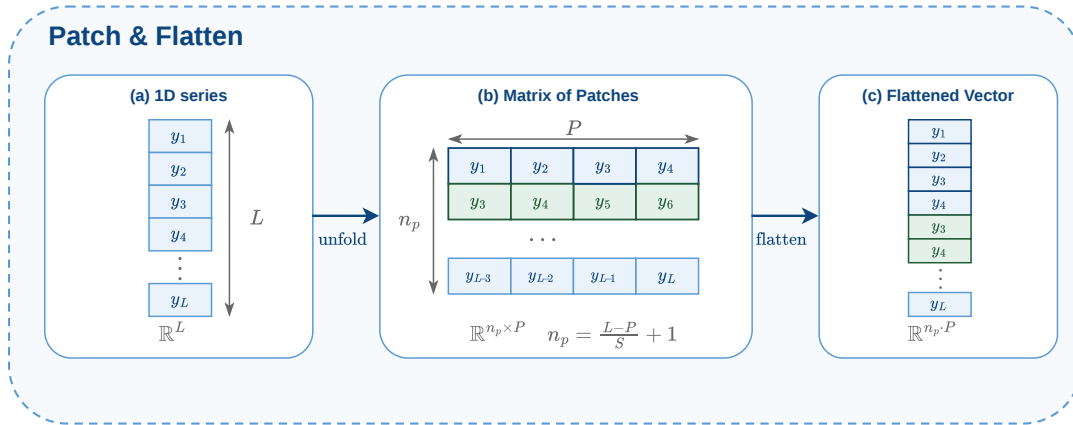


Fig. 5.3. The univariate series of length L (a) is unfolded into a matrix of n_p overlapping patches of length P (b), and the resulting matrix is flattened into a single vector of dimension $n_p \cdot P$ (c). Coloured cells highlight a pair of consecutive overlapping patches.

5.2.2. Input Projection

The flattened vector is projected into the model dimension d_{model} by a small block consisting of a linear layer, layer normalisation and a GELU activation:

$$\mathbf{h}^{(0)} = \text{GELU}(\text{LN}(W_{\text{in}} \mathbf{p} + b_{\text{in}})) \in \mathbb{R}^{d_{\text{model}}}, \quad (5.3)$$

with $W_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times (n_p \cdot P)}$ and $d_{\text{model}} = 64$ in our configuration. Layer normalisation stabilises the scale of the projected representation across batches, and GELU [48] is preferred over ReLU as the non-linearity of choice: unlike ReLU, GELU is smooth and weights its input by a probabilistic mask derived from a Gaussian CDF, which avoids the hard-zero regime of ReLU and tends to ease gradient flow during training.

5.2.3. Residual Blocks

The projected vector $\mathbf{h}^{(0)}$ is then refined by a stack of N residual MLP blocks. A natural alternative at this point would have been to use Transformer blocks, as PatchTST [47] or other recent architectures do. NeuroSym-CBF departs from that option because attention concentrates each token on a small subset of the remaining ones, whereas the entire context window of a time series typically carries useful signal in the form of trend, seasonality and base noise. Recent work has shown that, on several forecasting benchmarks, attention-based models can be matched or outperformed by considerably simpler architectures [49], suggesting that the inductive bias of self-attention is not always well aligned with the structure of time series. In our setting the rest of the pipeline expects a single summary vector per series, so a residual MLP, which treats every position of the flattened context with the same structural weight, produces this summary directly without an additional pooling stage.

Each block applies a two-layer MLP wrapped in a skip connection. Writing $\mathbf{h}^{(\ell)}$ for the input to the ℓ -th block,

$$\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + f_{\text{MLP}}(\mathbf{h}^{(\ell)}), \quad f_{\text{MLP}} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}, \quad (5.4)$$

where f_{MLP} is the composition $\text{Linear} \rightarrow \text{LN} \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear} \rightarrow \text{LN} \rightarrow \text{Dropout}$. Both linear layers preserve the dimension d_{model} , layer normalisation is applied after each linear transformation to keep activations on a stable scale, and dropout is used at two points to regularise the block.

The use of skip connections follows the residual learning principle introduced by He et al. [19], and has been successfully transferred to time-series forecasting through architectures such as N-BEATS [25], which similarly stacks fully-connected blocks with skip connections. The intuition is the same in both settings: by writing each block as the identity plus a learnable correction, the gradient finds a short, unobstructed path back through the network, which keeps deep stacks trainable and biases the encoder towards small, incremental updates of the representation. This bias is particularly convenient for

time series, whose informative patterns are usually smooth and require gentle refinement rather than abrupt non-linear remappings.

In our configuration we stack $N = 4$ two-layer MLP blocks with skip connections and dropout rate $p = 0.1$ applied inside each MLP after each linear layer.

$$\mathbf{h} = \mathbf{h}^{(N)} \in \mathbb{R}^{d_{\text{model}}}. \quad (5.5)$$

This vector is the only output of the encoder and will serve to (1) predict the neuro-symbolic basis function coefficients and (2) provide the time-series signal that the multimodal fusion stage combines with the metadata embedding, as described in Section 5.3.

5.3. Multimodal Fusion and Concept Bottleneck

The time-series encoder of Section 5.2 produces a hidden representation $\mathbf{h} \in \mathbb{R}^{d_{\text{model}}}$ that captures the dynamics of the past context window. The metadata modality, on the other hand, has already been encoded once at preprocessing time. As detailed in Subsection 3.3.2, a pretrained T5-base encoder maps the natural-language description of each series, namely its title, units and tags, into a fixed embedding $\mathbf{m} \in \mathbb{R}^{d_m}$ that summarises the full semantic identity of the series. These embeddings are precomputed and frozen.

The fused representation of $\mathbf{h}$ and $\mathbf{m}$ is built first, and then compressed into a small set of active concepts through a TopK Sparse Autoencoder. These concepts are the only channel through which the metadata modality reaches the forecast, as already anticipated in Equation 4.9 and represented in the overall pipeline in Figure 5.1.

5.3.1. Multimodal Fusion

The two modality vectors are first joined into a single representation by concatenation:

$$\mathbf{u}_0 = [\mathbf{h} \parallel \mathbf{m}] \in \mathbb{R}^{d_{\text{model}} + d_m}. \quad (5.6)$$

Concatenation is preferred over additive or multiplicative fusion because it preserves both vectors verbatim and lets the network learn how the two signals interact, rather than collapsing them prematurely into a shared space they do not naturally coexist in.

The concatenated vector is then passed through a small projection block that maps it to the input space of the concept bottleneck:

$$\mathbf{u} = \text{GELU}(\text{LN}(W_{\text{fuse}} \mathbf{u}_0 + b_{\text{fuse}})) \in \mathbb{R}^{d_{\text{sae}}}, \quad (5.7)$$

with $W_{\text{fuse}} \in \mathbb{R}^{d_{\text{sae}} \times (d_{\text{model}} + d_m)}$. This is the projection abstractly denoted Φ_{fuse} in Methodology (Equation 4.9); it is implemented here as a linear layer followed by layer normalisation and a GELU activation, so that the fused representation is properly scaled and non-linearly mixed before being handed to the bottleneck. Each coordinate of $\mathbf{u}$ already carries joint information from the two modalities and is therefore a meaningful unit on which the next stage can operate. In our configuration $d_{\text{sae}} = 128$.

5.3.2. TopK Sparse Autoencoder as Concept Bottleneck

The fused vector $\mathbf{u}$ is then passed through a sparse autoencoder with a TopK activation, which plays the role of the concept bottleneck of NeuroSym-CBF. The architecture follows the original K -sparse autoencoder of Makhzani and Frey [50], with the modern stabilisation tricks introduced by Gao et al. [31]. The encoder first centres the input by a learnable bias $\mathbf{b}_{\text{pre}}$ and projects it into an overcomplete dictionary space:

$$\mathbf{z}_{\text{pre}} = \text{ReLU}(W_{\text{enc}}(\mathbf{u} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_{\text{enc}}) \in \mathbb{R}^{d_z}, \quad (5.8)$$

with $d_z \geq d_{\text{sac}}$. The sparsity constraint is then applied directly: only the K largest activations are kept, and the rest are forced to zero:

$$\mathbf{z} = \text{TopK}(\mathbf{z}_{\text{pre}}, K), \quad \|\mathbf{z}\|_0 \leq K. \quad (5.9)$$

In our configuration, $d_z = 128$ and $K = 16$, so each series activates 12.5% of the dictionary. The autoencoder also defines a decoder $\hat{\mathbf{u}} = W_{\text{dec}} \mathbf{z} + \mathbf{b}_{\text{pre}}$ that reconstructs the input from the sparse code, but the decoder output is not used in the forward pass of the forecast: NeuroSym-CBF is trained end-to-end on the forecasting objective alone, with no auxiliary reconstruction term in the loss.

Two design choices distinguish this bottleneck from the standard formulation of Concept Bottleneck Models [26], as already anticipated in Subsection 4.3.2. First, sparsity is enforced structurally rather than emerging from the data: the dictionary is overcomplete and the Top- K selection retains at most K of its d_z entries as non-zero for any given series, with the rest hard-zeroed. Because the ReLU applied before the selection already suppresses most pre-activations, the active set is often smaller than K , so the budget acts as an upper bound rather than a fixed count. This contrasts with the original formulation, where each concept is an independent classifier and a sparse activation pattern is expected but not architecturally imposed. Second, no concept supervision is provided. The Top- K selection is the only mechanism enforcing structure on $\mathbf{z}$, and the meaning of each dictionary coordinate emerges from training rather than being annotated in advance.

A useful property of this design is that the bottleneck is editable by construction. During the forward pass, the K dimensions with the largest pre-activation values in $\mathbf{z}_{\text{pre}}$ are kept active and the rest are hard-zeroed, as described above. On top of this automatic selection, any dimension of $\mathbf{z}$ can be further modified at inference time, and that change propagates through the rest of the model in a predictable way through the FiLM modulator described next. The same vector $\mathbf{z}$, sparse by construction and grounded in both modalities, is what the FiLM modulator (Section 5.4) uses to adjust the symbolic coefficients of the forecast.

5.4. Concepts influence through FiLM Modulation

The sparse concepts $\mathbf{z}$ produced by the bottleneck are not used directly to compute the forecast. Their role, formalised in Section 4.4 of Methodology, is to intervene on the

symbolic coefficients predicted from the time-series modality: the unimodal forecast remains the reference, and the multimodal signal acts on it through a small, transparent affine transformation. This section describes how that mechanism is implemented in NeuroSym-CBF.

The K active concepts in $\mathbf{z}$ are passed through a small MLP that predicts the FiLM parameters:

$$(\boldsymbol{\gamma}, \boldsymbol{\beta}) = W_2 \text{ReLU}(W_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2, \quad \boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_\theta}, \quad (5.10)$$

with hidden dimension $d_{\text{film}} = 64$ and output dimension $2d_\theta = 26$, split evenly into the scale and shift vectors. These parameters are applied element-wise to the symbolic coefficients $\boldsymbol{\theta}_0 \in \mathbb{R}^{d_\theta}$ predicted from $\mathbf{h}$ alone:

$$\hat{\boldsymbol{\theta}} = \boldsymbol{\gamma} \odot \boldsymbol{\theta}_0 + \boldsymbol{\beta}, \quad (5.11)$$

producing the modulated coefficients that feed the symbolic head described in the next section. The structure of this operation is illustrated in Figure 5.4: each γ_i rescales its corresponding coefficient and each β_i shifts it, so the metadata signal can amplify, attenuate or offset every component of the forecast independently, but never alter the symbolic decomposition itself.

A small but important implementation detail is the initialisation of the MLP. The weights of its last linear layer are set to zero and its biases are set so that $\boldsymbol{\gamma} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$ at the start of training, which makes the modulation an identity transformation: $\hat{\boldsymbol{\theta}} = \boldsymbol{\theta}_0$ initially, and the forecast that comes out of the symbolic head is exactly the one a unimodal model would produce. As training progresses and the metadata signal proves useful, the MLP gradually learns to deviate from this identity. The model therefore starts from the same operating point as its unimodal counterpart and only spends capacity on multimodal modulation when it is worth it, which is a convenient inductive bias for a setting where many series carry metadata of variable informativeness.

5.5. Symbolic Forecasting Head

The symbolic head is the final stage of NeuroSym-CBF and the only one that touches the forecast values directly. By the time the model reaches this point, two pieces of information have been prepared upstream: the unimodal symbolic coefficients $\boldsymbol{\theta}_0 \in \mathbb{R}^{d_\theta}$, predicted from the time-series hidden state $\mathbf{h}$ by a linear layer; and their modulated version $\hat{\boldsymbol{\theta}} = \boldsymbol{\gamma} \odot \boldsymbol{\theta}_0 + \boldsymbol{\beta}$ produced by the FiLM step of Section 5.4, which has already absorbed the multimodal signal carried by the sparse concepts $\mathbf{z}$. The head takes $\hat{\boldsymbol{\theta}}$ and turns it into the forecast over the $H = 18$ months of the horizon, following the closed-form expression formalised in Section 4.2 of Methodology and depicted in Figure 4.1.

The full coefficient vector is

$$\hat{\boldsymbol{\theta}} = [c_0, c_1, a_1, \dots, a_{S-1}] \in \mathbb{R}^{S+1}, \quad (5.12)$$

FiLM Modulation of Symbolic Coefficients

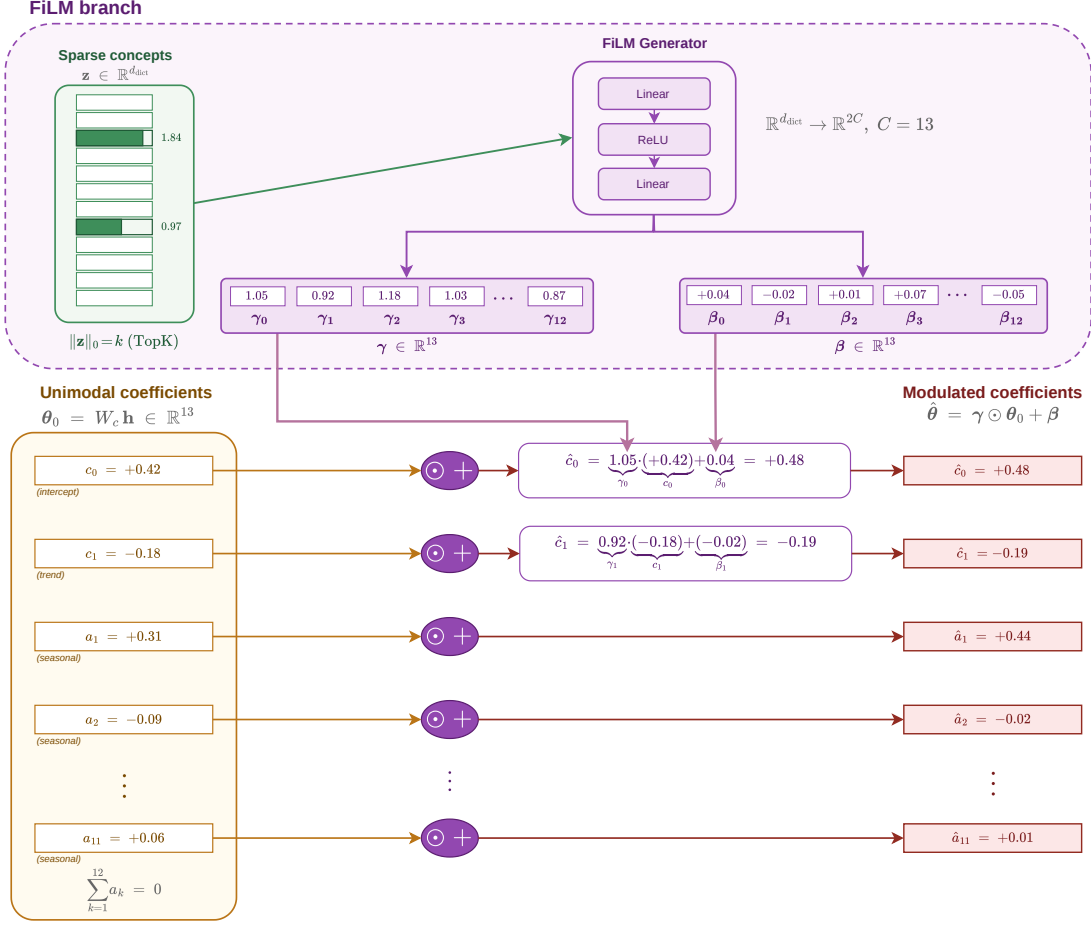


Fig. 5.4. The sparse concept vector $\mathbf{z}$ is mapped by a simple MLP (the FiLM generator) to a scale vector γ and a shift vector β , both of dimension d_θ . The unimodal symbolic coefficients θ_0 , predicted from the time-series representation $\mathbf{h}$, are then rescaled and shifted element-wise: $\hat{\theta} = \gamma \odot \theta_0 + \beta$.

with $S = 12$ and therefore $d_\theta = S + 1 = 13$ entries: one intercept, one trend slope and eleven seasonal coefficients. The forecast at month $h \in \{1, \dots, H\}$ is reconstructed component by component, as discussed next.

5.5.1. Trend Component

The intercept c_0 and the slope c_1 jointly define the linear part of the forecast. Rather than evaluating the slope against the raw month index, NeuroSym-CBF uses a standardised time index $\tilde{t}_h$ that is centred around zero and scaled to unit variance over the horizon:

$$\tilde{t}_h = \frac{h - \mu_t}{\sigma_t + \varepsilon}, \quad \mu_t = \frac{1}{H} \sum_{h=1}^H h, \quad \sigma_t = \sqrt{\frac{1}{H} \sum_{h=1}^H (h - \mu_t)^2}. \quad (5.13)$$

The reason for this standardisation is purely practical: it keeps the magnitude of the trend basis comparable to that of the intercept and the seasonal coefficients, so that no component dominates the optimisation simply because of its scale. With this choice, c_0 keeps

a clean interpretation as the unconditional level of the forecast and c_1 as the change per standard unit of horizon length.

5.5.2. Seasonal Component

The seasonal part of the forecast is the most subtle of the three components, and the one that benefits from a slightly more explicit description than its formal definition in Methodology suggests.

The coefficient vector contains $S - 1 = 11$ seasonal entries $a_1, \dots, a_{S-1}$, even though the seasonality period is $S = 12$. The twelfth value is not free: a sum-to-zero constraint forces the seasonal terms to average to zero over a full year, so that the seasonal component does not silently introduce a level shift into the forecast that would otherwise belong to c_0 . Concretely, the head builds a fixed projection matrix $\Phi \in \mathbb{R}^{S \times (S-1)}$, shared across all series and constructed once at initialisation time:

$$\Phi = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \\ -1 & -1 & \cdots & -1 \end{pmatrix} \in \mathbb{R}^{S \times (S-1)}. \quad (5.14)$$

The first $S - 1$ rows form an identity block, so each free coefficient a_k is routed to its own month; the last row is $[-1, -1, \dots, -1]$, which assigns to the twelfth month the negative sum of the others, enforcing the sum-to-zero constraint. This matrix maps the predicted coefficients to a vector of S monthly seasonal deviations:

$$\mathbf{s} = \Phi [a_1, \dots, a_{S-1}]^\top \in \mathbb{R}^S, \quad \sum_{m=1}^S s_m = 0. \quad (5.15)$$

Each entry s_m admits a direct reading: the deviation of month m from the annual baseline. To stabilise the optimisation, the columns of Φ are normalised in ℓ_2 norm at construction time, which rescales every direction of the seasonal basis to the same magnitude as the trend and intercept components by a shared constant factor, without altering the routing structure or the sum-to-zero constraint; the network absorbs this constant into the learned magnitude of the coefficients. Since the constraint is built into Φ rather than imposed by a regulariser, it is satisfied exactly throughout training.

Once $\mathbf{s}$ is available, the seasonal contribution at horizon step h is read off the entry corresponding to the calendar position of that month, $s_{m(h)}$ with $m(h) = ((h-1) \bmod S) + 1$, exactly as defined in Section 4.2.

5.5.3. Forecast Equation

With the trend and seasonal components in place, the forecast at month h follows the closed-form expression already given in Equation 4.5: an intercept, a linear trend evaluated on the standardised time index, and the seasonal entry corresponding to the calendar position of h . Sweeping h from 1 to H produces the full horizon $\hat{\mathbf{y}} \in \mathbb{R}^H$. The output remains in the normalised space defined by the per-instance preprocessing of Section 4.1.2; recovering the original scale is a post-processing step that lives outside the model.

A worked example. To make the equation concrete, suppose that for a particular series the modulated coefficients yield $c_0 = 0.5$, $c_1 = 0.02$ and monthly seasonal deviations $[s_1, \dots, s_{11}] = [0.10, 0.06, -0.02, \dots, 0.05]$, which by the sum-to-zero constraint imply $s_{12} = -\sum_{m=1}^{11} s_m$. The forecast at $h = 1$ reads the seasonal entry of its calendar month, $s_{m(1)} = s_1 = 0.10$, giving

$$\hat{y}_1 = 0.5 + 0.02 \cdot \tilde{t}_1 + 0.10 \approx 0.59,$$

with a small negative contribution from $\tilde{t}_1$, since month 1 falls below the centred horizon mean. The forecast at $h = 12$ picks up the constrained entry s_{12} , and at $h = 13$ the cycle repeats, so $m(13) = m(1) = 1$ and the seasonal term is again $s_1 = 0.10$. The full eighteen-month horizon is built by sweeping h from 1 to $H = 18$: at each step the calendar month $m(h)$ selects the corresponding seasonal deviation $s_{m(h)}$, which is added to the intercept and the trend, with each component contributing transparently to the final value.

This closed-form structure is what makes the head genuinely interpretable: the multimodal information has only been allowed to influence the forecast through the thirteen entries of $\hat{\boldsymbol{\theta}}$, and each of those entries has an understandable role.

5.6. Loss Function

NeuroSym-CBF is trained end-to-end with the objective of minimizing the Mean Absolute Error (MAE) between the forecast and the ground-truth horizon, computed only on positions that correspond to real observations:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}, \mathbf{k}) = \frac{\sum_{h=1}^H k_h |\hat{y}_h - y_h|}{\sum_{h=1}^H k_h + \varepsilon}, \quad (5.16)$$

where $\mathbf{k} \in \{0, 1\}^H$ is the horizon mask defined in Section 4.1.2 and $\varepsilon = 10^{-8}$ guards against division by zero. All valid horizon steps contribute equally to the loss: the mask excludes only padded positions of series shorter than H months, not a weighting scheme over the horizon. A full discussion of MAE and the remaining evaluation metrics is provided in Section 6.1.

We adopt MAE for its interpretability and robustness to outliers, since it penalizes deviations linearly in the target scale. As discussed in [51], the optimal loss depends on the

error distribution, and MSE may be more suitable when large outliers should be penalized more strongly. The ℓ_1 formulation also keeps the gradient bounded, which improves numerical stability when training over millions of windows with mixed scales. The absolute value is non-differentiable only at the single point $\hat{y}_h = y_h$, where the subgradient is bounded in $[-1, 1]$ and assigned a fixed value by the autograd engine.

No auxiliary loss is added to this objective. The TopK-SAE described in Section 5.3.2 could in principle be regularised with a reconstruction term, as is standard in the sparse autoencoder literature, but in NeuroSym-CBF the bottleneck is trained purely through the forecasting signal. The only training-time intervention on the SAE is a periodic decoder normalisation (described in Section 5.7), which keeps the magnitudes of the dictionary elements under control without contributing to the loss.

5.7. Training Setup and Evaluation Protocol

With the architecture defined, what remains is to specify how NeuroSym-CBF is trained: the data split, the evaluation regime across the different context lengths and the optimisation details during training.

5.7.1. Train and Test Split

The dataset is split chronologically per series. For each series in the corpus, the last $H = 18$ months are reserved for evaluation, and the rest is used for training. Concretely, given a series of length T , the training portion is the prefix of length $T - H$, and the test sample is the most recent block of H values, evaluated using the L months immediately preceding it as context. This guarantees that no future information leaks into training and that the evaluation reflects the genuine forecasting setting in which the model would be deployed: predicting the next eighteen months from past observations.

Training samples are generated through a sliding-window scheme over each training prefix. For every valid position t , a context window of length up to L ending at t and a horizon of length H starting at $t + 1$ are extracted, with the constraint that the context cannot be shorter than $L_{\min} = \min(13, L)$ months. Shorter contexts are left-padded to length L with the padding value, and the binary mask described in Section 4.1.2 prevents padded positions from contributing to the loss. The lower bound $L_{\min} = 13$ ensures that every training window contains at least one full annual cycle plus an extra observation, which is the minimum amount of history required for the seasonal component of the symbolic head to be observable.

5.7.2. Evaluation across Context Lengths

A separate model is trained for each evaluated context length $L \in \{8, 10, 15, 24, 30\}$, and all of them are evaluated on the same fixed test split: one forecast per series, comparing the predicted eighteen months against the held-out ones. Training a dedicated model per context length is what makes the comparison across L meaningful: each model is allowed to specialise to the regime it is being tested on, and the differences in performance can be attributed to the amount of historical information available rather than to a mismatch between training and inference conditions.

The number of training windows for each configuration is summarised in Table 5.1. The counts are not strictly equal across context lengths because the lower bound $L_{\min} = \min(13, L)$ admits more or fewer windows per series depending on L . For $L \geq 13$ the lower bound saturates at 13 and every configuration sees exactly the same set of training windows; for $L < 13$ the bound drops with L and a few additional shorter windows enter the pool. The test set is shared across all configurations and contains 194,442 windows, one per series.

TABLE 5.1. NUMBER OF TRAINING WINDOWS FOR EACH EVALUATED CONTEXT LENGTH. SERIES WITH INSUFFICIENT HISTORY ARE FILTERED OUT BEFORE WINDOW GENERATION; THE TEST SET CONTAINS 194,442 WINDOWS IN ALL CASES.

Context length L	Training windows
8	35,606,295
10	35,217,411
15	34,634,085
24	34,634,085
30	34,634,085

At evaluation time, the model is applied to each test window once and the predicted forecast is compared against the held-out horizon. Performance is reported per-sample using a set of standard forecasting metrics, including MAE, sMAPE and MASE, together with their median and tail percentiles to capture not only average accuracy but also the behaviour on the harder series of the dataset. The full definition of each metric and the analysis of the results will be developed over Chapter 6.

5.7.3. Optimisation Details

The optimiser configuration combines standard defaults with light empirical validation. The learning rate of 10^{-3} , weight decay of 10^{-4} and gradient clipping at 1.0 follow widely-used AdamW conventions; among the alternatives tried informally during development, these produced the most stable training across all context lengths. The cosine annealing

schedule decays the step size smoothly towards the end of training, and the number of epochs was fixed empirically at three, after which the train MAE no longer improves and the model begins to overfit its training distribution.

Two implementation details specific to the TopK-SAE deserve a mention. After every optimisation step, the columns of the SAE decoder are renormalised to unit ℓ_2 norm whenever they exceed it. This is a standard stabilisation trick in TopK sparse autoencoders [31]: it helps prevent degenerate solutions and reduces the risk of dead latents while keeping sparsity directly controlled. Likewise, the FiLM head is initialised so that the modulation begins as the identity transformation, as already discussed in Section 5.4.

It is important to add that the T5 metadata encoder remains frozen throughout training; only the parameters of the time-series encoder, fusion projection, TopK-SAE, FiLM modulator and symbolic head are updated.

6. EXPERIMENTAL WORK

After defining the structure, components and training details of NeuroSym-CBF, a series of experiments were carried out over this model, its unimodal neuro-symbolic ablation, the main SoTA models such as Chronos-2, Chronos-bolt-base (Amazon) [20], [22] and TimesFM (Google) [21], and AutoETS as a classical statistical baseline. The evaluation metrics used, the main results, the impact of context length across models, and the evolution of the error over the forecasting horizon are discussed throughout this chapter.

6.1. Evaluation Metrics

Time series forecasting results are typically evaluated through quantitative metrics that measure the prediction error. It is worth noting that a forecasting error reflects the difference between the forecast and the actual observed value; it does not necessarily indicate a mistake, but rather captures the inherently unpredictable component of the series [46]. Since the FRED dataset contains series with fundamentally different scales, all metrics are computed on the min-max normalised scale to ensure fair cross-series comparisons, following the scale-independence principle advocated by [52] and adopted in standard benchmarks such as M4 [8].

6.1.1. Mean Absolute Error (MAE)

The Mean Absolute Error is probably the most intuitive and widely used metric in forecasting. For a sequence of H horizon steps, it is defined as:

$$\text{MAE} = \frac{1}{H} \sum_{h=1}^H |y_{t+h} - \hat{y}_{t+h}| \quad (6.1)$$

where y_{t+h} is the actual value and $\hat{y}_{t+h}$ is the predicted value at step h . Its main appeal is that it penalises all errors equally and its interpretation is direct: the average absolute deviation from the true value on the normalised scale. Beyond being an evaluation tool, MAE is also the training objective used to optimise NeuroSym-CBF and its unimodal ablation (see Section 5.6), making it particularly relevant throughout this chapter.

6.1.2. Symmetric Mean Absolute Percentage Error (sMAPE)

The sMAPE [53] is a relative metric that partially addresses the scale sensitivity of standard percentage errors. It is defined as:

$$\text{sMAPE}(\%) = \frac{1}{H} \sum_{h=1}^H \frac{|y_{t+h} - \hat{y}_{t+h}|}{(|y_{t+h}| + |\hat{y}_{t+h}|) / 2} \cdot 100 \quad (6.2)$$

By symmetrising the denominator with the mean of the actual and predicted absolute values, sMAPE avoids the unbounded error behaviour of the standard MAPE when true values are close to zero, and treats over- and under-forecasting more evenly, and expresses forecasting error as a percentage. It has become a common secondary metric in forecasting benchmarks [8] precisely because it provides a relative picture that complements the absolute view of the MAE.

6.1.3. Mean Absolute Scaled Error (MASE)

The MASE [52] is perhaps the most robust of the three metrics used here, though also the most involved to interpret at first glance. The core idea is simple: it normalises the MAE of the model against the MAE of a *naïve seasonal baseline*, which predicts each future value as the observed value one seasonal period earlier in the context window. Formally:

$$\text{MASE} = \frac{\text{MAE}_{\text{model}}}{\text{MAE}_{\text{naïve}}} = \frac{\frac{1}{H} \sum_{h=1}^H |y_{t+h} - \hat{y}_{t+h}|}{\frac{1}{L-m} \sum_{t=m+1}^L |y_t - y_{t-m}|} \quad (6.3)$$

where L is the length of the context window, m is the seasonality period (set to $m = 12$ here, consistent with the predominantly monthly nature of the FRED series), and H is the prediction horizon at test time. The denominator lives entirely in the in-sample context: it measures how a naïve seasonal baseline would do by repeating the value observed m steps earlier, with no dependence on H whatsoever. The numerator is the model MAE over the out-of-sample horizon.

The reading is direct: $\text{MASE} < 1$ means the model beats the naïve baseline; $\text{MASE} > 1$ means it does not. Because the scaling is internal to each series, the metric compares cleanly across the heterogeneous pool of FRED series [52]. That said, it is significant to remark that the numerator averages the model error over $H = 18$ horizon steps, while the denominator reflects a one-step-ahead seasonal baseline. A MASE above 1 does not necessarily mean the model fails at every step: it can simply reflect that multi-step forecasting is harder than the single-step naïve benchmark suggests. Series where the naïve baseline MAE falls below 10^{-3} are dropped from the computation to avoid division issues on near-constant series.

6.1.4. Pairwise Win Rate

To compare models at the sample level a pairwise win rate is used. For any two models A and B evaluated on the same set of N test windows, the win rate of A over B is defined as:

$$\text{WR}(A > B) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{MAE}_A^{(i)} < \text{MAE}_B^{(i)}] \quad (6.4)$$

where $\text{MAE}^{(i)}$ is the per-sample MAE for the i -th test window. This non-parametric measure is complementary to mean metrics: a model can achieve a lower mean MAE by performing well on a small subset of outlier series, while the win rate reflects consistent per-sample superiority. Results are presented as win-rate matrices where entry (i, j) gives the fraction of samples for which row model i achieves a lower MAE than column model j .

6.1.5. Mean Rank

As an aggregator of metrics, we report the mean rank of each model across all evaluation settings as a scale-independent summary of relative performance. For each combination of context length and metric, models are ranked from best to worst (rank 1 to 6), with lower values indicating better performance.

Formally, let $\mathcal{C}$ denote the set of context lengths and $\mathcal{M}$ the set of evaluation metrics. The mean rank of a model f is defined as:

$$\text{MeanRank}(f) = \frac{1}{|\mathcal{C}| \cdot |\mathcal{M}|} \sum_{c \in \mathcal{C}} \sum_{m \in \mathcal{M}} \text{rank}(f, c, m) \quad (6.5)$$

where $\text{rank}(f, c, m)$ is the position of model f under context c and metric m .

This aggregation provides a robust comparison across different metrics, as it depends only on relative ordering rather than absolute scale. Mean rank is widely used in benchmark comparisons [10] as it summarises consistency across multiple evaluation settings in a single value.

Throughout the results, MAE is reported as the mean over all test windows, which aligns with its role as the training objective and ensures consistency with the loss being optimised. Apart from that, it is prioritised scale-free and statistically robust evaluation metrics, following standard recommendations in the forecasting literature [46]. In particular, we report MASE as the mean, consistent with its original formulation and its widespread use as a reliable, comparable measure across series with different scales [8], [52]. For sMAPE, we report the median rather than the mean, as its distribution across the heterogeneous pool of FRED series is right-skewed; the median therefore provides a more stable and representative summary that is less sensitive to extreme values.

In addition, we report MAE_{p90} to explicitly capture performance on the tail of the error distribution, providing a complementary view of worst-case behaviour that is not reflected by average-based metrics alone. Together, these metrics ensure a fair and robust evaluation across series with diverse scales and statistical properties.

6.2. Baseline Models

To assess the forecasting performance of NeuroSym-CBF, it is evaluated against three state-of-the-art zero-shot foundation models (Chronos-2, Chronos-bolt-base and TimesFM), a classical statistical baseline (AutoETS), and a unimodal neuro-symbolic ablation that removes the multimodal components of the architecture (the Top- K SAE and the FiLM conditioning layers, described in Section 5.1), isolating the contribution of the metadata-driven modulation. AutoETS is particularly relevant here as a conceptual reference point: exactly like NeuroSym-CBF, it explicitly decomposes each series into error, trend and seasonal components but does so through purely statistical, per-series fitting with no learned representations and no external signal. All foundation models are evaluated without any task-specific fine-tuning or adaptation on the FRED dataset used in this work. That said, FRED is a publicly available repository and many of its macroeconomic series are widely circulated, so it is plausible that historical observations from a good number of them have appeared in the pretraining corpora of these models.

6.2.1. Chronos-2

Chronos-2 [22] is the latest pretrained forecasting model released by Amazon Web Services AI Labs, published in 2025. It extends the original Chronos [20] framework, which framed time series forecasting as a language modelling problem over a discretised value vocabulary, into a genuinely universal model capable of handling univariate, multivariate, and covariate-informed forecasting within a single architecture. The key architectural innovation is a group attention mechanism that allows the model to share information across related time series within a batch, enabling in-context learning (ICL) without any task-specific training. Rather than concatenating covariates into an extended context window, group attention operates across the batch axis, scaling gracefully with the number of variates. The model is built on a T5 encoder backbone with alternating time attention and group attention layers, and produces multi-step probabilistic forecasts through a quantile regression head over 21 quantile levels [20], [22].

Training relies heavily on synthetic multivariate data generated by multivariatizers⁵ since high-quality real multivariate pretraining data is scarce. The base (120M parameter) model is trained in two stages, first with a context length of 2,048 and then extended to 8,192 through long-context post-training. On the three major benchmarks evaluated in its

⁵Multivariatizers are just procedures that impose cross-variate dependencies on univariate base generators

paper: fev-bench [54], GIFT-Eval [10] and Chronos Benchmark II; Chronos-2 achieves state-of-the-art results, outperforming all competing foundation models by a statistically significant margin [22]. On the economics and finance subset of GIFT-Eval [10], which is the domain closest to the FRED dataset used in this work, Chronos-2 ranks first among all pretrained models with a CRPS of 0.735 and a MASE of 0.772, and holds the top position on the overall leaderboard with a MASE of 0.698 and a CRPS of 0.485. It is evaluated here in univariate zero-shot mode, which is the fair comparison given the setup of this work.

6.2.2. Chronos-bolt-base

Chronos-bolt-base [20] is the production-optimised successor to the original Chronos family, also developed at AWS AI Labs. Chronos-bolt adopts a non-autoregressive architecture designed for significantly faster inference⁶. Despite this efficiency-first design, Chronos-bolt-base remains a competitive zero-shot forecaster. On the GIFT-Eval [10] economics and finance subset it achieves a CRPS of 0.762 and a MASE of 0.799, and on the full leaderboard a MASE of 0.808 and a CRPS of 0.574. It should be noted that GIFT-Eval flags Chronos-bolt-base as having potential test leakage, meaning that some evaluation series may overlap with its pretraining corpus. This is worth bearing in mind when interpreting comparisons, both on GIFT-Eval and on the FRED dataset used here, which contains well-known macroeconomic series that could plausibly appear in public pretraining data.

6.2.3. TimesFM

TimesFM [21] is a decoder-only time series foundation model developed by Google Research, introduced in 2024. Its design philosophy is explicitly borrowed from large language models: the model is a decoder-style transformer that processes time series as sequences of non-overlapping input patches, predicting the next output patch autoregressively. A key architectural choice is that the output patch length (128 time steps) is kept larger than the input patch length (32 time steps), reducing the number of autoregressive steps needed for long horizons without losing the flexibility of decoder-only training. The 200M parameter model is pretrained on a large and diverse corpus of roughly 100B time points, dominated by Google Trends and Wikipedia pageview statistics, supplemented with synthetic data from ARMA processes, seasonal patterns and piecewise linear trends [21].

In its zero-shot evaluation, TimesFM comes close to the accuracy of fully supervised models trained individually on each dataset, which was one of the first demonstrations that a single pretrained model could generalise across domains, granularities and horizon lengths without any task-specific adaptation [21]. On the GIFT-Eval [10] economics and

⁶The fastest among all pretrained models evaluated on fev-bench in terms of median runtime [22]

finance subset it achieves a CRPS of 0.761 and a MASE of 0.824, ranking below both Chronos variants on this domain. On the full GIFT-Eval leaderboard it records a MASE of 1.077 and a CRPS of 0.680, and like Chronos-bolt-base it carries a test leakage flag. It is evaluated here under the same strict zero-shot, univariate configuration as the other foundation models.

6.2.4. Unimodal Neuro-Symbolic Ablation

To isolate the contribution of the multimodal metadata component, a unimodal ablation of NeuroSym-CBF is included as an additional baseline. This model, referred to as *NeuroSym (Unimodal)* throughout the evaluation, shares the full neuro-symbolic backbone with NeuroSym-CBF (the same patch-based encoder, residual blocks and symbolic coefficient head that parameterises the forecast as a closed-form combination of intercept, linear trend and monthly seasonal components) but removes the metadata conditioning entirely. Concretely, the Top- K Sparse Autoencoder and the FiLM modulation layers are stripped out, so the model forecasts exclusively from the numerical context. All other training conditions are kept identical: same dataset splits, context and horizon lengths, optimiser, learning rate schedule and random seed, ensuring that any observed performance difference is attributable solely to the presence or absence of the multimodal signal.

6.2.5. Exponential Smoothing (AutoETS)

AutoETS is a classical statistical baseline from the `statsforecast` library [55], built on the state space framework for exponential smoothing formalised by Hyndman et al. [56]. The model automatically selects the best combination of error, trend and seasonal components (each of which can be additive, multiplicative or absent) by maximising the likelihood and minimising the corrected Akaike Information Criterion (AICc), a finite-sample correction of the AIC that penalises model complexity to avoid overfitting when selecting among candidate configurations [46]. This explicit decomposition into the same three structural primitives (level, trend, seasonality) that NeuroSym-CBF encodes as learnable symbolic coefficients makes AutoETS a natural conceptual reference: both approaches share the *same interpretable forecast structure*, but AutoETS derives it through per-series statistical fitting with no learned representations and no external signal, while NeuroSym-CBF estimates it globally via a neural backbone conditioned on metadata.

A practical concern with very short context windows is whether AutoETS can fit a model with a seasonal component when the history is shorter than one full seasonal cycle (e.g. $\text{ctx} = 8$ against a monthly seasonality of $m = 12$). This is handled automatically by the `statsforecast` implementation: when the available context is insufficient to identify a seasonal structure, the AIC-based model selection simply discards the seasonal component and fits a simpler ETS variant. A `SeasonalNaive` fallback was registered for robustness, but in practice it was never triggered across all 194,442 test series and all five

context lengths, confirming that the automatic model selection degrades gracefully under short-context conditions.

6.3. Main Results

All models are evaluated on a fixed forecast horizon of $H = 18$ steps, consistent with the monthly frequency of the FRED series and corresponding to a 1.5-year lookahead. This horizon spans more than one seasonal cycle, which matters for assessing whether forecasts remain stable beyond the first period. It is also not an arbitrary choice: $H = 18$ has been the standard for monthly forecasting benchmarks since the M1 competition [57] through to M4 [8]. Longer horizons such as $H = 24$ would further stress-test stability but at the cost of usable history per series. As defined in Section 5.7.2 to study how the amount of available history affects performance, each model is tested under five context lengths ($T \in \{8, 10, 15, 24, 30\}$ months), yielding 30 experimental configurations across the six models. The results are discussed from three angles: overall accuracy across configurations, sample-level pairwise comparisons, and how the forecasting error evolves step by step over the horizon.

6.3.1. Aggregate Forecasting Accuracy

Table 6.1 reports the forecasting accuracy of all six models across all context lengths under four metrics. Two consistent patterns emerge regardless of context length:

- First, all models improve monotonically as the context window grows, a logical and visible trend across all metrics and model families, confirming that longer history is universally beneficial in this domain.
- Second, and more importantly, NeuroSym-CBF achieves the best mean MAE and best MASE in every single one of the five context configurations.

The gains of NeuroSym-CBF over its unimodal ablation on mean MAE are 7.5% at $T = 8$, 7.6% at $T = 10$, 12.6% at $T = 15$, 6.7% at $T = 24$, and 6.9% at $T = 30$, directly quantifying the contribution of the metadata conditioning. On MASE, the picture is consistent within the stable regime ($T \geq 15$): the improvements are 10.6% at $T = 15$, 7.3% at $T = 24$, and 8.0% at $T = 30$ confirming that the metadata conditioning improves the forecasting beyond what is explained by the training objective alone.

The advantage extends further to the tail of the error distribution: NeuroSym-CBF achieves the best MAE_{p90} across all five context lengths without exception. The gap is particularly striking against the foundation models: at $T = 10$, NeuroSym-CBF reduces the 90th-percentile error by 12.7% over Chronos-2 and 15.7% over TimesFM-200M, and even against the unimodal ablation the improvement ranges from 5.5% to 8.3% depending

on context. This systematic reduction in worst-case error, on top of the average improvements, suggests that the FiLM modulation is most beneficial on the harder series of the pool, where the numeric history alone is least sufficient, rather than merely shifting the mean.

Among the zero-shot foundation models, Chronos-2 is the strongest competitor at short and medium context lengths, consistently ranking second on both MAE and MASE at $T \leq 24$. At $T = 30$, however, Chronos-bolt-base overtakes it on mean MAE (0.389 vs 0.393) and median sMAPE (60.7% vs 63.2%). Although somewhat unexpected given Chronos-2’s stronger performance across shorter contexts, this suggests that the patch-based direct forecasting strategy used by Chronos-bolt may provide advantages under specific long-context settings in our benchmark. AutoETS ranks last on mean MAE and MASE across all contexts, but remains competitive on MAE_{p90} at $T = 15$ (second best with 1.139), likely because its explicit decomposition into trend and seasonal components captures dominant periodic structure in a subset of series, even if it cannot generalise across the full heterogeneous pool.

It is worth noting a discontinuity in the MASE values between $T \leq 10$ and $T \geq 15$ visible across all models in Table 6.1. For short context lengths, MASE clusters above 3.0 for every model; from $T = 15$ onwards it drops sharply by more than 50%, for example from 3.137 to 1.447 for NeuroSym-CBF. This is not an anomaly in model performance but an artefact of the MASE denominator: when $T < m = 12$, the naïve seasonal baseline cannot be computed over a full seasonal cycle, making the denominator artificially small and inflating the ratio uniformly across all models. From $T = 15$ onwards the MASE stabilises into a meaningful and directly comparable regime, and it is in this regime that the ranking differences discussed above are most informative.

6.3.2. Pairwise Win Rates and Mean Rank

While the aggregate metrics in Table 6.1 capture model performance in expectation, they do not reveal how often one model actually outperforms another on the same individual series. As described in Section 6.1.4, the pairwise win rate addresses this by counting, for each pair of models, the fraction of test windows in which one produces a lower per-sample MAE than the other. Figure 6.1 shows the average win-rate matrix of NeuroSym-CBF against all baselines, averaged across all five context lengths over the $N = 194,442$ test windows evaluated per configuration. Values above 0.50 indicate row model wins more often than it loses against the column model.

The row corresponding to NeuroSym-CBF is the most favourable in the matrix. Against the Neuro-Sym (Unimodal) it achieves an average win rate of 0.61, confirming that the metadata conditioning provides a consistent per-sample advantage over the same architecture without it. Against AutoETS the margin is similar (0.62), while against the foundation models the win rates are narrower but still positive: 0.52 against both Chronos-2 and TimesFM-200M, and 0.53 against Chronos-bolt-base. It is worth noting that win

TABLE 6.1. FORECASTING ACCURACY OF ALL MODELS
ACROSS CONTEXT LENGTHS $T \in \{8, 10, 15, 24, 30\}$. **BOLD**
INDICATES THE BEST VALUE PER ROW; UNDERLINE
INDICATES SECOND BEST. $\downarrow$ LOWER IS BETTER.

T	Metric $\downarrow$	Model					
		NeuroSym-CBF	Neuro-Sym (Unimodal)	Chronos-2	Chronos- bolt-base	TimesFM 200M	AutoETS
8	MAE $_{\mu}$	0.7047	0.7619	<u>0.7516</u>	0.7773	0.7593	0.7808
	MAE $_{p90}$	1.4652	<u>1.5534</u>	1.6430	1.6873	1.7185	1.6300
	MASE $_{\mu}$	2.8949	3.1174	<u>3.1093</u>	3.2445	3.1575	3.2380
	sMAPE $_{med}$ (%)	79.76	<u>83.99</u>	88.82	92.50	87.30	91.23
10	MAE $_{\mu}$	0.6376	0.6902	<u>0.6874</u>	0.7029	0.6934	0.7088
	MAE $_{p90}$	1.2893	<u>1.3638</u>	1.4774	1.5052	1.5295	1.4317
	MASE $_{\mu}$	3.1371	<u>3.3831</u>	3.4190	3.5206	3.4745	3.5194
	sMAPE $_{med}$ (%)	76.15	<u>80.03</u>	87.59	89.00	84.12	87.93
15	MAE $_{\mu}$	0.5448	0.6234	<u>0.5685</u>	0.5765	0.5888	0.5931
	MAE $_{p90}$	1.0700	1.1670	1.1911	1.2099	1.2623	<u>1.1391</u>
	MASE $_{\mu}$	1.4468	1.6175	<u>1.5175</u>	1.5295	1.5415	1.5664
	sMAPE $_{med}$ (%)	76.23	90.06	80.26	81.55	<u>79.44</u>	80.93
24	MAE $_{\mu}$	0.4182	0.4480	<u>0.4365</u>	0.4390	0.4635	0.4828
	MAE $_{p90}$	0.8107	<u>0.8657</u>	0.8754	0.8781	0.9360	0.9027
	MASE $_{\mu}$	1.4685	1.5840	<u>1.5073</u>	1.5187	1.5886	1.7179
	sMAPE $_{med}$ (%)	65.01	68.40	66.76	<u>65.13</u>	67.71	73.75
30	MAE $_{\mu}$	0.3747	0.4025	0.3933	<u>0.3893</u>	0.4072	0.4366
	MAE $_{p90}$	0.7280	0.7779	<u>0.7765</u>	0.7769	0.8112	0.8179
	MASE $_{\mu}$	1.5065	1.6376	1.5542	<u>1.5352</u>	1.6017	1.7986
	sMAPE $_{med}$ (%)	<u>61.04</u>	64.94	63.22	60.65	61.07	69.98

rates in the range 0.52–0.53 against strong zero-shot models are not surprising given the inherent stochasticity and heterogeneity of macroeconomic time series: even a consistently better model will not dominate on every individual series. More broadly, small performance gaps between top-ranked forecasting models are a well-documented phenomenon in the forecasting literature, where differences among leading methods have historically remained within a few percent [2]. That said, these margins are evaluated over $N = 194,442$ test windows, a sample large enough that even small differences in win rate are statistically meaningful rather than noise. Under a binomial model with $p = 0.51$, the 95% confidence interval on any reported win rate is approximately ± 0.002 , so even the narrowest margins observed are well above the threshold of statistical noise. The full per-context win-rate matrices are provided in Appendix A.

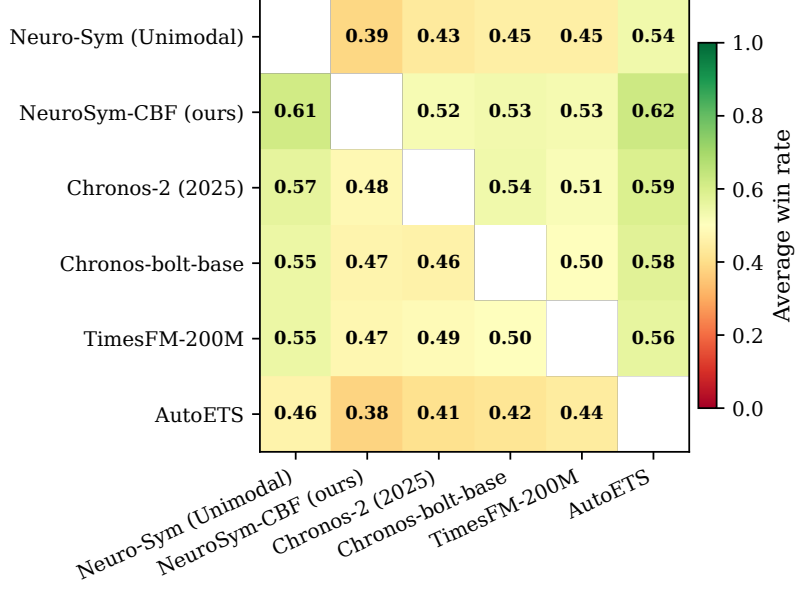


Fig. 6.1. Average pairwise win-rate matrix across all five context lengths. Each cell (i, j) reports the fraction of series on which the row model i achieves a lower error than the column model j , averaged over the five context lengths; values above 0.5 (green) indicate that the row model wins more often than the column model.

These win rates, combined with the MAE_{p90} results discussed in Section 6.3.1, point to an asymmetric dominance pattern: NeuroSym-CBF not only wins on more individual series than it loses, but when it loses the margin is small, and when it wins the margin is large. This asymmetry is precisely what drives the consistent MAE_{p90} advantage observed across all five context lengths, and it is a stronger result than the win rate alone suggests.

To provide a single summary statistic that is robust to the scale and distributional differences of individual metrics, a mean rank is computed for each model. As defined formally in Subsection 6.1.5, for every combination of context length and metric, all six models are ranked from best (1) to worst (6), and the ranks are averaged across all five context lengths. This aggregation is intended purely as a reading aid: each individual subtask is evaluated over a large number of series and therefore already carries strong statistical meaning on its own; the mean rank collapses that information into a single number for ease of comparison, at the cost of treating all context lengths and metric choices as equally important. Three metric sets are considered:

- **Main** combines the four metrics reported in Table 6.1 (MAE , MAE_{p90} , MASE , $\text{sMAPE}_{\text{med}}$).
- **Mean** uses only mean-aggregated metrics (MAE mean, MASE mean, sMAPE mean).
- **Median** uses only median-aggregated metrics (MAE median, MASE median, sMAPE median).

Reporting all three allows the robustness of the ranking to be assessed across different aggregation choices, directly addressing any concern about metric selection bias.

TABLE 6.2. MEAN RANK OF EACH MODEL AVERAGED OVER ALL FIVE CONTEXT LENGTHS, UNDER THREE METRIC SETS.

Model	MAIN ↓	MEAN ↓	MEDIAN ↓
NeuroSym-CBF	1.05	1.33	1.93
Chronos-2	2.90	2.60	2.47
Neuro-Sym (Unimodal)	3.70	4.07	4.80
Chronos-bolt-base	4.00	3.60	3.27
TimesFM-200M	4.30	3.73	2.93
AutoETS	5.05	5.67	5.60

NeuroSym-CBF places first under all three metric sets, with a Main mean rank of 1.05, nearly a full rank ahead of the closest competitor, Chronos-2 at 2.90. The gap widens under mean aggregation and narrows slightly under median, reflecting the fact that NeuroSym-CBF’s edge is stronger on mean-based metrics where the tail of the distribution contributes more weight. Chronos-2 consistently ranks second across all three sets, confirming it as the strongest zero-shot baseline. The per-context breakdown of all three ranking sets is provided in Table 1 in Appendix A.

6.3.3. Error Evolution Across the Forecast Horizon

The aggregate metrics discussed in Section 6.3.1 summarise performance across all 18 forecast steps, but they do not reveal how the relative advantage of NeuroSym-CBF evolves as the horizon grows. As noted by [46], most nonstationary time series models produce a rapidly growing uncertainty over the forecast horizon: if one model captures the underlying dynamics slightly better, the gap in error is expected to widen with each additional step. The following analysis quantifies how the margin of NeuroSym-CBF over each baseline changes step by step.

Figure 6.2 shows the MAE at each of the 18 forecast steps for all six models at $T = 8$ and $T = 30$. Two observations hold across both panels:

- All models degrade monotonically: there are no reversals or plateaus, which confirms the hypothesis that the task difficulty grows uniformly with the horizon regardless of architecture.
- The curves are nearly indistinguishable in the first steps, but from approximately step 6 onwards NeuroSym-CBF (red) begins to separate consistently below the rest, widening the gap through to the final step. By step 18 at $T = 8$, NeuroSym-CBF reaches a MAE of 0.972 against 1.095 for the nearest competitor (TimesFM-200M);

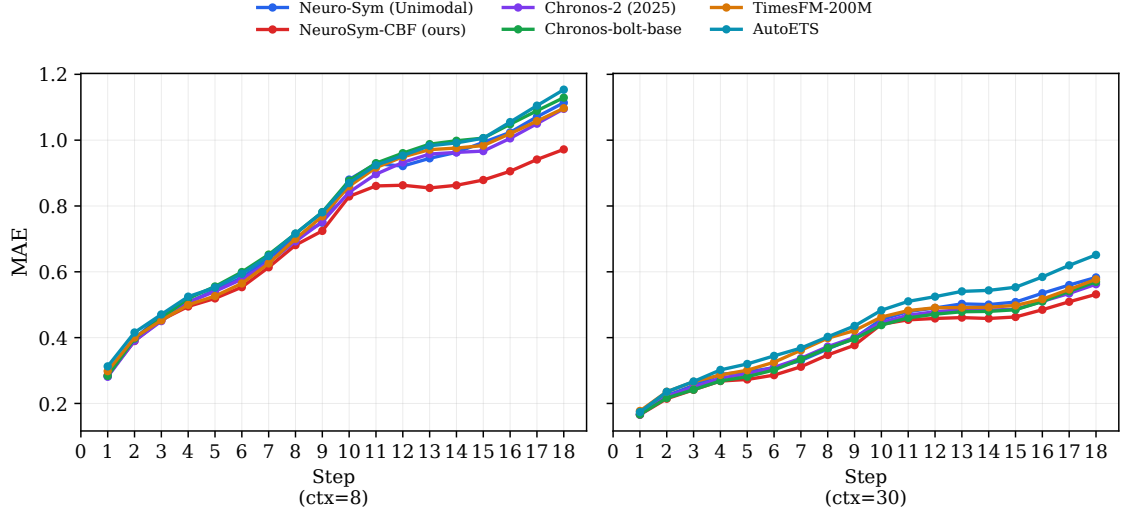


Fig. 6.2. MAE at each forecast step ($h = 1, \dots, 18$) for all six models at context lengths $T = 8$ (left) and $T = 30$ (right).

at $T = 30$ the separation is narrower but equally consistent, with NeuroSym-CBF at 0.532 against 0.562 (Chronos-2). Full per-step values for all context lengths are reported in Tables 2–6 in Appendix A.

This late-horizon separation is consistent with the structural properties of NeuroSym-CBF: the symbolic coefficient head parameterises the forecast as a closed-form combination of trend and seasonal components, which imposes a regularising structure that prevents the error from compounding freely at long horizons in the way that purely neural decoders tend to do. A secondary contributing factor is the training objective itself: since the MAE is averaged uniformly across all 18 forecast steps, the gradient signal is dominated by the later steps of the horizon, where absolute errors are largest, which may produce a mild relative disadvantage at the very first steps compared to a horizon-weighted loss. Both effects reinforce each other and jointly explain why the relative gap between NeuroSym-CBF and the baselines is smallest near $h = 1$ and widens progressively towards $h = 18$.

Figure 6.3 complements this picture by plotting the absolute MAE gap between each baseline and NeuroSym-CBF at every forecast step, defined as in Equation 6.6, where positive values indicate that NeuroSym-CBF produces a smaller error than the baseline at that step.

$$\Delta_h = \text{MAE}_{\text{baseline model},h} - \text{MAE}_{\text{NeuroSym-CBF},h} \quad (6.6)$$

At $T = 8$ the gap is near zero or slightly negative for some baselines in the first few steps, then grows steadily from step 6 onwards, reaching between 0.12 and 0.18 by step 18 depending on the baseline. At $T = 30$ the pattern is qualitatively similar but compressed: the initial parity holds longer and the final gap is smaller in absolute terms, reflecting the

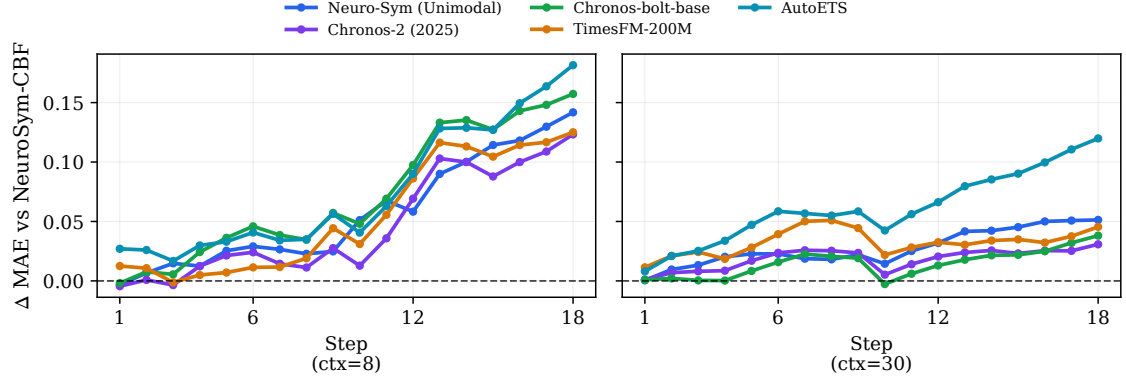


Fig. 6.3. Absolute gap in MAE between each baseline and NeuroSym-CBF at each forecast step, for $T = 8$ (left) and $T = 30$ (right).

overall lower error level at longer contexts. AutoETS shows the largest gap throughout both panels, while Chronos-2 remains closest to NeuroSym-CBF across the full horizon.

Table 6.3 quantifies the percentage MAE reduction of NeuroSym-CBF over each baseline at selected horizon steps ($h \in \{1, 3, 6, 12, 18\}$) across all the context lengths as represented in Equation 6.7. A positive value indicates NeuroSym-CBF is better; negative values indicate the baseline outperforms NeuroSym-CBF at that step.

The improvement over the Neuro-Sym (Unimodal) is positive at virtually every cell and grows with the horizon, confirming that the metadata conditioning is most valuable when the prediction must look further ahead. Against the foundation models the picture is more nuanced: at $T = 15$, NeuroSym-CBF is outperformed at short horizons ($h = 1$) by Chronos-2 (-20.6%) and Chronos-bolt-base (-23.6%), which reflects the structural advantage of these zero-shot models in the immediate next-step prediction when given a moderate context. However, NeuroSym-CBF recovers consistently by $h = 6$ and maintains a positive margin through $h = 18$ in every context, confirming that its advantage is durable across the full horizon rather than confined to a specific regime.

$$\% \Delta_h = \frac{\text{MAE}_{\text{baseline},h} - \text{MAE}_{\text{ref},h}}{\text{MAE}_{\text{baseline},h}} \times 100 \quad (6.7)$$

TABLE 6.3. PERCENTAGE MAE REDUCTION OF NeuroSym-CBF OVER EACH BASELINE AT SELECTED HORIZON STEPS, PER CONTEXT LENGTH.

T	Baseline	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
8	Neuro-Sym (Unimodal)	-1.1	3.2	5.0	6.3	12.7
	Chronos-2	-1.6	-0.8	4.2	7.4	11.3
	Chronos-bolt-base	-0.7	1.2	7.7	10.2	13.9
	TimesFM-200M	4.2	-0.3	2.0	9.1	11.4
	AutoETS	8.6	3.6	6.9	9.5	15.7
10	Neuro-Sym (Unimodal)	3.0	3.6	6.5	7.3	11.2
	Chronos-2	0.3	-1.5	6.5	9.5	11.1
	Chronos-bolt-base	-1.8	-1.9	7.7	12.0	13.2
	TimesFM-200M	4.3	-2.1	3.3	11.1	11.3
	AutoETS	8.2	1.6	8.3	10.7	15.8
15	Neuro-Sym (Unimodal)	11.0	10.9	12.5	12.2	14.9
	Chronos-2	-20.6	-6.2	4.8	5.7	7.8
	Chronos-bolt-base	-23.6	-9.1	4.4	7.8	10.0
	TimesFM-200M	-15.3	-3.4	9.1	9.0	8.7
	AutoETS	-14.0	-2.7	8.0	7.8	15.2
24	Neuro-Sym (Unimodal)	-0.0	3.2	7.0	6.6	9.0
	Chronos-2	1.5	0.6	7.1	4.6	5.4
	Chronos-bolt-base	1.4	-0.4	6.5	4.8	7.3
	TimesFM-200M	10.6	11.9	15.1	8.4	8.4
	AutoETS	6.5	7.6	16.0	12.4	17.8
30	Neuro-Sym (Unimodal)	0.3	5.2	7.4	6.5	8.8
	Chronos-2	0.6	3.2	7.6	4.3	5.5
	Chronos-bolt-base	0.5	0.2	5.2	2.8	6.7
	TimesFM-200M	6.4	9.1	12.1	6.6	7.9
	AutoETS	4.6	9.5	17.0	12.6	18.4

6.4. Qualitative Forecast Analysis

Beyond the metrics reported in Section 6.3, it is also informative and more tangible to inspect how each model behaves on individual series. Aggregate scores can hide qualitative differences in forecast shape, bias, and reactivity to recent context, all of which matter in practice. To this end, nine series are randomly sampled from the test set and the predictions of every evaluated model are overlaid on the same axes. The same procedure is repeated for the two context lengths that delimit the evaluation range, $\text{ctx} = 8$ and $\text{ctx} = 30$, which respectively correspond to the shortest and longest context regimes analysed in this work.

Figures 6.4 and 6.5 show the resulting panels. Each subplot displays the input context, the ground-truth target, and the forecasts produced by the unimodal Neuro-Sym ablation, the proposed NeuroSym-CBF, Chronos-bolt-base, Chronos-2, TimesFM-200M, and AutoETS. Series are shown in their normalised scale, consistent with the preprocessing described in Chapter 3.

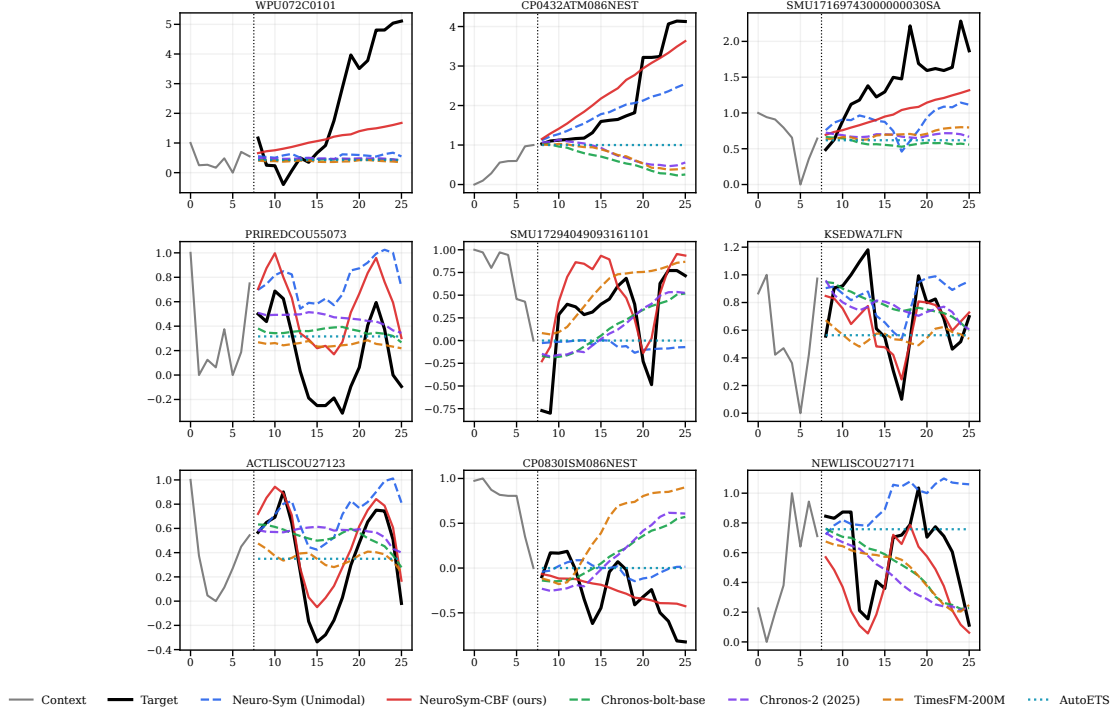


Fig. 6.4. Qualitative comparison of all evaluated models on nine randomly sampled test series with context length $\text{ctx} = 8$.

Several patterns emerge consistently across the two regimes. AutoETS produces forecasts that are smooth but largely insensitive to recent context, often collapsing to flat or weakly linear trajectories; this is expected behaviour for a classical statistical baseline and matches its weaker quantitative performance. The pretrained foundation models produce richer trajectories and capture local dynamics with notable fidelity, especially under $\text{ctx} = 30$ where the additional history provides them with a stronger anchor for in-context inference. However, this expressivity comes at a cost: under the short-context regime ($\text{ctx} = 8$), foundation models occasionally produce trajectories that diverge sharply from the ground truth, hallucinating dynamics that are not supported by the available history.

This behaviour is visible, for instance, in the upper-centre panel of Figure 6.4, where the model commits confidently to a trajectory that bears little resemblance to the actual evolution of the series. Such hallucinations are a known failure mode of in-context sequence models when the conditioning window is too short to disambiguate the underlying process, and they motivate the inclusion of structural inductive biases and metadata-driven modulation as explored in this work.

The contrast between the unimodal Neuro-Sym ablation and the proposed NeuroSym-

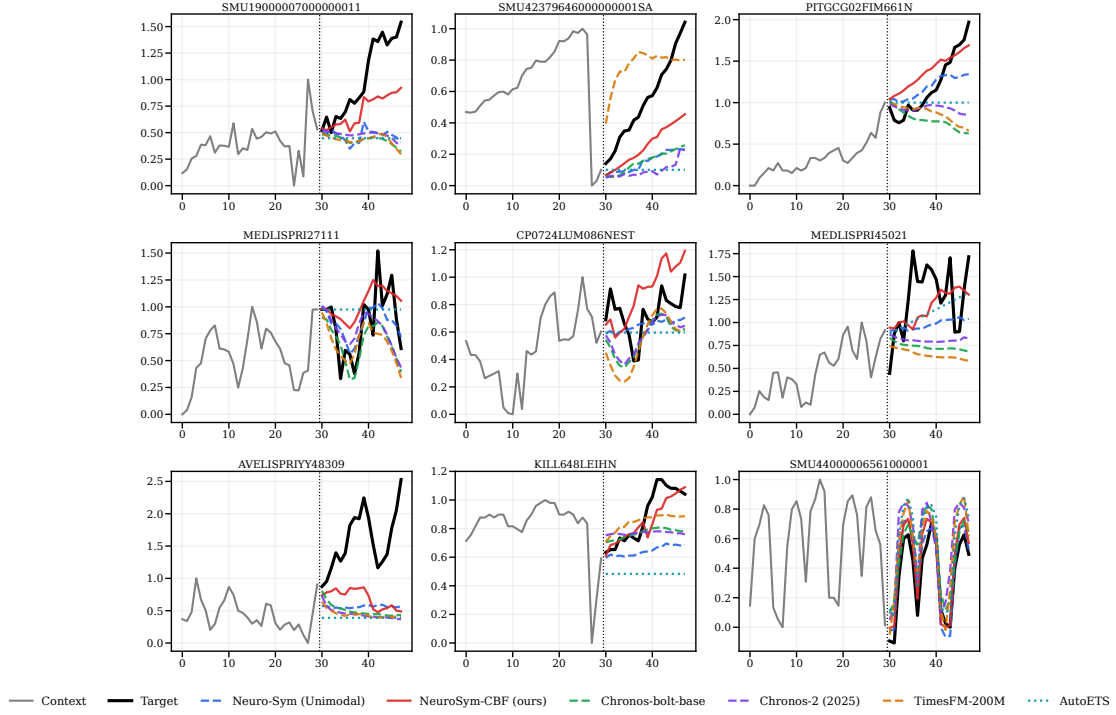


Fig. 6.5. Qualitative comparison of all evaluated models on nine randomly sampled test series with context length $\text{ctx} = 30$.

CBF is the most relevant comparison for this work, as both share the same neuro-symbolic backbone and differ only in the multimodal metadata injection through the FiLM module. Under $\text{ctx} = 8$, where the input signal alone provides limited information about the underlying dynamics, NeuroSym-CBF tends to produce forecasts that follow the target more closely, while the unimodal ablation occasionally over- or under-shoots in level or slope. Under $\text{ctx} = 30$ both variants converge to similar trajectories on most series, which is consistent with the intuition that the marginal value of metadata decreases as the autoregressive context becomes more informative. This observation is examined more rigorously in Subsection 6.5.3, where the effect of FiLM on the symbolic coefficients is quantified.

It is worth noting that these panels are intended as a qualitative complement to the aggregate metric. Since the nine series are randomly sampled, there is no intention to show model improvements here but showing that the figures illustrate the qualitative regime in which each family of models operates.

6.5. Interpretability and Concept-Level Analysis

The core contribution of the work is interpretability, an aspect that has motivated the architectural choices made throughout the previous chapters. The analysis is deliberately concentrated on the two context lengths that delimit the evaluation range, $\text{ctx} = 8$ and $\text{ctx} = 30$, so that the qualitative differences between the short- and long-context regimes

are made explicit. Both regimes are reported side by side whenever they convey complementary information; when the two contexts produce qualitatively indistinguishable results, the $\text{ctx} = 30$ counterpart is relegated to Appendix A. Four complementary experiments are reported: the structure of the learned concept space, the distributional effect of multimodal information on the symbolic coefficients, the specific contribution of the FiLM module, and a counterfactual ablation study that probes the controllability of individual concepts.

6.5.1. Concept Space Overview

A part of the interpretability in NeuroSym-CBF comes together with the TopK Sparse Autoencoder (SAE) that operates over the fused representation $[h_{\text{ts}} || h_{\text{meta}}]$. By construction, the encoder enforces an exact $L_0 = k$ activation pattern through a hard top- k selection on the post-ReLU activations, with $k = 16$ and a dictionary size of 128 concepts. As stated in Section 5.6, the SAE is trained jointly with the rest of the model and does not carry an auxiliary reconstruction or sparsity loss.

Two complementary diagnostics are reported to characterise the resulting concept space: (i) the empirical activation frequency of each concept across the evaluation set, and (ii) the average influence of each concept on the symbolic coefficients produced by the FiLM modulator.

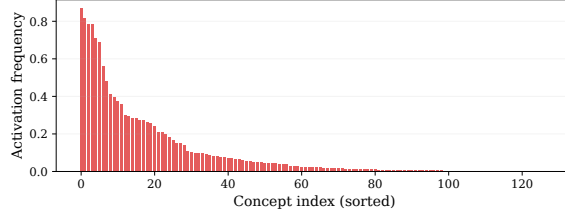
TABLE 6.4. SAE CONCEPT USAGE STATISTICS ON THE EVALUATION SET FOR THE TWO CONTEXT-LENGTH REGIMES. A CONCEPT IS CONSIDERED "DEAD" IF IT NEVER ACTIVATES ACROSS THE EVALUATION SAMPLES.

Context	Dict. size	Top-K	Mean active per sample	Live concepts
$\text{ctx} = 8$	128	16	13.70	124 / 128 (96.9%)
$\text{ctx} = 30$	128	16	13.81	112 / 128 (87.5%)

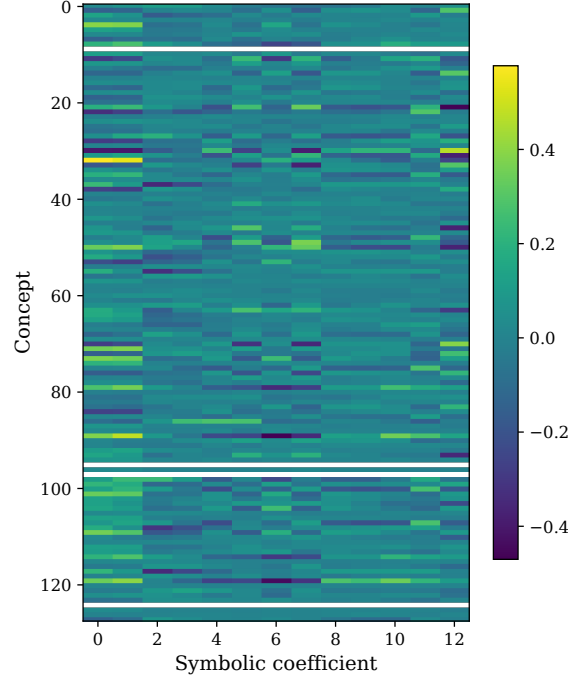
Table 6.4 summarises the basic usage statistics. Two observations are worth noting. First, although the structural budget is $k = 16$, the empirical mean number of active concepts per sample is slightly below this value (13.7–13.8) indicating that the effective sparsity is stronger than the nominal TopK constraint for some series. Second, the live concept population is noticeably larger under $\text{ctx} = 8$ (124 live concepts) than under $\text{ctx} = 30$ (112 live concepts). A plausible interpretation, consistent with the qualitative observations of Section 6.4, is that when the autoregressive context provides less information about the underlying dynamics the model is encouraged to recruit a wider portion of the multimodal concept vocabulary to compensate.

Beyond the basic usage statistics of Table 6.4, the activation frequency profile and the coefficient-influence heatmap exhibit qualitatively the same structure under both context

lengths. Following the criterion established at the beginning of the section, only the $\text{ctx} = 8$ versions are reported here; the $\text{ctx} = 30$ counterparts are deferred to Appendix A.



(a) Activation frequency per concept, sorted in descending order.



(b) Average symbolic coefficient $\Delta\theta$ induced by each concept (rows) on each of the 13 symbolic coefficients (columns).

Fig. 6.6. Structure of the learned concept space at $\text{ctx} = 8$.

Figure 6.6a displays the per-concept activation frequency across the evaluation set, sorted in descending order. The profile follows the long-tailed shape commonly reported for TopK-SAEs: less than 10 concepts activate on a large fraction of samples (above 0.6), a broader middle band activates on 0.1–0.4 of samples, and a tail approaches zero. The dead concepts identified in Table 6.4 correspond to the flat segment at the right end of the curve. This shape is informative because it indicates that concepts are neither collapsing onto a small number of dominant units nor spreading uniformly across the entire codebook. Instead, activation mass is distributed across a subset of concepts, creating a balance between sparsity and utilisation.

Figure 6.6b provides a complementary view of *what* the live concepts encode in terms of symbolic behaviour. Each row corresponds to one of the 128 SAE concepts and each

column to one of the 13 symbolic coefficients, with cell values reporting the average $\Delta\theta$ induced by the concept on the FiLM-modulated coefficients. Three patterns can be observed from Figure 6.6b:

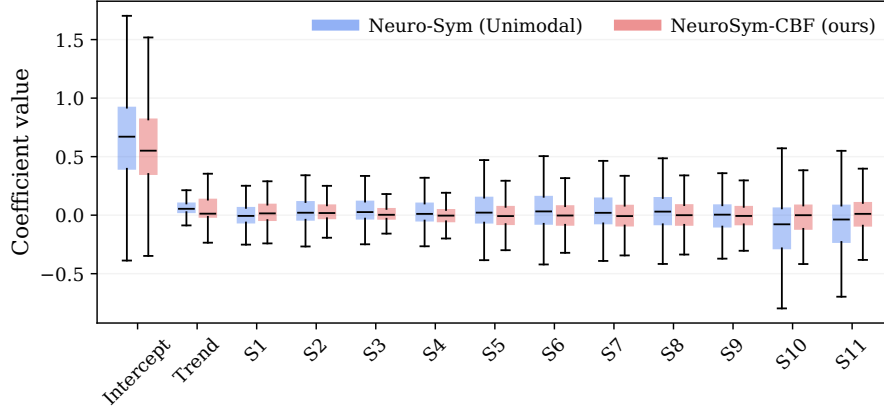
- The heatmap is clearly non-uniform. Most cells remain close to zero, while a smaller subset exhibits pronounced positive or negative effects, indicating that only a limited number of concepts strongly modulate the symbolic parameters.
- The white horizontal bands corresponding to dead concepts (visible approximately around rows ~ 10 , ~ 95 , and ~ 125) align with Table 6.4, confirming that the diagnostics are consistent.
- No individual concept dominates all coefficient columns. Instead, active concepts tend to concentrate their influence on specific subsets of coefficients: some primarily affect the intercept and trend terms, whereas others are associated with particular seasonal components.

6.5.2. Symbolic Coefficients: Ablation vs. Multimodal Model

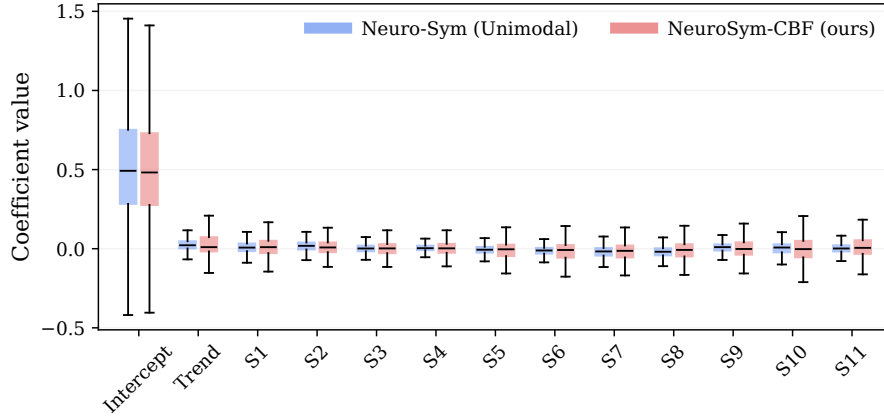
The most direct way to assess whether multimodal metadata changes the symbolic reasoning of the model is to compare the distribution of the 13 neuro-symbolic coefficients ($\theta_{\text{intercept}}, \theta_{\text{trend}}, \theta_{S_1}, \dots, \theta_{S_{11}}$) between the unimodal ablation (Neuro-Sym (Unimodal)) and the full multimodal model (NeuroSym-CBF). As defined in the Section 6.2, both models share the same neuro-symbolic backbone; the only difference is that NeuroSym-CBF additionally routes the fused representation through the SAE and applies FiLM modulation to the coefficients. Any systematic distributional shift between the two can therefore be attributed to the multimodal pathway.

Figure 6.7 shows the resulting boxplots for both context lengths. Two main observations emerge:

- The intercept coefficient shows the largest and most consistent shift between the two models. Under $\text{ctx} = 8$, NeuroSym-CBF exhibits a wider and displaced intercept distribution relative to the ablation, indicating that multimodal information systematically adjusts the predicted level of the series in ways that the autoregressive context alone cannot determine.
- The effect is substantially more pronounced at $\text{ctx} = 8$ than at $\text{ctx} = 30$. Under the longer context, the distributions of both models converge, with notably reduced interquartile ranges and a smaller shift between them. This confirms the pattern observed qualitatively in Section 6.4: the marginal contribution of metadata decreases as the autoregressive signal becomes more informative.



(a) $\text{ctx} = 8$



(b) $\text{ctx} = 30$

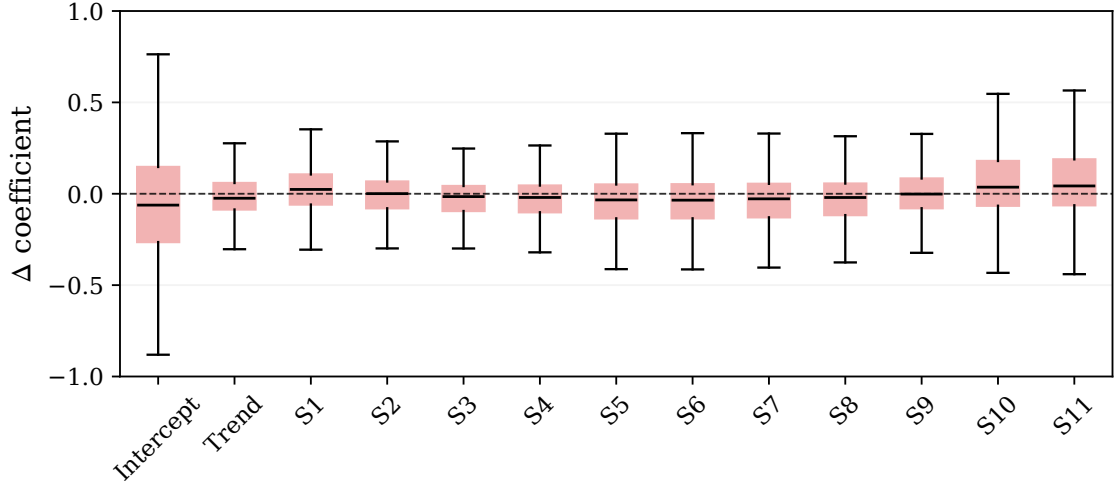
Fig. 6.7. Distribution of the 13 neuro-symbolic coefficients for the unimodal ablation (Neuro-Sym (Unimodal)) and the proposed multimodal model (NeuroSym-CBF), at $\text{ctx} = 8$ (top) and $\text{ctx} = 30$ (bottom). Each box covers the IQR; whiskers extend to the 5th and 95th percentiles.

It is important to contextualise the magnitude of these shifts. The boxplots report distributions of coefficient values across the full evaluation set, meaning that even a modest displacement in the median or a moderate widening of the interquartile range translates into a systematic change applied to every forecast in that distributional regime. The quantitative improvements reported in Section 6.3 are a direct consequence of these distributional adjustments: small but consistent coefficient shifts, when accumulated across thousands of series and horizon steps, produce the error reductions observed in the aggregate metrics.

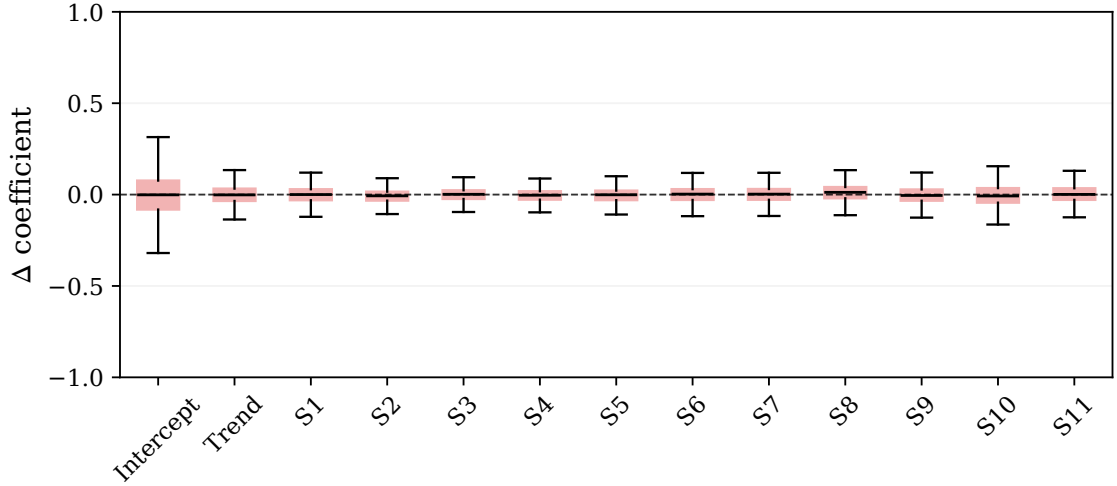
6.5.3. The Role of FiLM Modulation

The previous subsection established that the symbolic coefficients of NeuroSym-CBF differ systematically from those of the unimodal ablation but this subsection decomposes that shift into two complementary views: the net difference in final coefficients between the

two models, and the internal modulation applied by the FiLM module within NeuroSym-CBF.



(a) $\text{ctx} = 8$

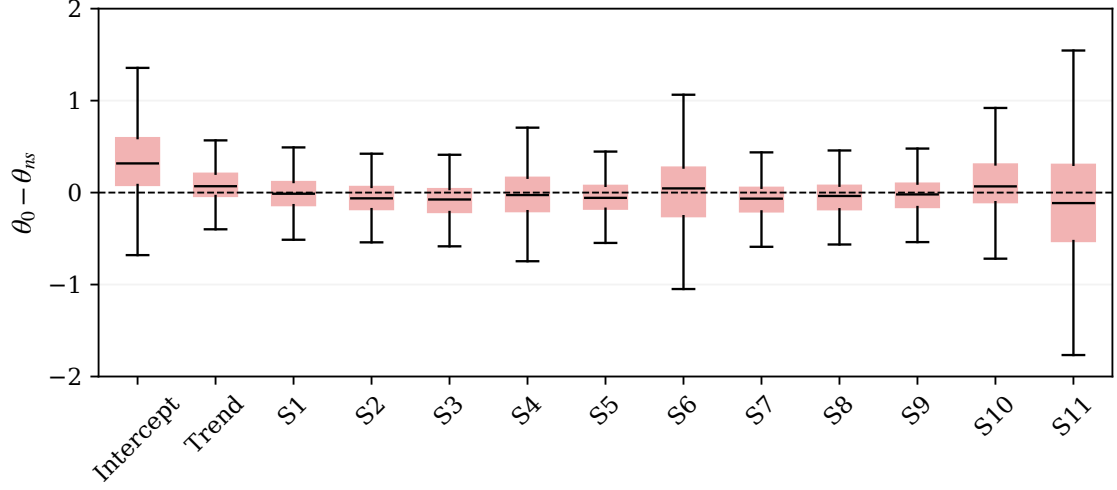


(b) $\text{ctx} = 30$

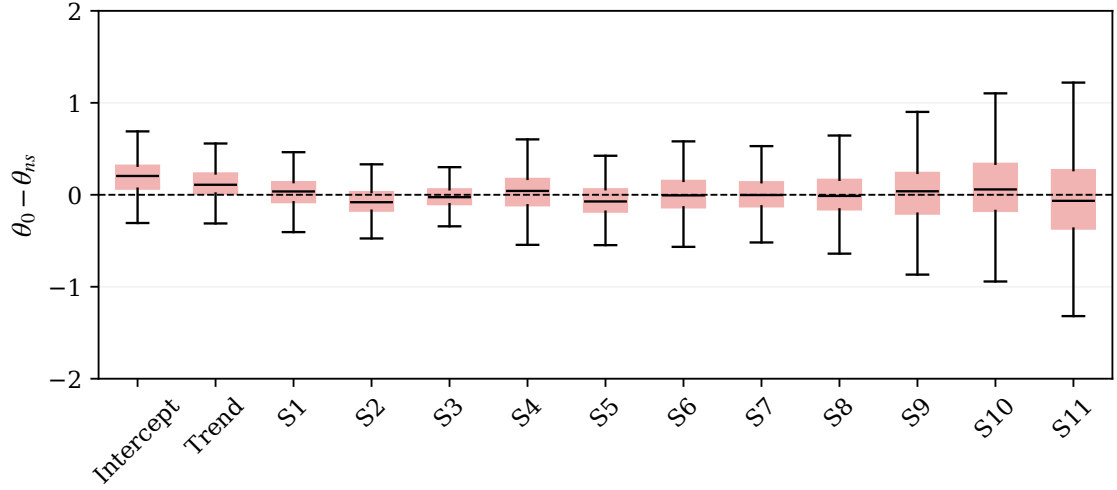
Fig. 6.8. Distribution of the coefficient difference $\theta^{\text{CBF}} - \theta^{\text{NS}}$ across the evaluation set, at $\text{ctx} = 8$ (top) and $\text{ctx} = 30$ (bottom).

Figure 6.8 shows the distribution of $\theta^{\text{NeuroSym-CBF}} - \theta^{\text{NeuroSym (Unimodal)}}$ for each of the 13 symbolic coefficients at both context lengths. The picture is consistent with the boxplot comparison of Section 6.5.2: the intercept term concentrates the largest shifts, the trend and some seasonal components show moderate but non-negligible displacements, and the overall effect is substantially more pronounced at $\text{ctx} = 8$ than at $\text{ctx} = 30$, where the distributions tighten considerably around zero. This contrast across context lengths reinforces the interpretation that multimodal metadata fills an information gap that the autoregressive context cannot cover when the available history is short.

Figure 6.9 isolates the internal effect of the FiLM module by reporting the difference between the modulated and pre-modulation coefficients within NeuroSym-CBF. Before



(a) ctx = 8



(b) ctx = 30

Fig. 6.9. Distribution of the internal FiLM modulation ($\theta_{\text{mod}} - \theta_0$) within NeuroSym-CBF, at ctx = 8 (top) and ctx = 30 (bottom).

interpreting it is needed to state that in the proposed architecture, the pre-FiLM coefficient branch, which is derived solely from the time-series hidden state via a linear projection, is not directly supervised or regularised independently of the rest of the model. The forecasting loss is applied to the final modulated coefficients θ_{mod} , which means the pre-FiLM branch and the FiLM modulator are optimised jointly and co-adapt throughout training. As a consequence, the magnitude of the internal modulation visible in Figure 6.9 should not be interpreted as evidence that the pre-FiLM coefficients are poorly estimated; rather, it reflects a learned division of labour between the two branches in which the pre-FiLM projection and the FiLM correction jointly converge to the final coefficient values.

For the same reason, a direct comparison between the pre-FiLM coefficients and those of the unimodal ablation would not be meaningful, since the ablation optimises its coefficients directly whereas the pre-FiLM branch in NeuroSym-CBF is never required to

produce good forecasts on its own.

With this caveat in mind, Figure 6.9 shows that the FiLM module is not a passive component. The modulation applied to the intercept and, to a lesser extent, the trend and seasonal terms is substantial, indicating that the concept-conditioned (γ, β) pairs produced by the FiLM MLP actively reshape the coefficient space rather than leaving it unchanged. As with Figure 6.8, the effect is stronger at $\text{ctx} = 8$, which is consistent with the model learning to rely more heavily on the metadata pathway when the time-series context is limited.

6.5.4. Concept Intervention

The previous subsections have shown that multimodal metadata and the FiLM module systematically reshape the symbolic coefficients of the model. A natural next question is whether the individual concepts produced by the SAE are themselves meaningful units of behaviour. In particular, if a single concept is silenced, does the resulting forecast change in a coherent and predictable manner?

To test this, the two most active SAE concepts on the evaluation set are identified by ranking concepts by their mean activation. For each of these concepts, the top 10% of samples in which the concept fires most strongly are selected as the intervention group ($n = 19,445$). On this group, two forward passes are performed: one with the model running as usual, and one in which the activation of the target concept is forced to zero before reaching the FiLM modulator, leaving all other concepts untouched. The difference between the two outputs isolates the causal effect of that single concept on both the forecast trajectory and the symbolic coefficients:

$$\Delta\hat{y} = \hat{y}_{\text{on}} - \hat{y}_{\text{off}}, \quad (6.8)$$

$$\Delta\theta = \theta_{\text{on}} - \theta_{\text{off}}. \quad (6.9)$$

A positive value indicates that the active concept pushes the forecast (or the corresponding symbolic coefficient) upwards relative to a counterfactual world in which the concept had not fired.

Figures 6.10 and 6.11 report two representative concepts per context length, deliberately chosen to illustrate qualitatively different behaviours. Each row of a figure corresponds to one concept and contains two panels: the mean $\Delta\hat{y}$ across the horizon with the interquartile band (left), and the mean $\Delta\theta$ for each of the 13 symbolic coefficients with one-standard-deviation error bars (right).

At $\text{ctx} = 8$, Concept 99 produces a strongly positive and monotonically increasing $\Delta\hat{y}$, reaching approximately +0.4 at the end of the horizon. The coefficient panel makes the mechanism explicit: the effect is concentrated almost entirely on the intercept and trend terms, with the seasonal coefficients essentially unaffected. Concept 30, by contrast, produces a negative and clearly non-monotonic $\Delta\hat{y}$ that resembles a downward-shifted sinusoidal wave: the effect deepens progressively, reaches its trough around horizon step

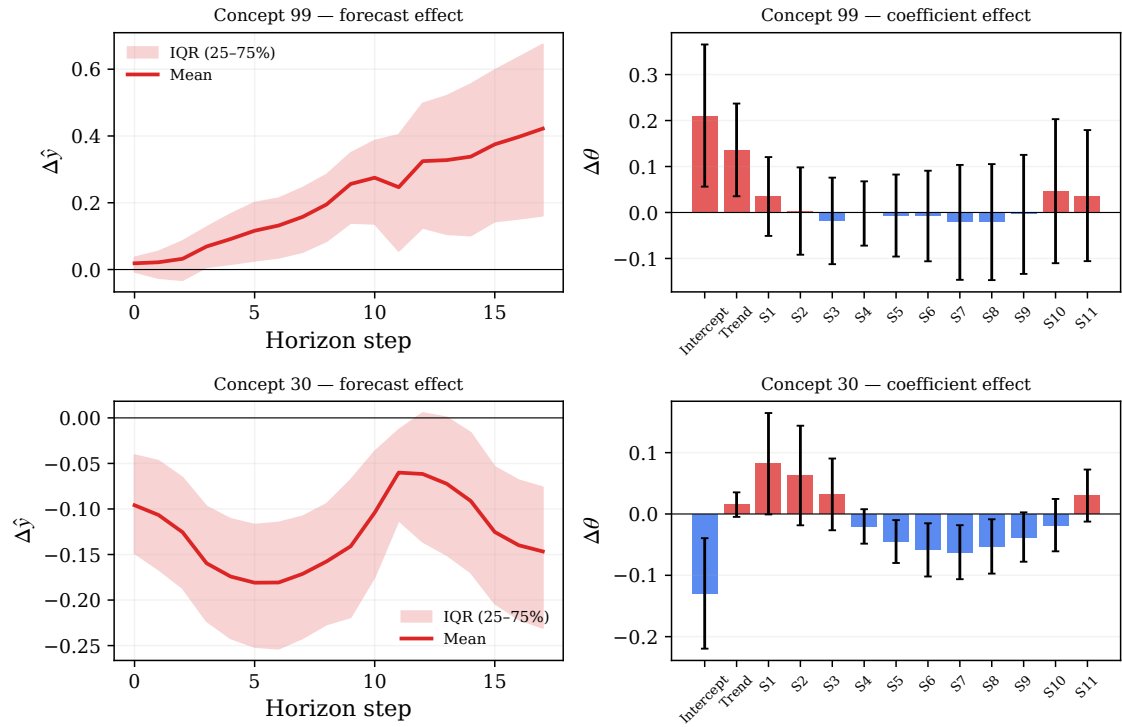


Fig. 6.10. Concept intervention at $\text{ctx} = 8$. Each row shows one SAE concept: mean $\Delta\hat{y}$ across the horizon with interquartile band (left) and mean $\Delta\theta$ per symbolic coefficient with one-standard-deviation error bars (right). Concept 99 (top) produces a positive trend-driven uplift; Concept 30 (bottom) a negative oscillatory effect distributed across the seasonal coefficients.

10–12, and partially recovers towards the end of the horizon. The corresponding coefficient effect is spread across the intercept and several seasonal components with alternating signs, which is consistent with a concept that encodes a cyclical or oscillatory.

At $\text{ctx} = 30$, the same kinds of behaviour are observed but with attenuated magnitudes, in line with the pattern established in the previous subsections. Concept 58 mirrors the role of Concept 99 in the short-context regime: a positive, monotonically increasing $\Delta\hat{y}$ driven by the intercept and trend, but with a final magnitude of around +0.2 rather than +0.4. Concept 81 produces a negative effect that is strongest at the early horizon steps and partially recovers towards the end, with a coefficient profile that mixes a negative intercept shift with positive contributions on the mid-range seasonal terms. As before, when the autoregressive context is richer, the individual concepts retain their interpretability but exert smaller corrections on the forecast.

These results support two claims that are central to the contribution of this work:

- The SAE concepts are not arbitrary: even though the model is trained on the forecasting objective alone, with no concept supervision or auxiliary reconstruction loss, silencing a single concept produces a coherent and structurally interpretable change in the model output, and the direction of that change can be read off the

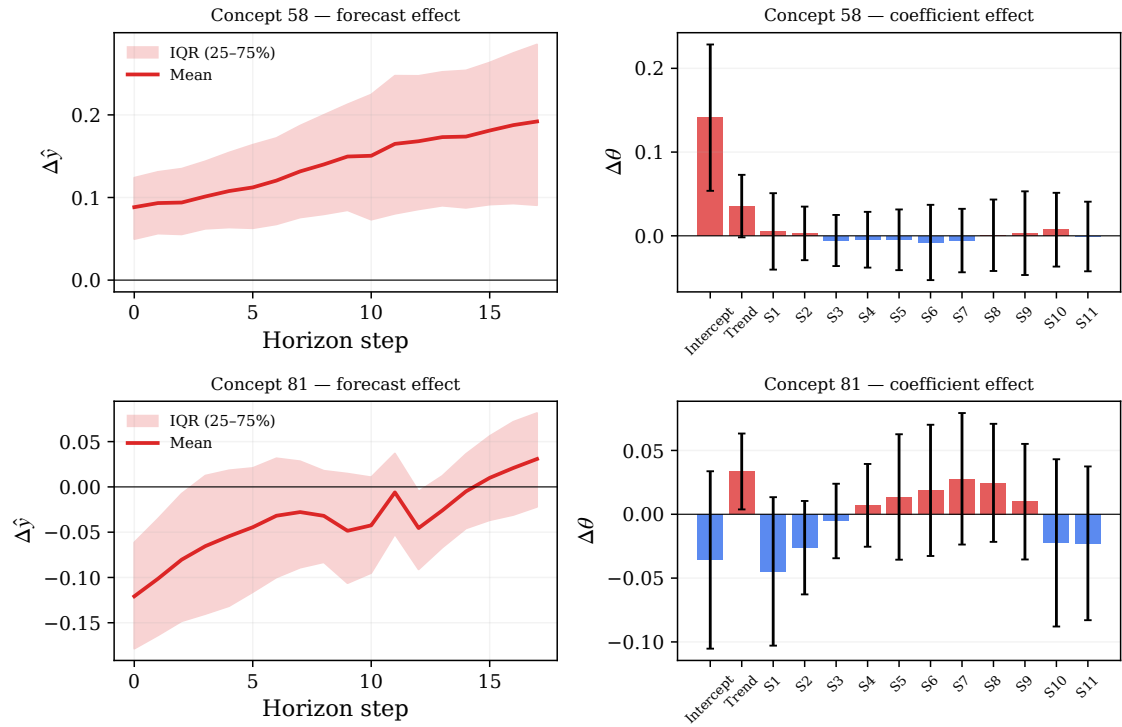


Fig. 6.11. Concept intervention at $\text{ctx} = 30$; same layout as Figure 6.10. Concept 58 (top) replicates the intercept-and-trend uplift of Concept 99 with attenuated magnitude; Concept 81 (bottom) produces a negative effect concentrated at early horizon steps with a mixed intercept and seasonal profile.

symbolic coefficient panel.

- The magnitude of the effect is concept-specific and depends on the autoregressive context, which is consistent with the role of multimodal metadata as a complementary information source.

A semantic characterisation of what each concept actually represents in terms of macroeconomic metadata is left out of scope with the idea of proposing it as a direction for future work in Chapter 9.

7. REGULATORY FRAMEWORK

This chapter examines the legal, ethical, and technical conditions that apply to the development and possible dissemination of the neuro-symbolic multimodal forecasting model described in this thesis. Four axes structure the discussion: the licensing terms of the underlying data source (FRED), the software and third-party model licences on which the implementation depends, what those terms jointly imply for publishing the dataset and the trained model, and the ethical and reproducibility standards that frame the use of macroeconomic data in machine learning.

The analysis is exploratory. Neither the dataset nor the trained model are published as part of this work, and the goal is to map the legal terrain so that future iterations of this research can proceed on solid footing.

7.1. Dataset licensing and the FRED Terms of Use

As described in Chapter 3, all data used in this work were obtained through the official FRED API of the Federal Reserve Bank of St. Louis [39], accessed with a registered API key. The numerical time series and the metadata tags retrieved with them fall under the FRED Services Terms of Use [58], which set the contractual baseline for any use, storage, or redistribution of FRED content.

7.1.1. The June 2024 update and the scientific TDM exception

On 18 June 2024 the Federal Reserve Bank of St. Louis revised the FRED Terms of Use [59]. The new version added a clause prohibiting the use of FRED Services or FRED Content "in connection with the development or training of any software program or system or machine learning, including, but not limited to, large language models, deep learning, generative artificial intelligence, or any other program or process commonly known as artificial intelligence" [58]. A parallel clause was added to the additional terms governing the FRED API, and a second prohibition was introduced restricting the storage, caching, or archiving of FRED content and its incorporation into any external database or compilation. Read literally, these clauses cover most of a modern machine learning pipeline that uses FRED data, from local persistence to model training and weight redistribution. Similar terms were added during 2023 and 2024 by other major data providers such as Reddit, The New York Times, and Penguin Random House in response to the rise of generative AI. The FRED version is unusually broad: it draws no distinction between the commercial training of foundation models and small-scale academic research, and it includes no explicit research or educational carve out.

That said, the contractual prohibition is not the final word for a research project car-

ried out within a Spanish university. Directive (EU) 2019/790 on copyright and related rights in the Digital Single Market [60] establishes in its Article 3 a mandatory exception for text and data mining (TDM) performed by research organisations for scientific research on works to which they have lawful access. Article 7(1) of the same Directive declares any contractual provision contrary to that exception unenforceable. The Spanish transposition, Real Decreto-ley 24/2021 [61], implements the exception in its Article 67, with universities included among recognised research organisations under Article 66. Specialised commentary on the Spanish transposition [62] argues that the imperative character of the research exception holds under the direct effect of the Directive even where the contract is governed by foreign law. The training pipeline of this thesis, conducted within a Spanish university for non-commercial scientific research and based on lawful API access, can therefore rely on a statutory exception that the FRED contractual ban cannot override. This legal basis covers the acts of reproduction, extraction, and analysis required to train and evaluate the proposed model. It does not authorise the public redistribution of the input data or of derivative artefacts such as model weights, which Section 7.2 discusses separately.

7.1.2. Copyright statuses and citation requirements

Beyond the general Terms of Use, every individual FRED series carries one of three copyright statuses, shown both in the Tags section of the series page and in the official citation guidance [63]. Series tagged *Public Domain: Citation Requested* are not under copyright and can be reused with attribution. Series tagged *Copyrighted: Citation Required* require explicit acknowledgement of both FRED and the original data provider. Series tagged *Copyrighted: Pre-approval Required* cannot be reused outside personal or educational settings without express written permission from the third-party rights holder, and the St. Louis Fed cannot grant that permission on their behalf [63].

The dataset assembled in this thesis was not filtered by copyright status, since the research use considered here concerns model training under lawful access rather than redistribution. These distinctions are therefore outside the immediate scope of the present work, but could become relevant in future publication scenarios involving datasets or derived artefacts.

7.2. Model and dataset publication: an exploratory analysis

This section asks whether the artefacts produced in this thesis (the source code, the trained model weights, and the derived dataset) could be made public, and under what conditions. None of these artefacts is released as part of the present work; the discussion below is meant as a reference for any future continuation of the project.

7.2.1. Source code and architecture

Publishing the source code and the architectural description of the model raises no legal difficulty. The code is the intellectual property of the author, the architectural building blocks (Concept Bottleneck Models [26] and FiLM conditioning [34]) are publicly described in prior literature, and no FRED content travels with the code. A release under a permissive license such as MIT or Apache 2.0, accompanied by a data preparation guide that instructs third parties to fetch the series from FRED with their own API key, is consistent with the Terms of Use and follows the access pattern already used by FRED-MD and FRED-QD [11], [12].

7.2.2. Trained model weights

Releasing the trained weights is the legally ambiguous case. The learned weights of time-series models trained on FRED data numerically encode the statistical regularities present in the underlying dataset, rather than constituting a literal copy or compilation of the data themselves [64]. The relevant prohibition in the Terms of Use refers to incorporating FRED content into a database, compilation, archive, cache, or other medium, and whether a parametric tensor counts as such a medium is an open question without case law to settle it. The Spanish TDM exception [61] arguably permits the creation of the weights as a byproduct of scientific TDM, but does not expressly authorise their public redistribution. A defensible position, should publication become desirable in the future, is to release the weights under a non-commercial research license such as CC BY-NC 4.0 or OpenRAIL-M, with a clear statement of intended use restricting them to academic replication of this thesis and analogous non-commercial research. A more conservative alternative is to withhold the weights and release only the training code, leaving third parties to reproduce them from their own API access.

7.2.3. The derived dataset

Direct redistribution of the dataset assembled in this thesis is the most constrained scenario. The Terms of Use explicitly prohibit storing, archiving, or incorporating FRED content into external databases or compilations, and the TDM exception does not displace that prohibition, since it covers extraction and analysis but not downstream redistribution. Two precedents show the narrow space within which redistribution has been achieved in practice. The FRED-MD and FRED-QD datasets [11], [12] are hosted by the St. Louis Fed itself, which implies institutional authorisation. The BVAR R package [65] ships a curated subset of FRED-MD and FRED-QD under a modified ODC-BY 1.0 license. A third pathway is the use of already-published derivative datasets such as the Monash Time Series Forecasting Archive [9], which includes a FRED-MD subset under CC BY 4.0.

Three pragmatic paths emerge for any future redistribution. The first is to refrain from

redistributing the dataset and rely on the data preparation scripts so that any interested researcher can rebuild it from the FRED API. The second is to release only the subset of series tagged Public Domain: Citation Requested, with a clear dataset card, the appropriate Suggested Citations, and a license modelled on the BVAR precedent. The third option, and arguably the cleanest in a research context, is to contact the St. Louis Fed directly, present the project, and request explicit authorisation for the publication of the derived dataset. Direct engagement with data providers is common practice in academic research, and the BVAR case shows that the St. Louis Fed has granted similar permissions when approached. Should the project continue, a sensible sequence is to release the source code first, then engage with the St. Louis Fed regarding the dataset, and only afterwards revisit whether and how to release the trained weights.

7.2.4. Software dependencies and third-party model licences

All software used in this work carries an OSI-approved permissive licence with no copy-left obligation on derivative work. The model is implemented in PyTorch [66] (BSD-3-Clause), with scikit-learn [67] (BSD-3-Clause) and statsforecast [55] (Apache-2.0) supporting evaluation and the AutoETS baseline. The only third-party model embedded in NeuroSym-CBF is the frozen T5-base encoder [43], loaded through HuggingFace Transformers [68] and released by Google under Apache-2.0. This is the licence that would govern a future weight release: Apache-2.0 expressly permits redistribution of derivative works, including fine-tuned or composite weights, provided the licence text and attribution notices are preserved, so the embedded encoder requires acknowledgement but raises no further obstacle. Chronos-2 [22], Chronos-bolt-base [20] and TimesFM-200M [21], also Apache-2.0, are used only to generate baseline forecasts and are not incorporated into NeuroSym-CBF.

7.3. Ethical considerations in macroeconomic data usage

The ethical framework adopted in this thesis combines the official guidance of the St. Louis Fed on the ethical use of FRED [69], covering respect for applicable law and for the communities affected by data work, privacy and confidentiality, honesty, accountability, transparency, and ongoing engagement with developments in data management. In practice, this means relying on the official API, rather than scraping the website, citing every series with the official Suggested Citation, never modifying FRED visualisations in ways that could suggest institutional endorsement, and documenting the limitations of the analysis openly.

Macroeconomic forecasts carry weight beyond their technical content. They feed policy discussions, investment decisions, and public expectations. The dataset used in this thesis is entirely non-personal, so privacy risks are negligible, but other concerns remain. The series available through FRED are heavily concentrated on the United States and

on advanced economies, which limits the generalisability of any model trained on them and introduces a representativeness bias that should be acknowledged when interpreting results.

Machine learning models for time series can also produce confident point forecasts that mask substantial uncertainty, and reporting forecasts without proper uncertainty quantification can mislead downstream users. The evaluation methodology adopted in this thesis and discussion of limitations in the results chapter are all consistent with this posture.

8. SOCIO-ECONOMIC ENVIRONMENT

This chapter covers the project from outside the technical work. The first section goes through how the six months of development were planned and organised. The second estimates what carrying it out actually cost. The third reflects on what a system like this could mean in practice, and where it might create problems as much as it solves them.

8.1. Project Planning and Timeline

The development of this thesis spans six months, from December 2025 to May 2026, organised into six phases that are shown in Figure 8.1. Most phases overlap with their neighbours, which reflects how the project actually worked: architectural choices forced revisiting the dataset pipeline, benchmark results fed back into model design, and writing happened throughout rather than at the end. The main tools used were Python with PyTorch and scikit-learn, the FRED API for data acquisition, a T5-based encoder for metadata embeddings, Kaggle for training, and Overleaf for writing in L^AT_EX.

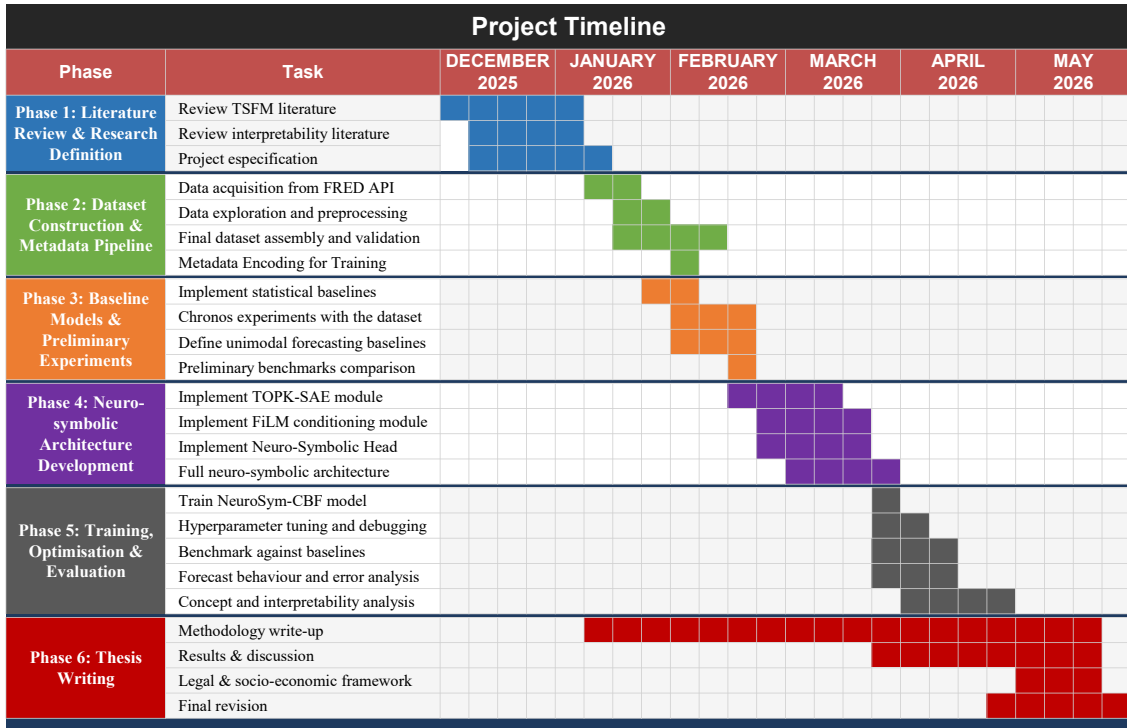


Fig. 8.1. Gantt chart of the project.

The first phase, running through December and January, was about building enough background to define a defensible research direction. Time series forecasting was an unfamiliar field, so a significant part of the effort went into understanding what the literature had actually established and where the open problems were. The hardest part was

conceptual: grasping how a model built around an interpretable symbolic equation could genuinely compete with more opaque approaches, rather than just being a principled but weaker alternative.

Phase 2 focused on constructing the dataset. Time series and metadata were pulled from the FRED API, which imposed a rate limit of 120 requests per minute that slowed things down considerably, especially when fetching tags. The more demanding work, though, was designing the metadata pipeline: deciding which descriptors to keep, how to represent them as a coherent text input per series, and how to encode that into dense embeddings via a T5-based model. Phases 3 and 4 ran largely in parallel across February to April. The baseline experiments with Chronos, TimesFM, and a unimodal trained model established the performance floor, while the neuro-symbolic architecture was being built at the same time. The real difficulty in Phase 4 was not implementing the individual modules but deciding where each one should sit within the forward pass, since those integration choices had direct consequences for interpretability and training behaviour.

The final two phases closed out the project. Phase 5 covered the full training and evaluation campaign, which was computationally intensive mostly because of the number of context lengths and the volume of metrics and plots required across all models. Phase 6 was not really a separate phase in practice: chapters were written as each block of work finished, with the notebooks serving as running documentation throughout.

8.2. Budget Estimation

The budget covers the full six-month development period and is broken into direct and indirect costs. Direct costs include all personnel and equipment that were actively used to produce the work. Indirect costs cover the background expenses that supported it.

8.2.1. Human costs

The largest cost driver is human time. The student dedicated an average of 25 hours per week over the six months of the project, totalling approximately 650 hours at a reference rate of 16€/h for a junior engineer. The thesis supervisor at UC3M held weekly one-hour sessions from February 2026 onward, contributing roughly 17 hours at 50€/h. An external research collaborator from the University of Sydney, consulted periodically throughout the project, contributed approximately 12 hours at a reference senior researcher rate of 100€/h. Table 8.1 summarises these figures.

8.2.2. Hardware and Software

The only hardware cost is the personal laptop used throughout the project, a Dell Latitude 7400 2-in-1 (Intel Core i7-8665U, 16 GB RAM, 512 GB SSD), originally priced at ap-

TABLE 8.1. HUMAN COSTS.

Role	Hours	Rate (€/h)	Cost (€)
Student	650	16	10,400€
UC3M Supervisor	17	50	850€
External Researcher (U. Sydney)	12	100	1,200€
Total			12,450€

proximately 2,580€ [70]. Assuming a useful life of three years and distributing the cost proportionally over the six-month project duration, the amortised contribution is 430€. Model training and evaluation were carried out on Google Kaggle’s free-tier GPU environment, which incurred no direct cost. Thesis writing was done in Overleaf under the UC3M institutional licence, also at no direct cost. Table 8.2 summarises the hardware and software costs.

TABLE 8.2. HARDWARE AND SOFTWARE COSTS.

Item	Amortisation	Cost (€)
Dell Latitude 7400 2-in-1	6/36 months	430€
Google Kaggle (free tier)	—	0€
Overleaf Premium (UC3M licence)	—	0€
Total		430€

Indirect Costs

Indirect costs cover electricity and internet connection over the six months of active development. The project was carried out while residing in shared accommodation in Sydney, Australia, with four occupants. Based on the average monthly electricity bill for a single-person household in Sydney of AUD 192 [71] and the average monthly NBN internet plan cost in Australia of AUD 85 [72], the proportional share for one occupant amounts to approximately AUD 69 per month. Applying an AUD-to-EUR conversion rate of 0.60, this yields roughly 41€ per month, rounded to 40€ for estimation purposes. Over six months, the total indirect cost is estimated at 240€.

TABLE 8.3. INDIRECT COSTS.

Concept	Duration	Monthly rate (€)	Cost (€)
Electricity and internet	6 months	40€	240€
Total			240€

8.2.3. Total Budget

Table 8.4 presents the full project budget. The total estimated cost is **13,120€**, the vast majority of which corresponds to human costs. This figure reflects what the project would cost if all contributions were compensated at standard market rates, even though several resources were provided at no direct cost to the student.

TABLE 8.4. TOTAL PROJECT BUDGET.

Cost category	Total (€)
Personnel	12,450€
Hardware and software	430€
Indirect costs	240€
Total	13,120€

8.3. Broader Socio-Economic Impact

The most direct application of this work is in settings where historical data is scarce. Many economically relevant series are short by nature: a new product launch, a recently created market index, or a macroeconomic indicator for an emerging economy may have very limited observations at the time a forecast is needed. In these situations, the textual description of a series (e.g. its title, category, units and institutional context) is typically available from the outset, even when its numerical history is not. This makes the approach directly relevant to a wide range of decision-makers: consumer goods companies forecasting demand for new products, financial institutions monitoring newly issued instruments, or national statistical offices in emerging economies that must produce timely projections with limited track records. For all of these, incorporating contextual metadata is not a refinement but more of a necessity.

A related application is macroeconomic nowcasting. Key economic indicators such as GDP or aggregate employment figures are published with a lag of several weeks or months after the reference period ends. In the meantime, policymakers, central banks, and analysts rely on intermediate estimates to inform decisions. A model that can condition on metadata describing what a series measures, its units, and its known structural properties can produce faster preliminary estimates that are more grounded than a purely numerical extrapolation. This type of interpretable, metadata-aware forecasting has clear institutional value in the context of central banks, finance ministries, and international economic organisations.

Beyond accuracy, the interpretability of the architecture matters in practice. Sectors such as banking, insurance, and public policy operate under regulatory frameworks that increasingly demand explainable predictions. A model that exposes its reasoning through a structured concept bottleneck is easier to audit and justify to regulators than a black-

box neural network, especially when both achieve similar accuracy. This is not a minor convenience: in many real deployment contexts, interpretability is not optional.

From an environmental standpoint, the trainable parameter count of NeuroSym-CBF is approximately 1.89×10^5 , which makes it lightweight to fine-tune and re-train relative to large foundation models, at the cost of delegating representational capacity to the frozen T5 encoder.

That said, this work also carries risks that should be acknowledged. A model producing interpretable macroeconomic forecasts might encourage over-reliance on automated outputs in contexts where human judgment and domain expertise should remain central. Concept activations can provide a false sense of transparency if the concepts themselves are not carefully validated against domain knowledge. And any forecasting system deployed at institutional scale without proper uncertainty quantification and human oversight creates the conditions for consequential errors to go undetected.

9. CONCLUSION

This final chapter closes the thesis by revisiting what has been built, what has been learned, and where the work could go next. Here, the contributions of the project and the main results that emerged from them are summarised, and the directions that remained outside the scope of this thesis but constitute natural extensions of the research line are detailed.

9.1. Summary of Contributions

This thesis set out with the objective to design a forecasting model that is interpretable by construction, that can incorporate textual metadata in a structured and editable way, and that remains competitive with the current generation of large pretrained time series foundation models. The work followed three parallel tracks: the construction of a metadata-enriched dataset, the design and training of the proposed architecture, and an extensive evaluation that compared it against strong baselines from across the forecasting spectrum.

The first contribution is the dataset itself. As discussed in Chapter 2, the time series forecasting field has largely treated series as anonymous numeric sequences, discarding the contextual information that accompanies them in practice. The dataset assembled in this work directly addresses that gap: 194,442 monthly macroeconomic series extracted from the FRED ecosystem, each paired with analyst curated metadata including its title, units, and a set of expanded tags. The construction pipeline described in Chapter 3 was designed for both the modelling part of this thesis but always with public release in mind, producing a clean, standardised resource that lowers the barrier to entry for future research in multimodal time series modelling. To the best of our knowledge, no comparable macroeconomic dataset providing structured, series level multimodal metadata is currently available, which makes this artefact a contribution in its own right.

The second contribution is NeuroSym-CBF, the neuro symbolic concept bottleneck forecaster introduced in Chapter 5. The architecture combines three key building blocks: a neuro-symbolic forecasting head that decomposes predictions into explicit intercept, trend and seasonal components through a closed-form symbolic decoder, a TopK sparse autoencoder acting as a concept bottleneck over a fused representation of the numeric and textual modalities [31], [50], and a FiLM modulator [34] that intervenes directly on the symbolic coefficients. The symbolic forecasting pipeline is supported by a compact patch based time series encoder inspired by [47], which provides an efficient temporal representation of the input sequence. The symbolic head ensures that the forecast is decomposable into interpretable temporal factors with a transparent reading for an analyst; the concept bottleneck provides an editable interface through which the metadata signal reaches the forecast; and the FiLM modulator constrains that interface to act through structured trans-

formations on parameters. The result is a model that is interpretable by design, and that exposes a controllable surface for human inspection and intervention.

The third contribution is the empirical evaluation reported in Chapter 6, which establishes that this interpretability comes at no cost in accuracy. NeuroSym-CBF achieves the best mean MAE and best MASE across all five evaluated context lengths, the best 90th percentile MAE without exception, and the lowest mean rank under three different metric aggregations. The improvements over the unimodal ablation, which shares the full neuro symbolic backbone but removes the multimodal pathway, range from 6.7% to 12.6% in mean MAE and confirm that the metadata conditioning is the source of the gain rather than the symbolic decomposition alone. These results are obtained with only 188,903 trainable parameters, roughly 639 times smaller than Chronos-2, the strongest zero shot foundation model evaluated in this work. While the comparison is not strictly equivalent, since models such as Chronos-2 operate in a zero-shot setting over broad pretraining distributions, the findings suggest that structured inductive biases and targeted multimodal conditioning may offer an alternative route to competitive forecasting performance in specialised domains where informative metadata is available.

Beyond the headline metrics, the interpretability analysis of Section 6.5 substantiates the architectural claims with concrete evidence. The concept space is well populated and functionally diverse; the symbolic coefficients shift systematically between the unimodal and multimodal variants of the model; the FiLM modulation actively reshapes the coefficient space; and silencing individual concepts produces coherent and structurally readable changes in the forecast. These evidences show that interpretability in NeuroSym-CBF is a property that emerges from the architecture and can be probed directly.

The combined impact of these contributions extends beyond the results. The model is particularly relevant for settings where historical data is scarce and where forecasts must be explainable, two conditions that arise routinely in macroeconomic nowcasting, the launch of new products and indicators, and any regulated environment where automated decisions are subject to scrutiny. Also, the radically smaller computational footprint of NeuroSym-CBF makes it deployable in resource constrained environments and aligns with the growing concern over the energy cost of large scale forecasting pipelines.

Taken together, the dataset, the model and the evaluation provide a coherent and self contained contribution that opens a productive research direction at the intersection of interpretable machine learning, multimodal modeling and time series forecasting.

9.2. Future Work and Limitations

The extensions described below are not independent of the current work’s limitations: most arise directly from design choices or constraints that could not be resolved within the scope of this thesis, and the line between what the model does not yet do and what it should do next is, in practice, the same line.

The most immediate extensions are the ones that do not change (a lot) the architecture. Although the current dataset focuses on monthly series, the architecture is not inherently tied to that frequency. Future work could make the seasonal period S configurable according to the time resolution specified in the metadata, allowing the same symbolic framework to operate on daily or quarterly data and substantially expanding the range of applicable series.

Alongside that, the dataset itself deserves to be published. The legal analysis in Chapter 7 maps the options; the most pragmatic route is to combine the public domain subset of FRED with macroeconomic repositories that carry permissive licensing by default, such as Eurostat and the OECD. The data pipeline, schema and metadata format described in Chapter 3 are designed with release in mind regardless of which path is taken.

A more significant gap is the absence of probabilistic forecasts. Chronos-2 [22], Chronos-bolt-base [20] and TimesFM [21] all produce full predictive distributions through quantile regression heads, and the comparison in Chapter 6 was therefore restricted to point-forecast metrics. Some preliminary experiments were conducted to assess whether the symbolic head could be extended to produce quantile forecasts: training NeuroSym-CBF with a pinball loss over the $\{0.5, 0.9\}$ quantiles at $T = 30$ yields a mean Q50 pinball of 0.190 and a mean Q90 pinball of 0.117, against 0.197 and 0.138 for Chronos-2 under the same evaluation. The 90% empirical coverage of NeuroSym-CBF reaches 85.8%, closer to the nominal level than the 83.5% obtained by Chronos-2. These are not systematic results and the comparison is not controlled for training conditions, but motivates the extension. A brief introduction to probabilistic forecasting metrics is provided in Appendix B.

On the interpretability side, the analysis in Section 6.5 leaves one open question deliberately unanswered: the actual meaning of the concepts. While the intervention experiments show that silencing a concept produces a coherent and interpretable change in the forecast, they do not assign semantic labels to individual dimensions. A natural next step is to characterise concepts post hoc by grouping series according to their strongest activations and inspecting the associated metadata. Concepts could then also be supervised: FRED tags provide a natural source of labels, enabling a hybrid bottleneck where some dimensions are aligned with predefined semantic attributes while others remain automatically discovered, in the spirit of [27]. Whether this additional structure improves interpretability without sacrificing forecast accuracy remains an empirical question.

The last direction is also the most open-ended. The architecture is not specific to macroeconomics: any setting where numeric series come paired with structured textual metadata is a candidate, from patient records in healthcare to product descriptions in retail or equipment specifications in energy systems. Whether the gains documented in Chapter 6 reflect something general about how textual context integrates with temporal structure, or are partly specific to the structure of FRED’s tag vocabulary, remains an open empirical question.

BIBLIOGRAPHY

- [1] R. Saqr, C. Bergmeir, B. Horvath, D. Schmidt, F. Rudzicz, and T. Lyons, “Seeking sota: Time-series forecasting must adopt taxonomy specific evaluation to dispel illusory gains,” *arXiv preprint arXiv:2603.15506*, 2026.
- [2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The m4 competition: Results, findings, conclusion and way forward,” *International Journal of forecasting*, vol. 34, no. 4, pp. 802–808, 2018.
- [3] F. Petropoulos et al., “Forecasting: Theory and practice,” *International Journal of Forecasting*, vol. 38, no. 3, pp. 705–871, 2022. doi: [10.1016/j.ijforecast.2021.11.001](https://doi.org/10.1016/j.ijforecast.2021.11.001). [Online]. Available: <https://doi.org/10.1016/j.ijforecast.2021.11.001>.
- [4] F. R. B. of St. Louis, *Federal reserve economic data (FRED)*, <https://fred.stlouisfed.org>, Accessed 2026-03-29.
- [5] J. Dong, H. Wu, Y. Wang, L. Zhang, J. Wang, and M. Long, *Metadata matters for time series: Informative forecasting with transformers*, 2024. arXiv: [2410.03806](https://arxiv.org/abs/2410.03806) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2410.03806>.
- [6] A. R. Williams et al., *Context is key: A benchmark for forecasting with essential textual information*, 2025. arXiv: [2410.18959](https://arxiv.org/abs/2410.18959) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2410.18959>.
- [7] F. X. Diebold and L. Kilian, “Measuring predictability: Theory and macroeconomic applications,” *Journal of Applied Econometrics*, vol. 16, no. 6, pp. 657–669, 2001. doi: <https://doi.org/10.1002/jae.619>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jae.619>.
- [8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The m4 competition: 100,000 time series and 61 forecasting methods,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020, M4 Competition. doi: <https://doi.org/10.1016/j.ijforecast.2019.04.014>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169207019301128>.
- [9] R. Godahewa, C. Bergmeir, G. I. Webb, R. J. Hyndman, and P. Montero-Manso, *Monash time series forecasting archive*, 2021. arXiv: [2105.06643](https://arxiv.org/abs/2105.06643) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2105.06643>.
- [10] T. Aksu et al., “GIFT-eval: A benchmark for general time series forecasting model evaluation,” in *NeurIPS Workshop on Time Series in the Age of Large Models*, 2024. [Online]. Available: <https://openreview.net/forum?id=Z2cMOOANFX>.
- [11] M. W. McCracken and S. Ng, “FRED-MD: A monthly database for macroeconomic research,” *Journal of Business & Economic Statistics*, vol. 34, no. 4, pp. 574–589, 2016. doi: [10.1080/07350015.2015.1086655](https://doi.org/10.1080/07350015.2015.1086655).

- [12] M. W. McCracken and S. Ng, “Fred-qd: A quarterly database for macroeconomic research,” *Review*, vol. 103, no. 1, pp. 1–44, Jan. 2021. doi: [10.20955/r.103.1-44](https://doi.org/10.20955/r.103.1-44). [Online]. Available: <https://ideas.repec.org/a/fip/fedlrv/90588.html>.
- [13] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco: Holden-Day, 1970.
- [14] R. G. Brown and A. D. Little, “Exponential smoothing: Forecasting and production planning for the tobacco industry,” ADL, A.D. Little; Philip Morris, chart; graph; report jzlc0130 (TID: dae94e00), Nov. 1956, Internal report; cited as Philip Morris Records, Industry Documents Archive, UCSF. [Online]. Available: <https://www.industrydocuments.ucsf.edu/docs/jzlc0130/>.
- [15] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and machine learning forecasting methods: Concerns and ways forward,” *PLoS ONE*, vol. 13, no. 3, e0194889, 2018. doi: [10.1371/journal.pone.0194889](https://doi.org/10.1371/journal.pone.0194889). [Online]. Available: <https://doi.org/10.1371/journal.pone.0194889>.
- [16] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, 1997. doi: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735). [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [17] G. Petneházi, *Recurrent neural networks for time series forecasting*, 2019. arXiv: [1901.00069](https://arxiv.org/abs/1901.00069) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1901.00069>.
- [18] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018. [Online]. Available: <https://arxiv.org/abs/1803.01271>.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, 2015. arXiv: [1512.03385](https://arxiv.org/abs/1512.03385) [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1512.03385>.
- [20] A. F. Ansari et al., “Chronos: Learning the language of time series,” *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=gerNCVqqtR>.
- [21] A. Das, W. Kong, R. Sen, and Y. Zhou, *A decoder-only foundation model for time-series forecasting*, 2024. arXiv: [2310.10688](https://arxiv.org/abs/2310.10688) [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.10688>.
- [22] A. F. Ansari et al., “Chronos-2: From univariate to universal forecasting,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.15821>.

- [23] P. Hitzler, A. Eberhart, M. Ebrahimi, M. K. Sarker, and L. Zhou, “Neuro-symbolic approaches in artificial intelligence,” *National Science Review*, vol. 9, no. 6, nwac035, Jun. 2022. doi: [10.1093/nsr/nwac035](https://doi.org/10.1093/nsr/nwac035). eprint: <https://academic.oup.com/nsr/article-pdf/9/6/nwac035/43952953/nwac035.pdf>. [Online]. Available: <https://doi.org/10.1093/nsr/nwac035>.
- [24] B. P. Bhuyan, A. Ramdane-Cherif, R. Tomar, and T. P. Singh, “Neuro-symbolic artificial intelligence: A survey,” *Neural Computing and Applications*, vol. 36, no. 21, pp. 12 809–12 844, 2024. doi: [10.1007/s00521-024-09960-z](https://doi.org/10.1007/s00521-024-09960-z). [Online]. Available: <https://link.springer.com/article/10.1007/s00521-024-09960-z>.
- [25] B. N. Oreshkin, D. Carпов, N. Chapados, and Y. Bengio, “N-BEATS: neural basis expansion analysis for interpretable time series forecasting,” *CoRR*, vol. abs/1905.10437, 2019. arXiv: [1905.10437](https://arxiv.org/abs/1905.10437). [Online]. Available: <http://arxiv.org/abs/1905.10437>.
- [26] P. W. Koh et al., “Concept bottleneck models,” *CoRR*, vol. abs/2007.04612, 2020. arXiv: [2007.04612](https://arxiv.org/abs/2007.04612). [Online]. Available: <https://arxiv.org/abs/2007.04612>.
- [27] T. A. Opsahl, “Hybrid concept-based models: Using concepts to improve neural networks’ accuracy,” in *Northern Lights Deep Learning Conference 2026*, 2026. [Online]. Available: <https://openreview.net/forum?id=Np3uIwXlt0>.
- [28] C. Álvarez-Rodríguez, E. Parrado-Hernández, J. Pérez-Aracil, L. Prieto-Godino, and S. Salcedo-Sanz, “Interpretable extreme wind speed prediction with concept bottleneck models,” *Renewable Energy*, vol. 231, p. 120 935, 2024. doi: <https://doi.org/10.1016/j.renene.2024.120935>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0960148124010036>.
- [29] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, *Sparse autoencoders find highly interpretable features in language models*, 2023. arXiv: [2309.08600](https://arxiv.org/abs/2309.08600) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2309.08600>.
- [30] T. Bricken et al., *Towards monosemanticity: Decomposing language models with dictionary learning*, Transformer Circuits Thread, 2023. [Online]. Available: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [31] L. Gao et al., *Scaling and evaluating sparse autoencoders*, 2024. arXiv: [2406.04093](https://arxiv.org/abs/2406.04093) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2406.04093>.
- [32] K. Oublal, Q. Bouniot, Q. Gan, S. Cléménçon, and Z. Akata, “TimeSAE: Sparse decoding for faithful explanations of black-box time series models,” *arXiv preprint arXiv:2601.09776*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.09776>.

- [33] F. Divo, M. Kraus, R. Härle, P. Ahrend, and K. Kersting, “Finding the zeitgeist in time series foundation models,” in *1st ICLR Workshop on Time Series in the Age of Large Models*, 2026. [Online]. Available: <https://openreview.net/forum?id=7SP3orvHPy>.
- [34] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. C. Courville, “Film: Visual reasoning with a general conditioning layer,” *CoRR*, vol. abs/1709.07871, 2017. arXiv: [1709.07871](https://arxiv.org/abs/1709.07871). [Online]. Available: <http://arxiv.org/abs/1709.07871>.
- [35] L. Zhang, L. Shen, Y. Zheng, S. Piao, Z. Li, and F. Tsung, *Lemole: Llm-enhanced mixture of linear experts for time series forecasting*, 2024. arXiv: [2412.00053](https://arxiv.org/abs/2412.00053) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2412.00053>.
- [36] Y. Li et al., *Apt: Affine prototype-timestamp for time series forecasting under distribution shift*, 2025. arXiv: [2511.12945](https://arxiv.org/abs/2511.12945) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2511.12945>.
- [37] X. Zhou, W. Wang, S. Qu, Z. Zhang, and C. Bergmeir, “Unveiling the potential of text in high-dimensional time series forecasting,” *arXiv preprint arXiv:2501.07048*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.07048>.
- [38] X. Zhou, W. Wang, F. J. Baldán, W. L. Buntine, and C. Bergmeir, “MoTime: A dataset suite for multimodal time series forecasting,” *arXiv preprint arXiv:2505.15072*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.15072>.
- [39] F. R. B. of St. Louis, *Fred api documentation*, <https://fred.stlouisfed.org/docs/api/fred/>, Accessed: 2026-03-30.
- [40] R. J. A. Little and D. B. Rubin, *Statistical Analysis with Missing Data*, 2nd. New York: Wiley, 2002. doi: [10.1002/9781119013563](https://doi.org/10.1002/9781119013563).
- [41] C. John, E. Ekpenyong, and C. Nworu, “Imputation of missing values in economic and financial time series data using five principal component analysis (pca) approaches,” *Central Bank of Nigeria Journal of Applied Statistics*, pp. 51–73, Aug. 2019. doi: [10.33429/Cjas.10119.3/6](https://doi.org/10.33429/Cjas.10119.3/6).
- [42] N. Bokde, M. W. Beck, F. Martínez Álvarez, and K. Kulat, “A novel imputation methodology for time series based on pattern sequence forecasting,” *Pattern Recognition Letters*, vol. 116, pp. 88–96, 2018. doi: <https://doi.org/10.1016/j.patrec.2018.09.020>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167865518306500>.
- [43] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” 2023. arXiv: [1910.10683](https://arxiv.org/abs/1910.10683) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1910.10683>.
- [44] Q. Lhoest et al., “Datasets: A community library for natural language processing,” *CoRR*, vol. abs/2109.02846, 2021. arXiv: [2109.02846](https://arxiv.org/abs/2109.02846). [Online]. Available: <https://arxiv.org/abs/2109.02846>.

- [45] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=cGDAkQo1C0p>.
- [46] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021, Accessed on 2026-05-06. [Online]. Available: <https://otexts.com/fpp3/>.
- [47] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, *A time series is worth 64 words: Long-term forecasting with transformers*, 2023. arXiv: [2211.14730](https://arxiv.org/abs/2211.14730) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2211.14730>.
- [48] D. Hendrycks and K. Gimpel, *Gaussian error linear units (gelus)*, 2023. arXiv: [1606.08415](https://arxiv.org/abs/1606.08415) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1606.08415>.
- [49] A. Zeng, M. Chen, L. Zhang, and Q. Xu, *Are transformers effective for time series forecasting?* 2022. arXiv: [2205.13504](https://arxiv.org/abs/2205.13504) [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2205.13504>.
- [50] A. Makhzani and B. Frey, *K-sparse autoencoders*, 2014. arXiv: [1312.5663](https://arxiv.org/abs/1312.5663) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1312.5663>.
- [51] A. Jadon, A. Patil, and S. Jadon, *A comprehensive survey of regression based loss functions for time series forecasting*, 2022. arXiv: [2211.02989](https://arxiv.org/abs/2211.02989) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2211.02989>.
- [52] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006. doi: <https://doi.org/10.1016/j.ijforecast.2006.03.001>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0169207006000239>.
- [53] S. Makridakis, “Accuracy measures: Theoretical and practical concerns,” *International Journal of Forecasting*, vol. 9, no. 4, pp. 527–529, 1993. doi: [https://doi.org/10.1016/0169-2070\(93\)90079-3](https://doi.org/10.1016/0169-2070(93)90079-3). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0169207093900793>.
- [54] O. Shchur et al., *Fev-bench: A realistic benchmark for time series forecasting*, 2026. arXiv: [2509.26468](https://arxiv.org/abs/2509.26468) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2509.26468>.
- [55] F. Garza, M. Mergenthaler Canseco, C. Challú, and K. G. Olivares, *Statsforecast: Lightning fast forecasting with statistical and econometric models*, PyCon Salt Lake City, Utah, US, 2022. [Online]. Available: <https://github.com/Nixtla/statsforecast>.

- [56] R. J. Hyndman, A. B. Koehler, R. D. Snyder, and S. Grose, “A state space framework for automatic forecasting using exponential smoothing methods,” *International Journal of Forecasting*, vol. 18, no. 3, pp. 439–454, Jan. 2002. [Online]. Available: <https://ideas.repec.org/a/eee/intfor/v18y2002i3p439-454.html>.
- [57] S. Makridakis et al., “The accuracy of extrapolation (time series) methods: Results of a forecasting competition,” *Journal of Forecasting*, vol. 1, no. 2, pp. 111–153, 1982. doi: <https://doi.org/10.1002/for.3980010202>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/for.3980010202>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/for.3980010202>.
- [58] Federal Reserve Bank of St. Louis, *Legal notices, information and disclaimers: FRED® services terms of use*, Updated 18 June 2024; accessed 3 March 2026, 2024. [Online]. Available: <https://fred.stlouisfed.org/legal/>.
- [59] Y. Fortova, *We’ve updated our terms of use: Action requested*, Posted 18 June 2024, Jun. 2024. [Online]. Available: <https://news.research.stlouisfed.org/2024/06/weve-updated-our-terms-of-use-action-requested/>.
- [60] European Union, *Directive (EU) 2019/790 of the european parliament and of the council of 17 april 2019 on copyright and related rights in the Digital Single Market and amending directives 96/9/EC and 2001/29/EC*, Official Journal of the European Union, L 130/92, 17 May 2019, 2019.
- [61] Jefatura del Estado (Spain), *Real decreto-ley 24/2021, de 2 de noviembre, de transposición de directivas de la unión europea en las materias de bonos garantizados, distribución transfronteriza de organismos de inversión colectiva, datos abiertos y reutilización de la información del sector público, ejercicio de derechos de autor y derechos afines aplicables a determinadas transmisiones en línea y a las retransmisiones de programas de radio y televisión, exenciones temporales a determinadas importaciones y suministros, de personas consumidoras y para la promoción de vehículos de transporte por carretera limpios y energéticamente eficientes*, Boletín Oficial del Estado, BOE-A-2021-17910, 3 November 2021, 2021.
- [62] P. A. de Miguel Asensio, *Real decreto-ley 24/2021 y sociedad de la información: Derechos de autor*, Blog Pedro A. de Miguel Asensio, Nov. 2021. [Online]. Available: <https://pedrodemiguelasensio.blogspot.com/2021/11/real-decreto-ley-242021-y-sociedad-de.html>.
- [63] Federal Reserve Bank of St. Louis, *Cite the data — Getting to Know FRED®*, Accessed 7 March 2026. [Online]. Available: <https://fredhelp.stlouisfed.org/fred/graphs/share-my-fred-graph/cite/>.

- [64] A. Naik, Y. Wu, M. Naik, and E. Wong, “Do machine learning models learn statistical rules inferred from data?” In *Proceedings of the 40th International Conference on Machine Learning*, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., ser. Proceedings of Machine Learning Research, vol. 202, PMLR, Jul. 2023, pp. 25 677–25 693. [Online]. Available: <https://proceedings.mlr.press/v202/naik23a.html>.
- [65] N. Kuschnig and L. Vashold, “Bvar: Bayesian vector autoregressions with hierarchical prior selection in r,” *Journal of Statistical Software*, vol. 100, no. 14, pp. 1–27, 2021. doi: [10.18637/jss.v100.i14](https://doi.org/10.18637/jss.v100.i14). [Online]. Available: <https://www.jstatsoft.org/index.php/jss/article/view/v100i14>.
- [66] A. Paszke et al., “Pytorch: An imperative style, high-performance deep learning library,” *CoRR*, vol. abs/1912.01703, 2019. arXiv: [1912.01703](https://arxiv.org/abs/1912.01703). [Online]. Available: <http://arxiv.org/abs/1912.01703>.
- [67] F. Pedregosa et al., “Scikit-learn: Machine learning in python,” *CoRR*, vol. abs/1201.0490, 2012. arXiv: [1201.0490](https://arxiv.org/abs/1201.0490). [Online]. Available: <http://arxiv.org/abs/1201.0490>.
- [68] T. Wolf et al., “Huggingface’s transformers: State-of-the-art natural language processing,” *CoRR*, vol. abs/1910.03771, 2019. arXiv: [1910.03771](https://arxiv.org/abs/1910.03771). [Online]. Available: <http://arxiv.org/abs/1910.03771>.
- [69] D. Mendez-Carbajo, “Ethical use of data with FRED®,” *Federal Reserve Bank of St. Louis Page One Economics*, Oct. 2021. [Online]. Available: <https://www.stlouisfed.org/publications/page-one-economics/2021/10/15/ethical-use-of-data-with-fred>.
- [70] S. L. Smith, *Dell latitude 7400 2-in-1 review*, Laptop Mag, Price as reviewed: \$2,802 with Intel Core i7-8665U, 16GB RAM, 512GB SSD, May 2019. Accessed: May 22, 2024. [Online]. Available: <https://laptopmag.com>.
- [71] Australian Energy Regulator, *Electricity bill benchmarks for residential customers*, Australian Energy Regulator, Average quarterly electricity bill for Sydney households estimated at AUD 636.95, based on AER reference pricing data, 2026. Accessed: May 20, 2026. [Online]. Available: <https://www.aer.gov.au/consumers/your-electricity-bill/electricity-bill-benchmarks>.
- [72] Canstar Blue, *Average internet bill per month in Australia*, Canstar Blue, Average monthly NBN plan cost in Australia reported as AUD 85, based on customer satisfaction survey data across NBN providers, Jun. 2025. Accessed: May 20, 2026. [Online]. Available: <https://www.canstarblue.com.au/internet/average-internet-bill-nbn/>.

- [73] T. Gneiting and A. E. Raftery, “Strictly proper scoring rules, prediction, and estimation,” *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007. DOI: [10.1198/016214506000001437](https://doi.org/10.1198/016214506000001437). eprint: <https://doi.org/10.1198/016214506000001437>. [Online]. Available: <https://doi.org/10.1198/016214506000001437>.
- [74] J. E. Matheson and R. L. Winkler, “Scoring rules for continuous probability distributions,” *Management Science*, vol. 22, no. 10, pp. 1087–1096, 1976. Accessed: May 22, 2026. [Online]. Available: <http://www.jstor.org/stable/2629907>.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
BVAR	Bayesian Vector AutoRegression
CBM	Concept Bottleneck Model
CPI	Consumer Price Index
CRPS	Continuous Ranked Probability Score
ETS	Error, Trend and Seasonality model
FiLM	Feature-wise Linear Modulation
FRED	Federal Reserve Economic Data
FRED-MD	Federal Reserve Economic Data – Monthly Database
FRED-QD	Federal Reserve Economic Data – Quarterly Database
GDP	Gross Domestic Product
GELU	Gaussian Error Linear Unit
LLM	Large Language Model
LOCF	Last Observation Carried Forward
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MASE	Mean Absolute Scaled Error
M4	M4 Forecasting Competition
MLP	Multi-Layer Perceptron
OECD	Organisation for Economic Co-operation and Development
OHLC	Open, High, Low, Close
PCA	Principal Component Analysis
PCE	Personal Consumption Expenditures
RNN	Recurrent Neural Network
SAE	Sparse Autoencoder
sMAPE	Symmetric Mean Absolute Percentage Error
T5	Text-To-Text Transfer Transformer
TCN	Temporal Convolutional Network
TSFM	Time Series Foundation Model

APPENDIX A: SUPPLEMENTARY RESULTS

Per-Context Win-Rate Matrices

The following figures show the full pairwise win-rate matrices for each of the five context lengths evaluated. Each entry (i, j) gives the fraction of the $N = 194,442$ test windows in which row model i achieves a lower per-sample MAE than column model j . The matrices complement the context-averaged summary shown in Figure 6.1 of Subsection 6.3.2.

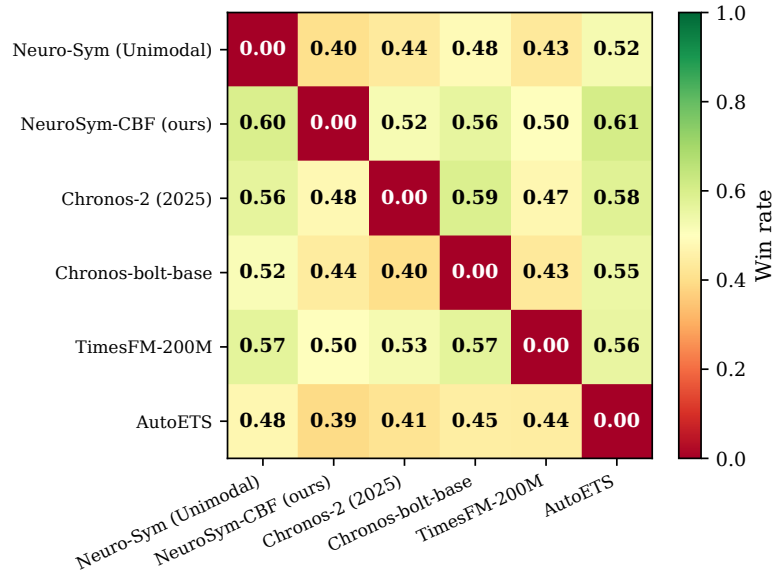


Fig. 1. Pairwise win-rate matrix at context length $T = 8$.

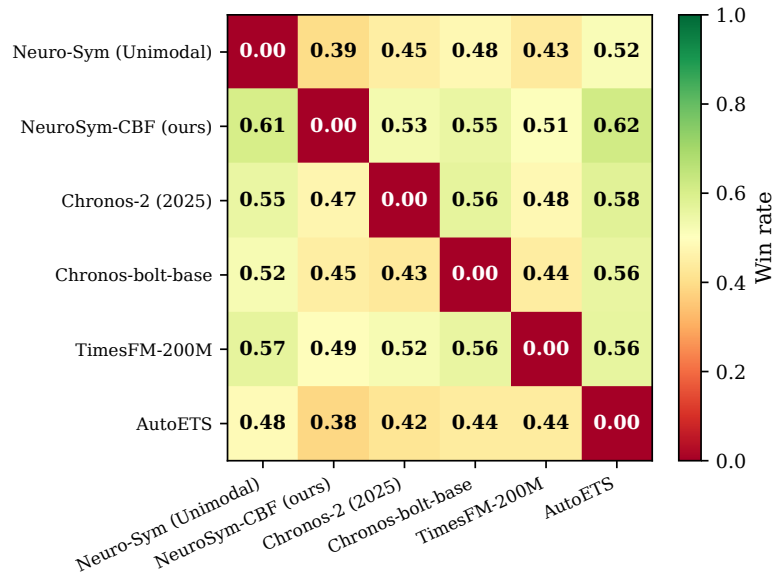


Fig. 2. Pairwise win-rate matrix at context length $T = 10$.

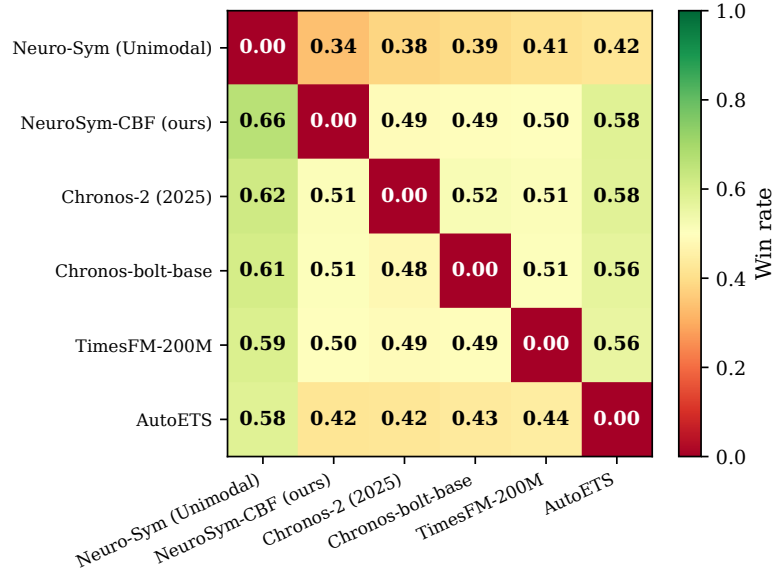


Fig. 3. Pairwise win-rate matrix at context length $T = 15$.

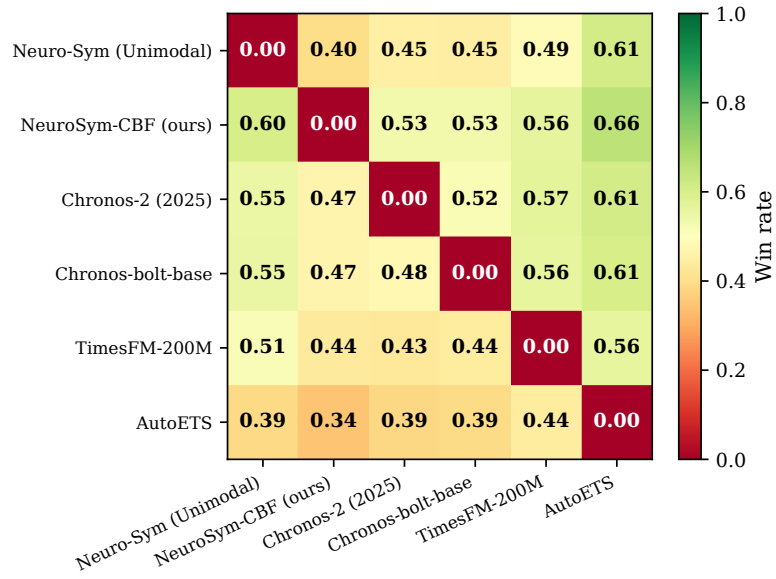


Fig. 4. Pairwise win-rate matrix at context length $T = 24$.

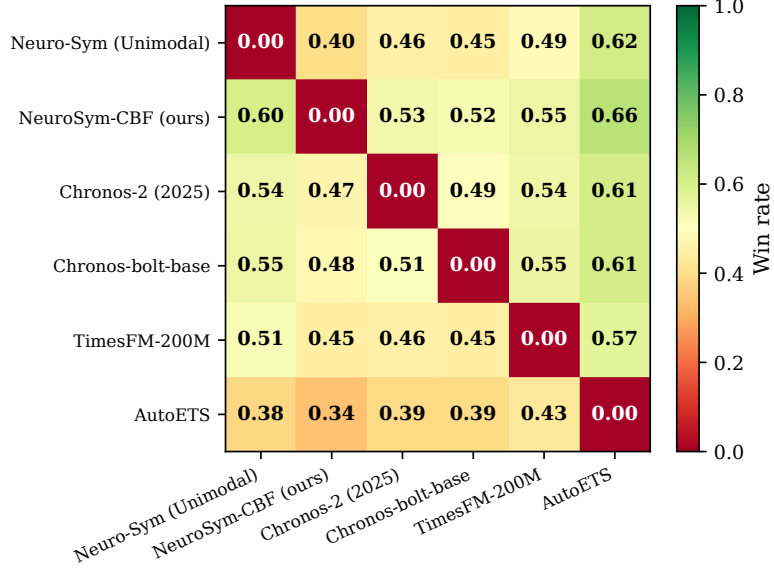


Fig. 5. Pairwise win-rate matrix at context length $T = 30$.

Per-Context Mean Rank Breakdown

Table 1 provides the per-context mean rank of each model under the three metric sets defined in Section 6.1, complementing the global summary in Table 6.2. Each cell is the average rank (1 = best, 6 = worst) of the model at that context length under the corresponding metric set.

TABLE 1. PER-CONTEXT MEAN RANK BREAKDOWN FOR ALL MODELS UNDER THREE METRIC SETS. EACH CELL IS THE AVERAGE RANK (1 = BEST, 6 = WORST) AT THAT CONTEXT UNDER THE CORRESPONDING METRIC SET.

Metric set	Model	$T = 8$	$T = 10$	$T = 15$	$T = 24$	$T = 30$	Mean
MAIN	NeuroSym-CBF	1.00	1.00	1.00	1.00	1.25	1.05
	Chronos-2	3.00	3.25	2.75	2.50	3.00	2.90
	Neuro-Sym (Unimodal)	2.75	2.25	5.25	3.75	4.50	3.70
	Chronos-bolt-base	5.50	5.50	4.00	3.00	2.00	4.00
	TimesFM-200M	4.00	4.25	4.00	5.00	4.25	4.30
	AutoETS	4.75	4.75	4.00	5.75	6.00	5.05
MEAN	NeuroSym-CBF	1.00	1.00	1.00	1.67	2.00	1.33
	Chronos-2	2.67	3.00	2.33	2.00	3.00	2.60
	Neuro-Sym (Unimodal)	3.00	2.33	6.00	4.33	4.67	4.07
	Chronos-bolt-base	5.33	5.33	3.33	2.33	1.67	3.60
	TimesFM-200M	3.33	3.67	3.33	4.67	3.67	3.73
	AutoETS	5.67	5.67	5.00	6.00	6.00	5.67
MEDIAN	NeuroSym-CBF	2.33	2.67	2.00	1.00	1.67	1.93
	Chronos-2	2.33	2.33	2.00	2.33	3.33	2.46
	Neuro-Sym (Unimodal)	4.00	4.00	6.00	5.00	5.00	4.80
	Chronos-bolt-base	4.67	4.33	3.33	2.67	1.33	3.27
	TimesFM-200M	2.00	2.00	3.00	4.00	3.67	2.93
	AutoETS	5.67	5.67	4.67	6.00	6.00	5.60

Detailed Horizon Step Analysis of MAE

Figures 6 to 10 show the MAE at each forecast step for all six models across all context lengths, complementing the two-panel figure in Subsection 6.3.3.

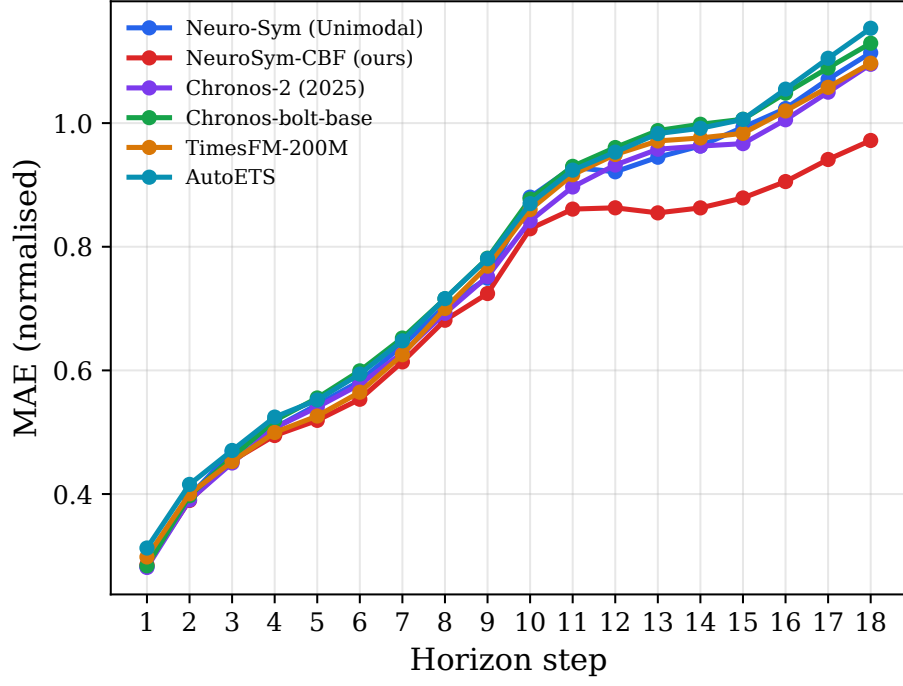


Fig. 6. MAE per horizon step at $T = 8$.

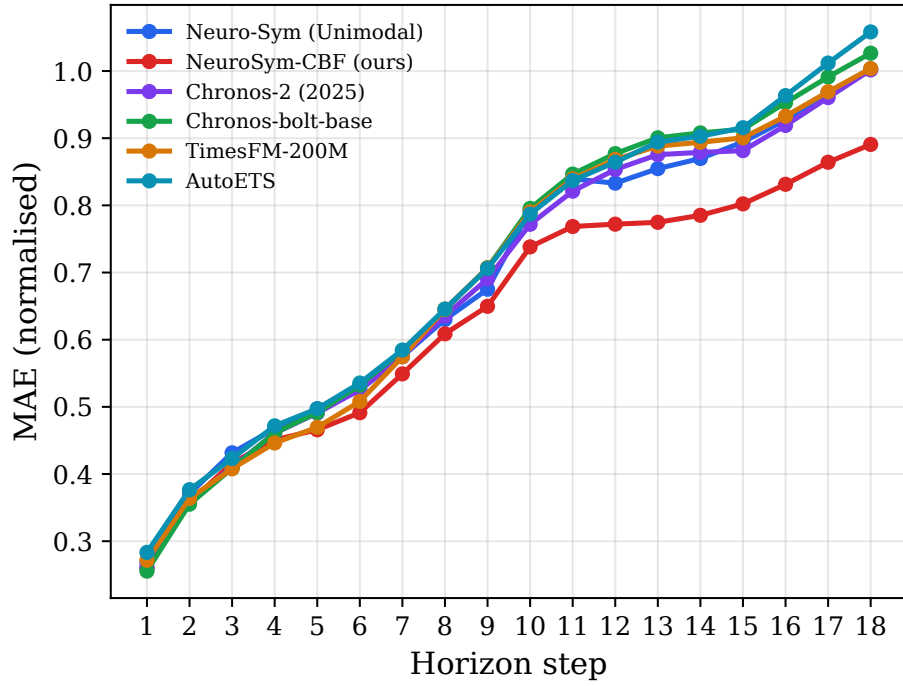


Fig. 7. MAE per horizon step at $T = 10$.

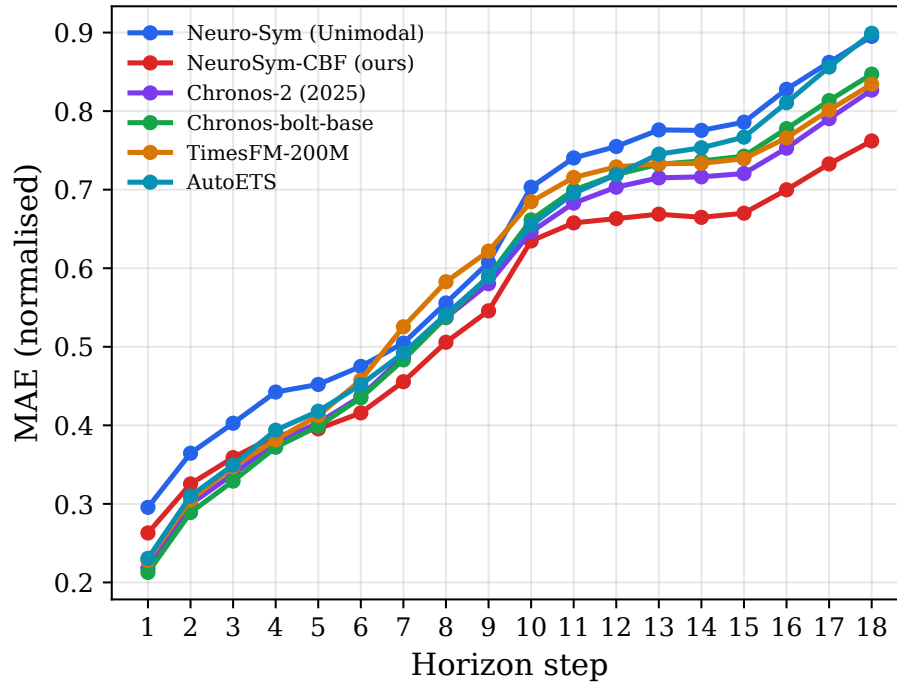


Fig. 8. MAE per horizon step at $T = 15$.

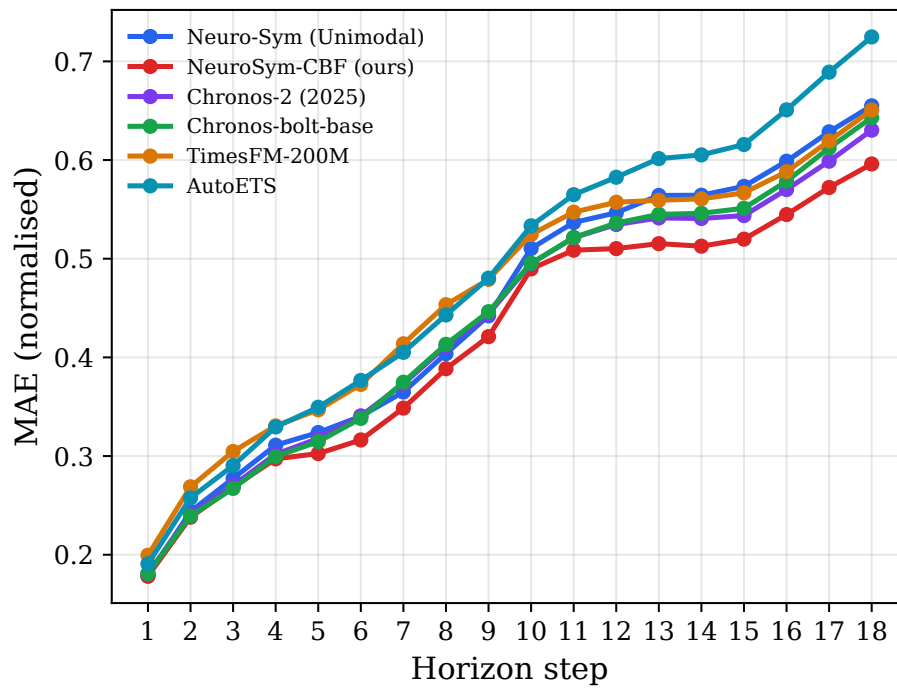


Fig. 9. MAE per horizon step at $T = 24$.

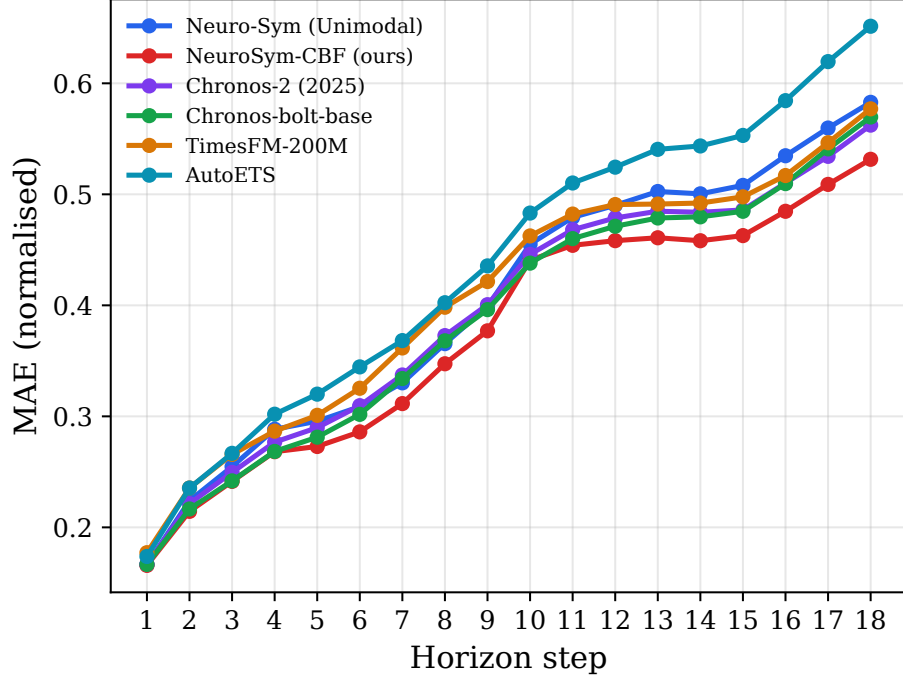


Fig. 10. MAE per horizon step at $T = 30$.

Also, Tables 2–6 provide the corresponding absolute values at selected steps $h \in \{1, 3, 6, 12, 18\}$.

TABLE 2. ABSOLUTE MAE AT SELECTED HORIZON STEPS,
 $T = 8$.

Model	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
NeuroSym-CBF	0.286	0.454	0.553	0.863	0.972
Neuro-Sym (Unimodal)	0.282	0.469	0.582	0.921	1.114
Chronos-2	0.281	0.450	0.577	0.932	1.095
Chronos-bolt-base	0.284	0.459	0.599	0.960	1.129
TimesFM-200M	0.298	0.453	0.565	0.949	1.097
AutoETS	0.313	0.470	0.594	0.953	1.153

TABLE 3. ABSOLUTE MAE AT SELECTED HORIZON STEPS,
 $T = 10$.

Model	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
NeuroSym-CBF	0.260	0.416	0.491	0.772	0.891
Neuro-Sym (Unimodal)	0.268	0.432	0.525	0.833	1.003
Chronos-2	0.261	0.410	0.525	0.853	1.002
Chronos-bolt-base	0.255	0.409	0.532	0.877	1.027
TimesFM-200M	0.272	0.408	0.508	0.868	1.004
AutoETS	0.283	0.423	0.535	0.864	1.058

TABLE 4. ABSOLUTE MAE AT SELECTED HORIZON STEPS,
 $T = 15$.

Model	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
NeuroSym-CBF	0.263	0.359	0.416	0.663	0.762
Neuro-Sym (Unimodal)	0.296	0.403	0.475	0.755	0.895
Chronos-2	0.218	0.338	0.437	0.703	0.827
Chronos-bolt-base	0.213	0.329	0.435	0.719	0.847
TimesFM-200M	0.228	0.347	0.458	0.729	0.834
AutoETS	0.231	0.349	0.452	0.719	0.899

TABLE 5. ABSOLUTE MAE AT SELECTED HORIZON STEPS,
 $T = 24$.

Model	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
NeuroSym-CBF	0.178	0.268	0.316	0.510	0.596
Neuro-Sym (Unimodal)	0.178	0.277	0.340	0.547	0.655
Chronos-2	0.181	0.270	0.341	0.535	0.630
Chronos-bolt-base	0.181	0.267	0.338	0.536	0.643
TimesFM-200M	0.200	0.305	0.373	0.557	0.651
AutoETS	0.191	0.290	0.377	0.583	0.725

TABLE 6. ABSOLUTE MAE AT SELECTED HORIZON STEPS,
 $T = 30$.

Model	$h = 1$	$h = 3$	$h = 6$	$h = 12$	$h = 18$
NeuroSym-CBF	0.166	0.241	0.286	0.458	0.532
Neuro-Sym (Unimodal)	0.166	0.255	0.309	0.490	0.583
Chronos-2	0.167	0.250	0.310	0.479	0.562
Chronos-bolt-base	0.167	0.242	0.302	0.471	0.570
TimesFM-200M	0.177	0.266	0.325	0.491	0.577
AutoETS	0.174	0.267	0.345	0.524	0.651

Additional Interpretability Results at $\text{ctx} = 30$

The interpretability analysis presented in Section 6.5 reports both context regimes ($\text{ctx} = 8$ and $\text{ctx} = 30$) side by side whenever they convey complementary information. In the cases where the two regimes produce qualitatively equivalent results, only the $\text{ctx} = 8$ version is shown in the main text and the $\text{ctx} = 30$ counterpart is collected in this appendix. These figures below are reported without further discussion, as the main observations have already been established in the body of the chapter.

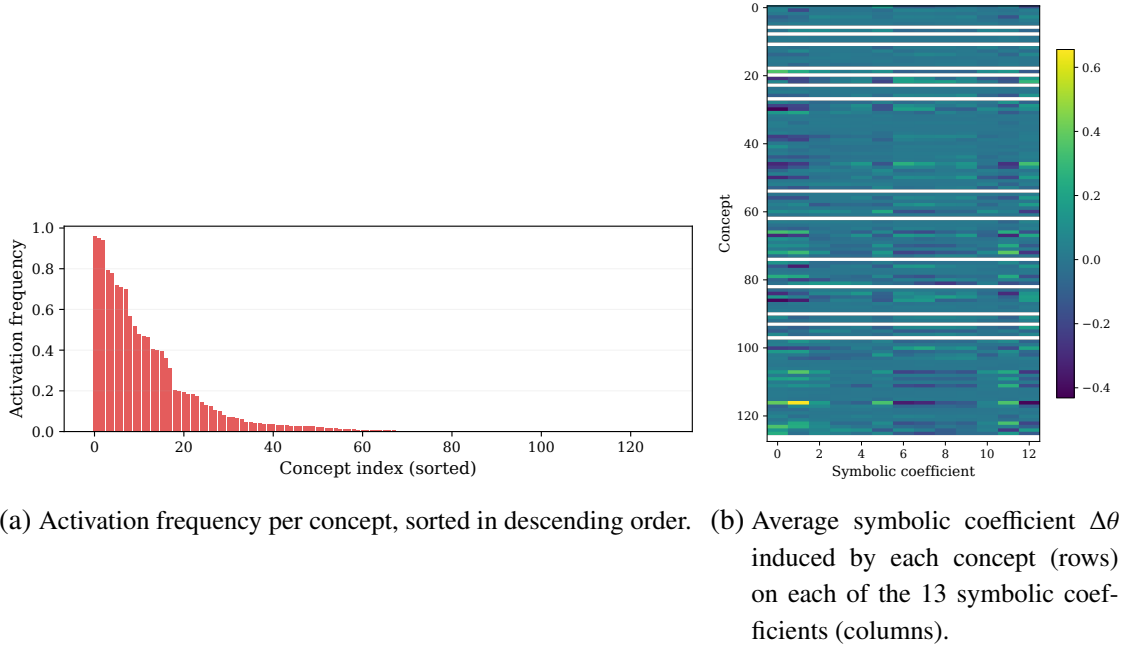


Fig. 11. Structure of the learned concept space at $\text{ctx} = 30$.

APPENDIX B: PROBABILISTIC FORECASTING METRICS

Point forecasting models produce a single predicted value per horizon step: its best guess of what the future will be. A probabilistic forecasting model instead produces a range of plausible futures, expressing not just a prediction but also how confident the model is in it. In practice this is done by predicting a set of quantiles: for example, a lower bound that the true value is expected to exceed 90% of the time, and an upper bound that it is expected to stay below 90% of the time. The gap between those bounds is the prediction interval. A narrow interval means the model is confident; a wide one means it is uncertain. To produce these intervals, the model must be trained with a different objective than the MAE used in this work, namely the pinball loss described below.

Quantile forecasting and the pinball loss.

The most common approach to probabilistic forecasting is to predict a set of quantiles $\tau \in (0, 1)$ directly. For a given quantile level τ , the model produces a threshold $\hat{q}_\tau$ such that the true value y is expected to fall below it a fraction τ of the time. Training is done by minimising the *pinball loss* (also called the quantile loss), defined as:

$$\mathcal{L}_\tau(\hat{q}_\tau, y) = \begin{cases} \tau \cdot (y - \hat{q}_\tau) & \text{if } y \geq \hat{q}_\tau \\ (1 - \tau) \cdot (\hat{q}_\tau - y) & \text{if } y < \hat{q}_\tau \end{cases} \quad (1)$$

The asymmetry is key: for $\tau = 0.9$, under-predicting the true value is penalised nine times more heavily than over-predicting it, which pushes the model to learn a threshold that the true value exceeds only 10% of the time. The median forecast corresponds to $\tau = 0.5$, where over- and under-predictions are penalised equally, recovering a loss equivalent to the MAE. When training over multiple quantiles simultaneously, the total loss is the mean pinball across all τ values.

Coverage.

Given a predicted interval $[\hat{q}_{\tau_l}, \hat{q}_{\tau_u}]$ defined by a lower and upper quantile, the empirical coverage at level $\tau_u - \tau_l$ is the fraction of test samples for which the true value falls within that interval:

$$\text{Coverage}_{(\tau_l, \tau_u)} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{q}_{\tau_l}^{(i)} \leq y^{(i)} \leq \hat{q}_{\tau_u}^{(i)}] \quad (2)$$

A well-calibrated model should achieve empirical coverage close to the nominal level: a 90% prediction interval should contain the true value roughly 90% of the time. System-

atic under-coverage indicates overconfident intervals and over-coverage indicates intervals that are too wide to be useful. The coverage reported in Section 9.2 uses $\tau_l = 0$ and $\tau_u = 0.9$, so the nominal level is 90%.

CRPS.

The Continuous Ranked Probability Score (CRPS) [73], [74] is a proper scoring rule that evaluates the full predictive distribution rather than individual quantiles. It is defined as:

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(z) - \mathbf{1}[z \geq y])^2 dz \quad (3)$$

where F is the predicted cumulative distribution function and y is the observed value. Lower CRPS is better. When only a finite set of quantiles is available, the CRPS can be approximated as a weighted sum of pinball losses across quantile levels, which is the form used by benchmarks such as GIFT-Eval [10]. It rewards both accuracy and calibration simultaneously, making it the standard metric for comparing probabilistic forecasters.

APPENDIX C: DECLARATION ON THE USE OF GENERATIVE AI

See the following page.

DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI) IN BACHELOR THESIS (TFG)

I have used Generative AI in my TFG

Check all that apply:

YES	NO
-----	----

If you have ticked YES, please complete the following 3 parts of this document:

Part 1: Reflection on legal, ethical and responsible behavior

Please be aware that the use of AI carries some risks and may lead to a series of serious academic consequences: the evaluation of your TFG could be compromised if the use of AI involves the use of confidential data, copyrighted materials, or personal data, used without complying with the conditions required in each case (authorization of the interested parties, authorization of the owners, following the instructions of the University).

Question	
1. In my interactions with AI tools, I have provided confidential data, always with the appropriate authorization of the data subjects. Confidentiality encompasses any information that a person or organization wishes to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).	
YES, I have used this data with permission	NO, I have not used confidential data
2. In my interaction with Generative AI tools, I have submitted copyrighted materials with the permission of those concerned.	
YES, I have used these materials with permission from the copyright holders, or without permission because they fall within one of the exceptions or limitations permitted by law: • Public domain work • Licensed work (Creative Commons licenses) • Use of excerpts for research purposes (fair use)	NO, I have not used copyrighted materials

3. In my interaction with Generative AI tools, I have submitted personal data with the consent of the data subjects.	
YES, I have used this data with the authorization of the interested parties and in accordance with the instructions contained in the guide approved by the University .	NO, I have not used personal data
4. My use of the Generative AI tool has respected its terms of use , as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people.	
YES	NO

Part 2: Declaration of technical use

Use the following model statement as many times as necessary, in order to reflect all types of iteration you have had with Generative AI tools. Include one example for each type of use where indicated: *[Add an example]*.

I declare that I have made use of the Generative AI system (Name of AI system/tool and version: ChatGPT, Gemini, Copilot...) **for:**

Documentation and drafting:

- *Supporting reflection in relation to the development of the work: iterative process of analysing alternatives and approaches using AI*

I have not used AI for this task.

- *Revision or rewriting of previously drafted paragraphs*

I have used Writefull (Overleaf) to check grammar errors at the end of my thesis as Claude to rewrite limited sentences for clarity and comprehension. The output of this tools was NOT used directly in the main text but reinterpreted by me for rewriting.

- *Search for information or answers to specific questions*

I have used Perplexity for information search but always contrasting its outputs with the real sources.

Develop specific content

Generative AI has been used as a support tool for the development of the specific content of the TFG, including:

- *Assistance in the development of lines of code (programming)*

I have JUST use AI in code for standardizing the format and palette of Python plots using matplotlib and seaborn.

- *Generation of diagrams, images, audios or videos*

I have not used AI for this task.

- *Optimisation processes*

*I used AI for rewriting my personal notes on the meetings with my tutor. This outputs were NOT used for the thesis itself, but for me to understand them better later.
I used AI (CHATGPT) for creating table structures in Latex as I was not that used to it. All the data included in the table was introduced manually.*

- *Data processing: collection, analysis, cross-checking of data...*

I used not used AI

- *Inspiration of ideas in the creative process*

I used AI for ideas in how to structure the diagram. AI has never created a graph for me.

Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

Generative AI is a useful tool that saves considerable time on routine and well-defined tasks. It also excels in delivering specific concepts explanations. However, it shows clear limitations when it comes to innovation and conciseness.